

NIA BCI

NIA for Processing & Training - NIA for Real Time Control



Final Project Report

Realisation of an Integrated Digital Environment for Human-Machine Interaction in Brain-Computer Interfaces

Design and Development of a Complete BCI Pipeline from Signal Processing to
Real-Time Motor Imagery Control

Giuliano Agostini
Sep 2025 – Sep 2026

*Towards my dream
of understanding what makes us human
our Nervous Systems*

Abstract

Brain-Computer Interfaces are among the most complex and fascinating technologies achieved by human beings. The study and research behind this particular neurotechnology represent one of the principals and most crucial themes in the field of neuroscience globally. The importance underlying BCIs lies in their direct relationship with the part of the human organism that makes us human: the Central Nervous System. The methods of interaction between these two parts take different forms, each characterising a specific sub-field within this developing technological domain. The possibility of modifying and adapting systems according to the particular characteristics of the patient is the key aspect of non-invasive and adaptable BCIs, which aim to restore specific functions of the human organism affected by neuropathology or by the consequences of critical neurotrauma. In order to provide this type of patient with a new method of communication and movement, non-invasive BCIs employ external instrumentation capable of interacting with specific biological parameters at the level of the CNS, so as to transduce them into something meaningful for a new communication or movement pathway. This project situates itself within this specific branch of BCIs, thereby aiming to create a solid foundation upon which the interaction between human and machine, such as digital devices of various natures, can become stable and practicable. Creating a solid foundation means building a bridge between the hardware component and the software tools, in order to establish a proper environment usable by both the experimenter and the end user to achieve a complete and versatile BCI system. The development process involves the use of open-source instrumentation, adapted to the specific needs and objectives of the present environment. The second and most crucial section of the project then consisted in defining the actual mechanism through which the studied hardware components could establish a starting point of communication, in order to transmit the neural features sampled at the level of specific regions of the CNS. The final outcome was therefore the creation of the NIA environment, in which the fundamental principles of BCIs are established and verified through several stages of verification, checking, optimisation and experimentation, so as to ensure an environment in which human-machine interaction in the field of BCIs can become real, practicable and adaptable.

TABLE OF CONTENTS

1.	<u>Introduction: Project Overview</u>	9
1.1	<u>Current State of the Art</u>	9
1.2	<u>Project Description</u>	12
1.3	<u>Research Aims and Objectives</u>	13
2.	<u>Background: Clinical and Technical Aspects</u>	15
2.1	<u>Medical and Neurological Foundations</u>	16
2.1.1	<u>The Primary Motor Cortex</u>	16
2.1.2	<u>Motor Control and Motor Imagery</u>	18
2.2	<u>Technological and Engineering Foundations</u>	20
2.2.1	<u>Principles of Electroencephalography</u>	20
2.2.2	<u>Bioelectronic Aspects</u>	24
2.3	<u>Biomedical Instrumentation and Software Environment</u>	26
2.3.1	<u>Instrumentation and Methods of use</u>	27
2.3.2	<u>Software and Digital Features</u>	31
3.	<u>First Part: NIA for Processing and User Training</u>	35
3.1	<u>Development and Creation of the Application</u>	36
3.1.1	<u>Front-end and User Interface</u>	36
3.1.2	<u>Back-end and Software Functionalities</u>	42
3.2	<u>Conclusion and Consolidation of the Application</u>	50
3.2.1	<u>Debugging and Optimisation</u>	51
3.2.2	<u>Exportation of the Finished Application</u>	55
3.3	<u>Utilisation and Experimental Procedure</u>	57
3.3.1	<u>Approach and Methods of use</u>	57
3.3.2	<u>Results and Considerations from the Trials</u>	71
4.	<u>Second Part: NIA for Real Time Control</u>	77
4.1	<u>Target Definition and Starting Phase</u>	78
4.1.1	<u>Tools and Purposes</u>	78
4.1.2	<u>Creation of the Environment</u>	80
4.2	<u>Functioning Flow and Verification of the Application</u>	82
4.2.1	<u>Relations and Connection Between Elements</u>	82
4.2.2	<u>Application Pipeline Founding</u>	90

4.3	Final Experimental Procedure		92
4.3.1	Individual Methods of Use		93
4.3.2	Results from the Collective Environment		96
5.	Conclusion: Achievements and Developments		99
5.1	Clinical and Technical Outcomes		99
5.2	Future Improvements		.101
6.	References and Figures		103



INTRODUCTION

1. Project Overview

The present document aims to describe and explain the entire development of the project, outlining both its *theoretical* background and *practical* implementation. The work is divided into several sections, each dedicated to a specific aspect of the project and designed to progressively guide the reader toward a comprehensive understanding of the final objectives achieved. Each chapter begins with a brief introductory section intended to provide an overview of the topics discussed and to contextualize the arguments presented throughout the document. Furthermore, every chapter is organized into multiple sub-sections that examine specific topics in greater detail, with the purpose of highlighting the fundamental concepts and methodologies on which the project is based. More specifically, this introductory chapter aims to provide a general overview of the project in order to familiarize the reader with its main objectives and overall structure. This preliminary explanation is essential, as it establishes the theoretical and practical foundations necessary to understand the procedures, methodologies and results discussed in the following chapters. The first section focuses on the *Current State of the Art* related to the technologies and devices involved in the project. In particular, attention will be given to recent developments, current applications and future perspectives pursued by the scientific community. This contextualization is important in order to position the project within the broader scientific and technological landscape, while also allowing comparisons with other devices belonging to the same field of application. Following this overview, the next section presents the project itself, describing the motivations that led to its development as well as the main objectives it aims to achieve. In addition, a preliminary introduction to the *instrumentation* and *methodologies* employed throughout the project will be provided, while more detailed analyses will be addressed in the dedicated chapters. Finally, the introductory chapter concludes with the presentation of the research aims, objectives, and expected outcomes. This will provide the basis for a final comparison between the initial expectations and the actual results obtained during the development of the project.

1.1 Current State of the Art

The concept of interaction between humans and digital devices has a long history, characterized by several stages, each involving different types of neurotechnology and methodologies. The first interest in the field of BCIs can be traced back more than a century ago, and even today these technologies remain at the centre of the scientific community's attention. Their revolutionary nature

represents a major step forward for human knowledge, and their progress has become extraordinary. Nowadays, it is possible to combine BCIs with other types of neurotechnology, enabling the extension of human brain capabilities, especially for patients affected by specific neuropathology, spinal cord injuries, or, more generally, trauma to the nervous system.

First of all, in order to highlight the history behind BCIs, it is important to underline that their development required several decades, and their origins can be localized into the second half of the 19th century. More specifically, this period corresponds to the rise of *EEG technology* and the growing understanding of the *electrical nature* of the CNS. In fact, during these years, the scientific community performed the first attempts at applying electrical stimuli to the nervous system through devices such as galvanometers. These experiments made it possible to understand how the CNS, and in general the tissues related to the nervous system, behaved in response to external electrical stimulation, with the aim of identifying the relationship between the observed phenomena and their direct causes. Towards the beginning of the 20th century, the increasing interest in this field of neuroscience enabled researchers to better understand how to exploit this electrical aspect of the nervous system. For instance, it is possible to mention *Vladimir Vladimirovich* and his investigations of action potentials within the animal brain, as well as *Hans Berger* and his studies related to the wave patterns generated by the CNS, together with the first introduction of the widely used 10–20 system for EEG measurements. This latter aspect is crucial, since it represented one of the first steps towards the development of non-invasive BCIs, where EEG technology is predominantly used.

Furthermore, with the rise of computers, the interest in studying how and why *specific waveforms* were generated became increasingly significant. In these years, the neocortex of the CNS became central to scientific research, since it was believed to represent the main source of the potentials associated with the different waveforms achievable through EEG technology. Subsequently, several types of electrical potentials and frequencies were investigated in non-invasive BCI experiments, including slow cortical potentials, direct current EEG shifts, event-related potentials and sensorimotor rhythms. More specifically, considering the target of the project presented within this document, event-related potentials are generated when the brain responds to stimuli or events. These events may be sensory, motor, or cognitive in nature, and they provide a convenient method for studying mental processes in a non-invasive way. BCI-related studies significantly advanced thanks to the research conducted by *L. A. Farwell* and *E. Donchin*, in which letters of the alphabet were displayed on a screen while the brain activity of subjects was recorded. Furthermore, the rapid development of *computer science*, together with the emergence of algorithms and advanced data-processing techniques, enabled increasingly rapid progress in the field of non-invasive EEG-based BCIs at the

beginning of the 21st century.^[1] Nowadays, the evolution of non-invasive BCIs has reached an extraordinary level, both from a *technological* and a *clinical* point of view. In fact, these systems possess exceptional clinical value due to their non-surgical nature, which guarantees greater safety, reliability and comfort for patients. Modern EEG-based systems completely eliminate many of the risks associated with invasive procedures, especially in fields such as paediatric and geriatric care, as well as in short-term interventions. Considering recent years, between 2023 and 2024 BCIs achieved important breakthroughs across three main domains: therapeutic management of linguistic and motor deficits, mental navigation research and emerging technological development. More specifically, regarding *motor rehabilitation*, non-invasive systems are capable of inducing neural reorganization in patients affected by spinal cord injuries through closed-loop cortical modulation. Both the hardware and software aspects have undergone significant advancements, making it possible to combine neurotechnology such as implantable neural probes (NPs), multi-electrode arrays (MEAs), electrocorticography (ECoG), EEG, functional near-infrared spectroscopy (fNIRS), functional magnetic resonance imaging (fMRI), and functional ultrasound (fUS).^[2] Focusing more specifically on EEG technology, and considering the purpose of this project, EEG-based systems represent also a promising alternative to implantable brain-controlled robotic systems, particularly in the field of upper-extremity robotic device control. In fact, impairments in upper-extremity functionality, especially affecting the hand, are among the most common consequences of stroke, with deficits observed in nearly half of all stroke patients. These impairments significantly compromise daily activities requiring precise and individual finger control, which conventional rehabilitation tools and gross motor BCIs often fail to address adequately. This limitation highlights the urgent need for *BCI-driven software* capable of decoding neural activity and restoring basic communication with the external environment.^[3] However, in order to achieve complete integration between humans and digital devices, not only the hardware aspect must be developed. A crucial role is also played by the software side, together with all the tools capable of facilitating communication between external medical instrumentation and the devices receiving the commands. These signal-processing and data-elaboration procedures are often extremely complex and difficult to achieve through simple neural signal sampling alone. Therefore, one of the fastest-growing concepts of the modern era, the so-called *Artificial Intelligence* (AI), plays a fundamental role in this context. In fact, many human activities are goal-oriented, with movements directed towards target locations such as search bars, buttons and icons on a computer, or physical objects such as cups, chips, door handles, keys or blocks. Once a goal is identified, human actions often become stereotyped and can therefore be assisted through an AI copilot. In this way, an AI copilot can help avoid universally incorrect or unlikely actions, thereby improving the overall performance of a non-invasive BCI system.^[4]

In conclusion, nowadays BCIs have reached a stage in which this technology is capable of combining different fields of *medicine* and *engineering* in order to achieve the common goal of enhancing human interaction with digital devices. The progress discussed above is particularly important for neurotraumatic and neuropathological patients, who often present impairments not only in their interaction with digital devices, but also in communication with other individuals. The devices involved in non-invasive BCIs are now capable of guaranteeing a high level of communication and integration, while simultaneously preserving patient safety and comfort during the use.

1.2 Project Description

The project presented throughout this document focuses on the development of a *non-invasive* and *adaptive BCI system*. The main aspects involved in its realization concern both the hardware instrumentation, mainly restricted to a specific anatomical area of the scalp, and the digital processing stage required to acquire, interpret and transform neural activity into executable commands for an external device.

From a general perspective, the instrumentation adopted in this project follows the fundamental structure commonly used in non-invasive BCI systems. More specifically, the signal acquisition and recording components remain external to the signal source, while dedicated elements placed in contact with the patient are responsible for transducing biological activity into electrical signals. These components are represented by the *electrodes*^[5] and the processing board. Together, these elements constitute the first stage of the system, in which communication occurs through a wired connection. The specific components selected for this stage will be described in detail in the following chapters. However, in order to provide a broad overview of the system, they can generally be described as an *EEG interface* capable of acquiring neural activity through the electrodes.^[6] Subsequently, the recorded signals are transmitted through wired connection to the processing board, where the first stage of signal elaboration is performed before transmission to the final device. This second communication stage is instead based on Bluetooth technology, allowing the system to become more efficient, portable and versatile. In particular, the connection between the processing board and the target device is achieved through a dedicated *USB dongle* connected to the selected device.

The final and most relevant part of the project is represented by the external device itself, which may consist of a computer or a portable device such as a smartphone or tablet. This stage represents the core of the signal processing and analysis pipeline. Through dedicated signal processing software,

which will be discussed in the corresponding chapters, the system will first undergo a training phase, followed by the implementation of methods aimed at effectively transforming neural activity into executable commands. Ultimately, the external instrumentation applied to the patient and the digital signal processing stage will operate together in order to achieve the objectives and purposes that will be presented in the following chapter.

1.3 Research Aims and Objectives

The project introduced in the previous chapter, and that will be described in greater detail throughout this document, has specific aims and objectives to achieve. In particular, the goals of the project can be divided into two closely connected sections. Therefore, this chapter aims to present them clearly, in order to explain the motivations and the design choices behind every aspect of the system that will be discussed during the development of the project.

First of all, the primary objective of the project concerns its practical and technological implementation. The aim is to develop a fully functional BCI system strictly related to the clinical pathologies and anatomical characteristics of the patients for whom it is intended. The completed system must be as *versatile*, *adaptable* and *comfortable* as possible for the users. The key aspect of the project is its adaptive nature, since the integration between the hardware and software components is designed to adjust to the specific needs and conditions of each patient. Regarding the hardware section, all biomedical components will be adaptable and modifiable according to the different situations and requirements faced by the system. Although the concept of the BCI system will remain unchanged as the core of the project, aspects such as positioning, materials and instrumentation may vary and evolve from an initial prototypal solution into a more consolidated and optimized device composed of adaptable components. For this reason, the selected instrumentation will mainly rely on *open-source* technologies, allowing greater freedom for modifications, upgrades and optimization processes during the early stages of development. Once the first development phases are consolidated, the future improvements and extensions, which will be discussed in the final sections of this document, will aim to bring the BCI system to a more advanced and reliable level, potentially transforming it into a daily assistive device or an experimental support system for patients. Concerning the software section, the objectives are also strongly related to the versatility and adaptability of the system. An initial step will involve selecting the most suitable tools and environments for the development of the software components of the BCI system. The software will mainly focus on two principal tasks. The first one concerns the *training* and *analysis* of neural signals,

since obtaining functional and reliable commands requires understanding the nature of the neural sources and identifying methods to enhance their effectiveness. Furthermore, the training sessions will play a crucial role in making the patient familiar and comfortable with the mechanisms on which the BCI system is based. As will be explained in the dedicated sections, the command generation process will rely on specific mental activities that require a high level of concentration. The combination of signal analysis and patient training will represent the foundation of the *processing* and *device-control* phase. In this stage, additional software environments will be employed to convert the neural signals, previously studied and trained, into effective commands capable of allowing the patient to communicate directly with the device. Moreover, once the prototypal phase is completed, the final system will include a dedicated digital environment in which the analysis, training and operational functionalities will be integrated into a single platform. This environment will not be restricted to a single device, since it will be designed to support installation and use across multiple systems and platforms.

On the other hand, the project also pursues objectives directly related to the patients who will use the completed BCI system. In particular, the system is intended to support patients affected by *neurological pathologies* or *neurotraumatic conditions*, whose motor and communication abilities are compromised as a consequence of disease or trauma, while their neural activity remains sufficiently functional to be detected, interpreted and used by the developed BCI system. Therefore, the final goal of the project is to provide a new method of *interaction* and *communication*, both with external devices and, most importantly, with other people.

The objectives described above outline a multidisciplinary environment in which biomedical instrumentation, medical knowledge and computer engineering are integrated together with the common purpose of improving the functionality, comfort, adaptability and overall *well-being* of the patients.

BACKGROUND

2. Clinical and Technical Aspects

The proposed project is based on a strongly multidisciplinary framework, as it combines concepts and methodologies originating from different scientific fields. More specifically, *BCI* systems lie at the intersection between the medical sciences and biomedical engineering. By integrating knowledge from these domains, it is possible to develop systems capable of establishing a direct interface between the *human body* and *technological devices*. Therefore, this chapter aims to provide a broad yet comprehensive overview of the theoretical foundations on which the project is based. Each discipline will be examined in detail to highlight the fundamental concepts required for the development of the proposed BCI system.

The chapter begins with an introduction to the neurological foundations underlying the project. In particular, the focus will be placed on the regions of the human body that constitute the object of study. The developed system interfaces with specific components of the *CNS*, as the acquisition and interpretation of neural signals are strictly related to physiological phenomena occurring within well-defined anatomical structures, which will be discussed in the following sections.

The engineering and technological foundations are then introduced. Particular attention will be devoted to the principles underlying the operation of the devices employed throughout the project. Since the proposed BCI system is primarily based on *EEG* technology, it is essential to describe the key concepts of this well-established neurotechnology and their relevance to the functionality of the system. Furthermore, the fundamental principles of bioelectronics will be presented, as the interface relies on biological processes directly involved in the generation and propagation of bioelectrical signals.

Finally, the chapter concludes with an overview of the instrumentation and software environment adopted for the project. This section introduces the practical aspects of the work by describing the biomedical instrumentation employed and the reason behind its selection. In addition, it provides a general overview of the hardware and software components that constitute the experimental framework, whose characteristics and methods of use will be discussed in the subsequent sections.

2.1 Medical and Neurological Foundations

This section introduces the medical and neurological foundations of the proposed project. Its purpose is to provide an overview of the anatomical structures of the CNS involved in the study, together with the physiological phenomena that constitute the basis of the developed BCI system. Particular attention is devoted to the Primary Motor Cortex, which represents the main area of interest throughout the project. Consequently, the following chapters will describe its anatomical organization, the mechanisms underlying the generation of neural signals, and the cognitive and physiological processes relevant to the proposed application.

2.1.1 The Primary Motor Cortex

The aim of this project is to interface with a specific region of the central nervous system: the *Primary Motor Cortex* (M1). The decision to focus on this region stems from its close relationship with the neural phenomena underlying the developed BCI system, which will be discussed in the following chapters.

Motor control represents one of the most complex functions of the nervous system, as it involves numerous anatomical structures operating in a coordinated manner. Although a large proportion of motor activities is performed automatically through the interaction of different neural circuits^[7], several key structures can be identified as playing fundamental roles in the planning, execution, and modulation of movement. Among these, the *M1* plays a central role, as it directly contributes to the voluntary control of motor units through the pyramidal system^[7]. In addition, the *basal ganglia*, through the extrapyramidal pathways, and the *cerebellum*, through the cerebellar efferent pathways, are both essential for the regulation and refinement of movement^[7]. However, since the present work focuses on the development of a BCI based on M1 activity, the following analysis will primarily concentrate on this region. From an anatomical perspective, the fibres of the pyramidal system originate predominantly from *Brodmann area 4*, located along the precentral gyrus and extending to the paracentral lobule^[8]. Within this region, a well-defined *somatotopic organization* is present, in which different body districts are represented according to an orderly arrangement. The superior portion of the gyrus corresponds to the lower limbs, the medial region represents the trunk and upper limbs, whereas the inferior portion is dedicated to the head and facial muscles^[8]. This organization is particularly relevant to the developed system, as electrode placement and the subsequent acquisition of electroencephalographic signals are specifically targeted at cortical areas associated with the movement of selected body districts, thereby allowing the analysis to focus on the neural activity that is most relevant for system control.



Brodmann Area 4 lateral (left) and medial (right) visualisation: 3D brain data is from Anatomography.[\[E1\]](#)

Beyond its role in movement execution, the M1 exhibits an extensive network of afferent and efferent connections with numerous cortical and subcortical regions. It projects to the *lower motor neurons* of the *spinal cord* and is functionally connected with areas such as the *Supplementary Motor Area* (SMA), the *Premotor Area* (PMA), the *parietal cortex*, the *dorsolateral prefrontal cortex*, the *basal ganglia*, and the *cerebellum*. These connections have been extensively documented in both primate studies and human neuroimaging investigations^[9]. The interaction among these structures makes M1 an integral component of motor learning processes, extending its involvement beyond the mere execution of movement. Furthermore, numerous electrophysiological and neuroimaging studies have demonstrated that the M1 also participates in the post-execution phases of movement and in the consolidation of motor skills. In particular, its neuronal activity has been observed to persist for several hundred milliseconds after movement execution, suggesting its direct involvement in the early stages of motor consolidation^[9]. Likewise, during the acquisition of new motor skills, brain activity progressively shifts from the *prefrontal regions* toward areas such as the *premotor cortex*, the *posterior parietal cortex*, the *cerebellum*, and the *M1* itself, further confirming the central role of the latter in the acquisition and consolidation of motor abilities^[9].

Considering its fundamental role in voluntary motor control, its anatomical organization, and its extensive connections with the principal regions involved in motor learning, the M1 represents the primary target for the development of the BCI system presented in this work. Understanding the functioning of this region and the neural phenomena that characterize it therefore constitutes the essential foundation for defining the methods used to acquire and interpret the brain signals employed to control the interface. The following chapter will therefore examine the mechanisms underlying both physical and mental motor control, which constitute the physiological basis of the developed system.

2.1.2 Motor Control and Motor Imagery

The neural phenomenon on which the present project is primarily based directly derives from the activity of the *Primary Motor Cortex* (M1). Motor control represents a highly complex set of processes that are not limited to the physical execution of movement, but also include cognitive and representational mechanisms. In this context, the key element for controlling the developed BCI system is *Motor Imagery*, namely the mental representation of a movement of a specific body segment, associated with specific patterns of brain activation, particularly within M1.

As discussed in the previous section regarding motor learning, prolonged practice of motor skills induces structural and functional changes within the nervous system. In particular, several studies have shown that mental practice alone, without the actual execution of the motor task, can produce significant improvements in motor performance^[9]. This indicates that *physical* and *mental* motor activity are not separate phenomena, but are instead closely interconnected and potentially exploitable in a combined manner. It has also been demonstrated that the combination of physical and mental training leads to greater performance improvements compared to either approach alone^[9]. In parallel, behavioural and neuroimaging studies have shown that *motor imagery* shares mental representations and neural mechanisms with *motor execution*^[9]. It is precisely within this overlap that a fundamental aspect of the developed system lies. In fact, the crucial aspect is the possibility that brain activity associated with motor imagination, that generates neural patterns similar to those of actual movement execution. These patterns can therefore be exploited as the basis for generating the commands of the BCI system.

Further exploring the relationship between imagination and motor execution, several studies have shown that the cortical areas activated during motor imagery largely overlap with those involved in motor planning and execution. In particular, activation is predominant in the medial *Supplementary Motor Area* (SMA), the *Premotor Cortex* (PMA), the *dorsolateral prefrontal cortex*, and the *posterior parietal cortex*^[9]. With specific regard to *M1*, its involvement in motor imagery is supported by experimental evidence obtained through Transcranial Magnetic Stimulation (TMS). An increase in the amplitude of motor evoked potentials (MEPs) has been observed during motor imagery tasks, indicating a *modulation of M1 cortical excitability* even in the absence of actual movement^[9]. Furthermore, studies in which participants were asked to mentally rotate images of hands and feet while single-pulse TMS was applied at different time points have shown that this modulation varies over time, suggesting the involvement of M1 in relatively late stages of the motor imagery process^[9]. Additional evidence indicates that *motor imagery* and *motor observation* can induce changes in the cortical representation of M1, increasing the area and volume of cortical maps associated with

primary and synergistic muscles. Moreover, studies on epileptic patients, based on intracortical electrophysiological recordings, have confirmed that the spatiotemporal distribution of cortical activity during actual movement largely overlaps with that observed during the *imagination* of the same movement^[9]. Overall, these findings indicate that M1, in addition to its central role in motor execution, also plays a relevant role in motor cognition^[9].

To fully understand the phenomenon of motor imagery, it must be considered not only from an anatomical perspective, but also from an electrophysiological one, in order to identify the origin of the resulting neural signal. *Neural signal decoding* is indeed one of the main challenges in applied neuroscience, and in the context of BCIs it is essential to understand the physiological mechanisms underlying signal generation itself. To interpret a subject's motor intention, it is useful to analyse the *neurophysiology of motor control*, in order to identify where the signal can be intercepted and which characteristics it may exhibit. In addition to what was mentioned before, The PPC integrates information from the somatosensory cortex, providing a representation of the current state of the body^[10]. The combination of sensory information, prior experience, and decision-making processes enables the definition of the sequence of actions to be performed. *Movement planning* is subsequently assigned to the PMA and SMA, both located in *Brodmann area 6*^[10]. Once the motor sequence has been planned, the activation of motor programs is regulated by the *basal ganglia*, which, through the direct pathway, facilitate the initiation of the selected movement, and through the indirect pathway inhibit inappropriate motor programs. Finally, *M1* is responsible for the execution of the motor command at the peripheral level. Pyramidal neurons in layer V of M1 project to the spinal cord via the corticospinal tract, contributing to the activation of spinal motor neurons and to the modulation of antagonist muscles through inhibitory interneurons, enabling the generation of coordinated movements involving multiple muscle groups^[10]. Although this framework allows a general understanding of motor control organization, several aspects remain not fully clarified, particularly regarding the cortical representation of movement. There is a long-standing debate on which term M1 uses to encode motor control^[11]. In this context, it has been proposed that motor control can be described within an *intrinsic parameter* space, which includes variables related to inter-physical and inter-anatomical sides of the human body^[11].

In conclusion, motor imagery represents the primary mechanism on which the developed system is based. The ability of the central nervous system to generate neural patterns associated with motor imagination constitutes the basis for generating commands directed to the device. Although these processes are intrinsic to the central nervous system, they must be interpreted by considering the complex interaction between the various cortical and subcortical structures involved in motor control.

In addition, beyond neurophysiological aspects, it will be essential to analyse *signal acquisition methods* and the *instrumentation* used to interface with the nervous system, topics that will be addressed in the following chapters.

2.2 Technological and Engineering Foundations

The physical composition and assembly of the system conceptually reflect an adaptation of modern technologies employed for the study of the electrical activity of the CNS. The technological and engineering framework underlying the developed system represents a fundamental aspect that deserves careful consideration in order to fully understand the technical principles that enable its operation. To this end, this chapter presents the main theoretical foundations that guided the design process. In particular, it focuses on the interface between the physical BCI system and the human body, with special emphasis on its neuro-electrophysiological nature.

First, a theoretical overview of the principles of electroencephalography will be provided. The developed system is in fact based on a substantially adapted reconstruction of a bio-signal acquisition system for the CNS. Therefore, the main mechanisms underlying the project presented in this work will be discussed.

Subsequently, since one of the primary subjects of investigation is the generation of bioelectrical signals, these phenomena will be described in detail in order to explain the principles underlying the interface between the BCI system and the human body.

Overall, this section aims to provide the reader with the theoretical background, necessary to understand the rationale behind the selection of the physical and digital instrumentation adopted for the implementation of the BCI system. These aspects will be discussed in detail in the following chapters of this document.

2.2.1 Principles of Electroencephalography

As introduced within the present macro-section, the developed system essentially represents an adaptation of a widely employed technology for the acquisition of electrical signals at the CNS level: *electroencephalography*.

Electroencephalography (EEG) is an electrophysiological investigation technique developed during the first half of the 20th century, whose main instrument is known as the *electroencephalograph*. The purpose of this methodology is the analysis of electrical activity associated with specific anatomical

regions of the CNS.^[13] Considering exclusively its *non-invasive* configuration, which is the one most consistent with the system developed in the present work, it is based on the use of dedicated instrumentation capable of amplifying spontaneous *electrical potential differences* measured between two points on the scalp surface.^[12] The signal acquired by the electroencephalograph primarily originates from the activity of *pyramidal cells* located within the upper layers of the cerebral cortex.^[13] From a historical perspective, despite its *high temporal resolution*, which is still unmatched by many of the previously mentioned neuroimaging techniques, EEG has progressively lost part of its predominant role in routine clinical practice.^[13] Nevertheless, it continues to represent a fundamental tool for the investigation of the mechanisms underlying the human encephalic nervous system, both in *healthy* individuals and in patients affected by *neurological disorders*.^[12] In particular, this methodology has proven essential for the assessment of epilepsy, sleep disorders, real-time monitoring of brain activity during surgical procedures, and the certification of brain death status.^[13]

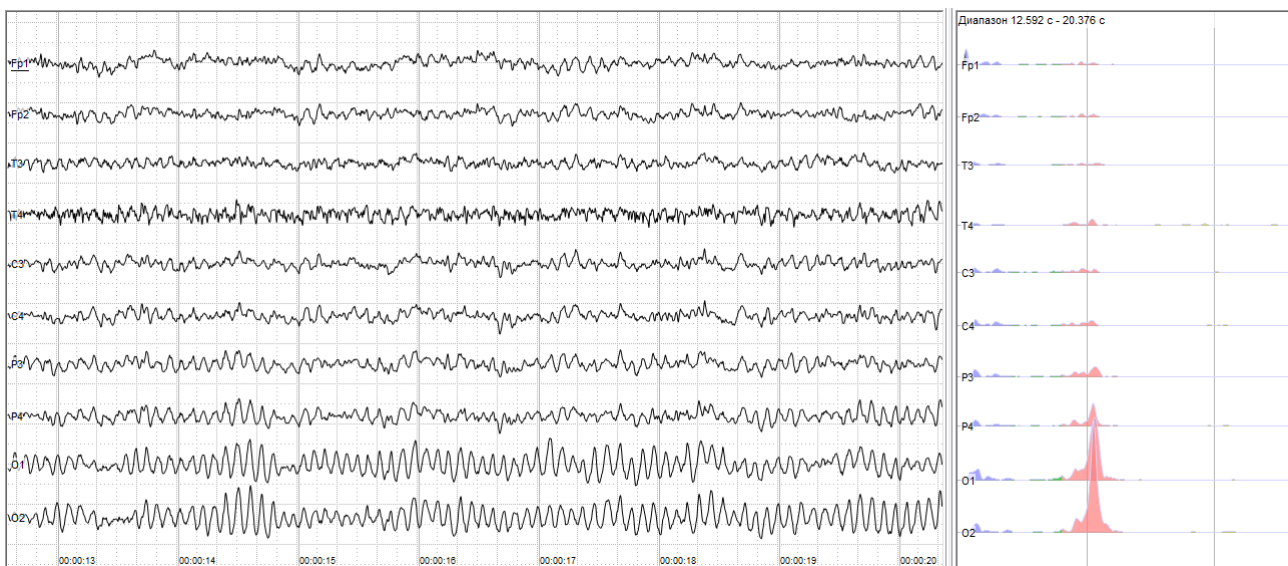
From a structural perspective, EEG systems can be characterized by different configurations for bio-signal acquisition. In the most modern versions and in advanced experimental BCI applications, such systems can reach configurations comprising from 256 up to 600 channels, with the aim of reducing spatial aliasing phenomena.^[14] However, simpler systems are also available, characterized by lower power consumption, high portability, and a reduced number of channels, while still maintaining the objective of maximizing clinical accuracy.^[14] Indeed, current clinical diagnosis based on digital EEG is generally performed through the use of 16 channels. Although the actual need for extremely high-precision systems for every patient and diagnostic condition remains a subject of debate, it is evident that future *wearable EEG* solutions will need to be designed to adapt to continuous technological developments and emerging application requirements.^[14]



Example of a modern EEG System Application: Prague conference EEG fet09.^[F2]

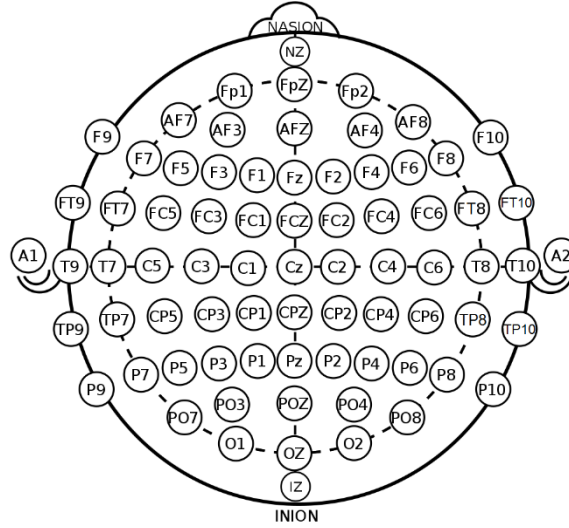
Regarding signal transmission and communication between the acquisition system and the data processing system, the connection can be implemented either through a *wired connection* or through *wireless communication*, depending on the structural characteristics that need to be optimized. Among these two alternatives, however, the latter, despite potentially appearing as the most modern and natural solution, is not frequently adopted in medical and strictly clinical contexts.^[14] Conversely, the wired connection between acquisition electrodes and the signal collection and processing system still maintains a predominant role in the development of EEG systems. Nevertheless, in applications beyond the exclusively clinical context, where advanced BCI configurations are considered, technological evolution will progressively lead to the replacement of this methodology, promoting greater device *miniaturization* through the implementation of wireless communication.^[14]

Originally, data acquired by EEG systems were represented through dedicated paper-based graphs known as *electroencephalograms*. Their subsequent evolution into current digital versions continues to represent variations in electrical potential differences between two points through a Cartesian coordinate system, specifically reporting time on the x-axis and voltage on the y-axis. Brain potentials can be obtained through a *bipolar* method, namely by considering the potential difference between two electrodes positioned on the scalp surface, or through a *monopolar* method, in which the measurement is performed between an active electrode and a reference electrode positioned on the earlobe or in other regions considered electrically inactive.^[12]



Example of a Digital Electroencephalogram: The sample of human EEG with prominent resting state activity - alpha-rhythm. Left - EEG traces (horizontal - time in seconds; vertical - amplitudes, scale 100uV). Right - power spectra of shown signals (vertical lines - 10 and 20Hz, scale is linear). Alpha-rhythm consists of sinusoidal-like waves with frequencies in 8-12Hz range (11Hz in this case) more prominent in posterior sites. Alpha range is red at power spectrum graph.^[13]

Regarding the components of the system that are directly in contact with the user, these are represented by the *electrodes*. These elements, commonly referred to as *applied parts*, are designed to interface directly with the user's scalp in a more or less invasive manner depending on the specific application. The main variables characterizing these components concern the *material* used for electrode fabrication and the *positioning* of the electrode system on the scalp. Considering the first aspect, electrodes can be distinguished according to the different materials and technologies used in their construction. Among the most commonly employed for EEG signal acquisition are *Ag/AgCl* electrodes.^[15] These electrodes may be used in combination with a conductive gel, which enhances electrical conduction at the electrode-skin interface. Instead, in the absence of such a gel, they are classified as *dry electrodes*, which constitute the primary focus of the present project. This type of electrode provides direct contact between the electrode and the scalp, allowing faster application and greater flexibility in positioning. On the other hand, it may exhibit higher interface impedance and lower robustness against power-line interference.^[15] With regard to electrode positioning, internationally standardized criteria define the arrangement of electrodes on the scalp according to the intended application. These criteria are described by the *International 10-20 System*, which is based on well-defined anatomical landmarks and anthropometric measurements, including the *nasion*, the *inion*, and the *preauricular points*.^[15] The name of this system derives from the two possible distances between adjacent electrodes, corresponding to 10% or 20% of the *nasion-inion* distance or the distance between the *preauricular points*. Each electrode position is identified by a combination of letters (Fp, F, C, P, O, and T), indicating the corresponding anatomical region, and numbers.^[15] Regarding the latter, even numbers identify electrodes located over the right hemisphere, whereas odd numbers identify those positioned over the left hemisphere. Electrode positions closer to the midline are assigned lower numbers, while progressively more lateral positions are identified by higher numbers.^[15] Depending on the application for which the system is designed, the density of electrodes distributed across the scalp may be either reduced or increased. In the present project, particular attention will be devoted to electrodes positioned over the *Primary Motor Cortex*, in both the right and left hemispheres. Consequently, signal acquisition will primarily focus on electrodes located over the *frontal* regions (F/FC) and along the *central sulcus* (C), with particular emphasis on positions close to the midline (z, 1/2, 3/4, and 5/6). The rationale behind these design choices will, however, be discussed in detail in the following chapters.



Positioning of the electrodes following the International Standard 10-20 System. [\[F4\]](#)

Specifically, the acquisition and characteristics of the EEG signal analysed in the present project are determined by specific acquisition patterns and temporal evolution, which depend on neuro-electrophysiological phenomena capable of modifying the electrical potential according to precise mechanisms. These aspects will be analysed and further discussed in the following subsection, with the aim of describing the bioelectrical characteristics that the EEG system acquires and processes.

2.2.2 Bioelectronic Aspects

The neural signal has a well-defined origin and is governed by *biochemical* and *bioelectrical* processes occurring at the cellular level. Therefore, the generation of the EEG signal is rooted in mechanisms that do not simply depend on the electrode–scalp interface, but rather originate from the electrophysiological dynamics of individual neurons. In this subsection, the main mechanisms underlying the generation of the neural signal will be examined, highlighting their direct relationship with the patterns observed in EEG recordings. Subsequently, the electroencephalographic phenomena of particular relevance to the operation of the developed system will be discussed.

The cells responsible for the generation and propagation of electrical signals within the nervous system are *neurons*, specialized excitable cells characterized by their ability to rapidly modify their membrane permeability to specific ionic species following the application of a stimulus. [\[16\]](#) Among the principal ionic species involved in this process are *sodium* (Na^+) and *potassium* (K^+). The permeability changes induced by the stimulus give rise to membrane depolarization, thereby initiating the *Action Potential* (AP). The generation of this phenomenon is strictly dependent on the voltage- and time-dependent properties of sodium and potassium channels. Following the application of a stimulus, the membrane permeability to the previously mentioned ions changes and, once the excitation threshold is reached, sodium permeability rapidly increases to values several orders of

magnitude greater than those of the other ionic species.^[16] As a consequence, the membrane potential approaches the sodium Nernst potential, corresponding to approximately +60 mV.^[16] After approximately 1 ms, however, the time-dependent properties of sodium permeability become predominant, leading to a progressive reduction in Na⁺ permeability. This phase is followed by an increase in potassium permeability, which accelerates membrane repolarization and produces a membrane potential temporarily lower than the resting membrane potential, a phenomenon known as hyperpolarization, before finally returning to resting conditions.^[16]

Focusing on the specific nature of neurons, the primary contribution to EEG signal generation originates *from pyramidal cells*, mainly due to their high density within the cerebral cortex.^[17] The synchronous activity of postsynaptic potentials, resulting from the simultaneous convergence of multiple synaptic inputs, generates a sufficient number of parallel current dipoles.^[17] Consequently, the activity of current sources within the neocortex can be represented as a layer of electrical dipoles whose overall macroscopic effect forms the basis of EEG signal detection and the subsequent recording of the electroencephalogram.^[17] The *electroencephalogram* therefore represents the primary tool for interpreting the neuroelectric phenomena underlying the operation of the developed system. From a descriptive perspective, the EEG signal is mainly characterized by three parameters: *amplitude*, *frequency*, and *morphology*. Regarding amplitude, EEG signals generally exhibit values in the order of a few *microvolts*.^[18] Therefore, the developed system must be capable of acquiring and analysing signals characterized by extremely low amplitudes, typically within a few tens of microvolts. With regard to frequency, specific frequency bands can be identified, each associated with different patterns of neural activity:^[18]

- *Delta* (δ) band: approximately 0.5–4 Hz.
- *Theta* (θ) band: approximately 4–8 Hz.
- *Alpha* (α) band: approximately 8–14 Hz.
- *Beta* (β) band: approximately 14–40 Hz.
- *Gamma* (γ) band: above approximately 40 Hz.

Considering the morphology of the EEG signal, different types of waveforms can be distinguished according to their shape and, in many cases, their duration.^[18] These include:

- *Sinusoidal* waves, characterized by a sinusoidal shape;
- *Sharp* waves, characterized by an approximately triangular morphology and a duration ranging from 80 to 200 milliseconds. Sharp waves located at the vertex are generally associated with arousal stimuli;

- *Spikes*, characterized by a duration shorter than 80 ms. They may be either positive, when they develop below the isoelectric baseline, negative, when they develop above, or biphasic when both components are present.

Having described the principal variables characterizing the EEG signal, attention can now be focused on the aspects that constitute the basis of signal transduction and, consequently, of the command generation system adopted in the present project. The developed BCI system is based on the analysis of a specific EEG frequency band belonging to the Alpha band, known as the μ *rhythm* (mu rhythm). This rhythm can generally be observed before and during both actual and imagined movement execution. Unlike the classical occipital Alpha rhythm, the cortical μ rhythm is characterized by a lateral distribution and is of particular interest in BCI applications because it can be modulated through motor imagery and monitored using non-invasive techniques.^[19] More specifically, the phenomenon of greatest interest within this frequency band is the *Event-Related Desynchronization* (ERD), a specific neural response that occurs during motor imagery and motor control tasks.^[19] Increased neuronal excitability within thalamocortical circuits produces a desynchronization of the EEG signal characterized by a reduction in its amplitude. Consequently, ERD can be physiologically associated with the activation of different cortical areas involved in sensory and cognitive information processing, as well as in the generation of voluntary movement.^[19] For example, it has been observed that, in modern BCI systems, the explicit learning of a motor sequence is accompanied by an enhancement of ERD over the contralateral central regions.^[19]

In conclusion, starting from the theoretical foundations of neural signal generation, it has been possible to introduce the principal EEG signal characteristics that constitute the basis of the developed BCI system. The identification and interpretation of these neurophysiological phenomena will provide the input to the hardware–software framework described in the following chapters, which are dedicated to the analysis of the components constituting the proposed system.

2.3 Biomedical Instrumentation and Software Environment

The project involved the use of different types of both physical and digital instrumentation. Specifically, dedicated hardware–software combinations were adopted to enable communication between the user of the BCI system and the device being controlled. This chapter therefore presents the rationale behind the selection of the instrumentation employed during the data acquisition phase, as well as the digital environments used for the processing and analysis of neural signals.

First, the physical components constituting the hardware section of the system will be described. Particular attention will be devoted both to the elements placed in direct contact with the user and to the devices responsible for communication with the digital system. The main hardware components will therefore be presented, together with the motivations behind their selection and the technical specifications required to understand their operation.

Finally, the software environments employed throughout the development of the project will be presented. They will be described according to the specific processing stage in which they are used, highlighting the main functionalities that enabled effective communication between the BCI system and the user.

2.3.1 Instrumentation and Methods of Use

Within this subsection, the instrumentation employed throughout the different stages of the project development and experimental activities will be described. Firstly, we will present the individual components of the system, the functional relationships among them, and the motivations underlying the selection of each element. Subsequently, the assembly procedure of the BCI system will be illustrated by describing the adopted methodology and the reasons that led to the definition of the final configuration. Finally, the chapter will conclude with an initial overview of the experimental methodologies adopted during the validation and testing phases of the developed system.

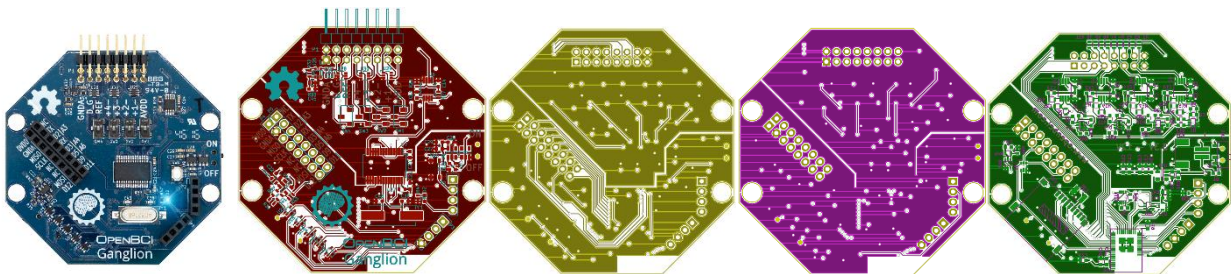
The selection of the components was primarily oriented toward the use of *Open-Source* hardware and software, with the aim of developing a BCI system that is versatile, functional, and easily adaptable according to the requirements that emerged throughout its development. The main components constituting the system are described below.

- *OpenBCI Ganglion Board (4-channels)*: the core of the BCI system is represented by the Ganglion Board developed by *OpenBCI*. This component serves as the signal acquisition board, to which the electrodes applied to the user's scalp are connected through the dedicated pins. The board is powered by a dedicated battery, and its operation is initiated through a dedicated ON/OFF switch. For safety reasons, it is housed inside a transparent plastic enclosure equipped with a dedicated hook, allowing it to be securely attached or carried during the experimental sessions. Furthermore, the Ganglion Board is responsible for the wireless transmission of the acquired signals, sending them to the processing device through a dedicated USB dongle, where the subsequent signal processing and experimental procedures are performed. More specifically, the Ganglion PCB is a board designed using KiCad, an

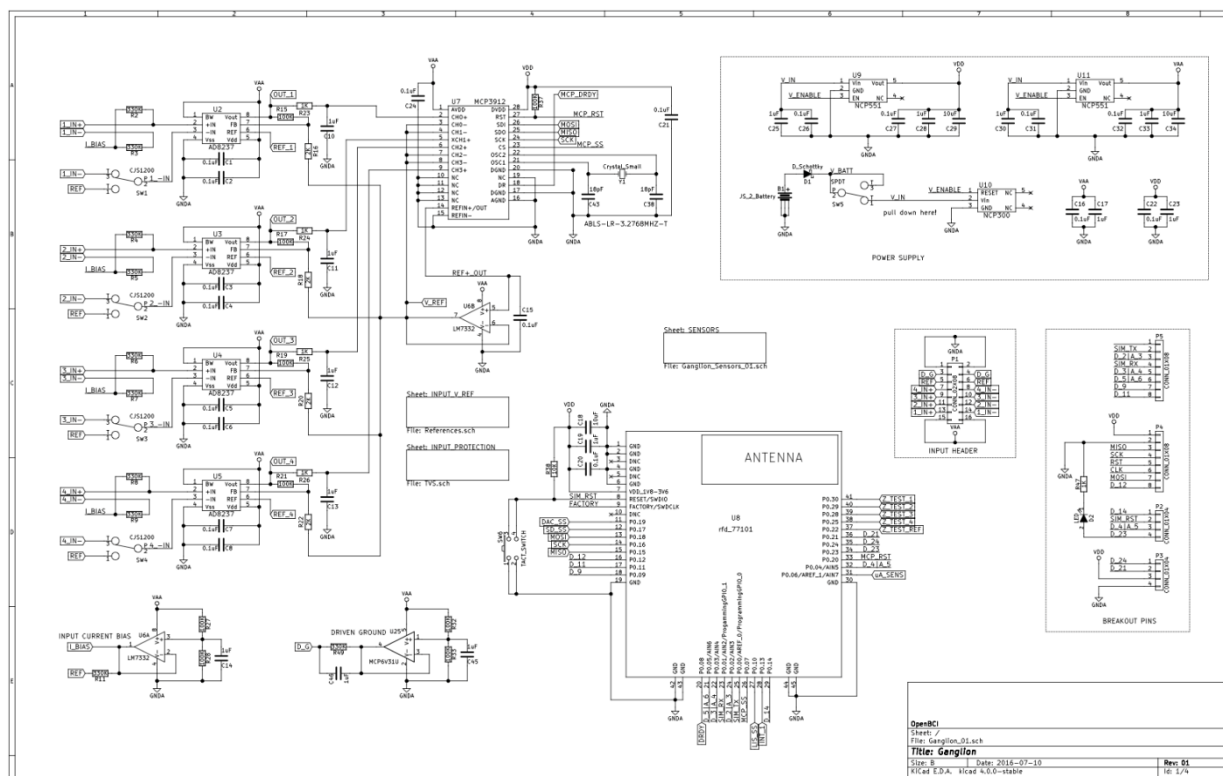
open-source software package for printed circuit board design and layout.^[20] Structurally, it consists of four channels and is equipped with sixteen Right Angle Header pins, to which electrodes and sensors can be connected. The input pins are connected to an AD8237 instrumentation amplifier (InAmp) buffer featuring differential inputs. The positive input is referred to as non-inverting (+), whereas the negative input is referred to as inverting (-).^[20] Among the available pins, the following are primarily employed for the development of the present project:

1. D_G = Driven Ground: 1.5 Volts. Connects to both the top and bottom pin.^[20]
2. REF = Combined Reference input. Connects to the top and bottom pin.^[20]
3. $+I-$ = Channel input #1. + input on top, - input on bottom.^[20]
4. $+2-$ = Channel input #2. + input on top, - input on bottom.^[20]
5. $+3-$ = Channel input #3. + input on top, - input on bottom.^[20]
6. $+4-$ = Channel input #4. + input on top, - input on bottom.^[20]

More specifically, since the objective of the project is the recording of neural activity through EEG, the negative inputs (input on bottom, -) of the individual channels are primarily used. This configuration is complemented by an additional board setting, namely the Inverting Input Select Switches, which consist of four small switches allowing the negative inputs of the InAmps to be connected either to their corresponding negative pin or to the REF pin.^[20] The default configuration of the board corresponds to the UP position. However, in the present project these switches are configured in the DOWN position, as this configuration enables the reference to be shared across two or more channels during EEG acquisition.^[20] Indeed, it allows one or more negative inputs to be connected simultaneously to the common reference pin, making it particularly suitable for electroencephalographic applications. To provide a comprehensive description of the Ganglion Board, representative images of the device, its PCB Layers, and its circuit schematic will be presented in the following pages.



From left to right: OpenBCI Ganglion Board, Top layer, Inner layer 1, Inner layer 2 and Bottom layer.^[F5]



From the top to the bottom and from left to right: Ganglion Board Circuit Schematic. [\[F6\]](#)

- *Electrodes*: the applied parts of the system are represented by the electrodes, which are non-invasively positioned on the user's scalp. In this case as well, components compatible with an open-source philosophy were selected to ensure full compatibility with the previously described Ganglion Board.^[21] The characteristics of the electrodes, both in terms of materials and application methodology, are intended to provide a non-invasive system that is adaptable to different anatomical characteristics and suitable for a wide range of applications, while allowing prolonged use and easy adaptation to the specific needs of the user. To achieve these objectives, dry electrodes were selected, thus avoiding the use of conductive gels or pastes that could cause discomfort during use. Specifically, EEG Polymer Dry Electrodes with Ag/AgCl tips were employed.^[22] Moreover, talking about the REF and D_G electrodes, we considered the use of Earclip Electrode with Ag/AgCl surface.^[23] The electrodes are positioned on the scalp according to the International 10-20 System, in the regions surrounding the Primary Motor Cortex, by means of a dedicated EEG cap. In the present project, signal acquisition is limited to electrode positions located along the central sulcus and to a small portion of the anterior frontal region, including, depending on the user's dominant hand, the following locations: Cz, C1/C2, C3/C4, and FC3/FC4. This configuration, combined with the selected electrodes, enables the acquisition of neural signals in temporal proximity

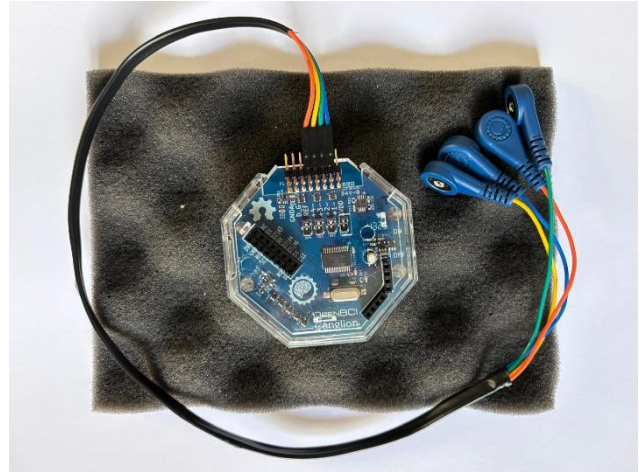
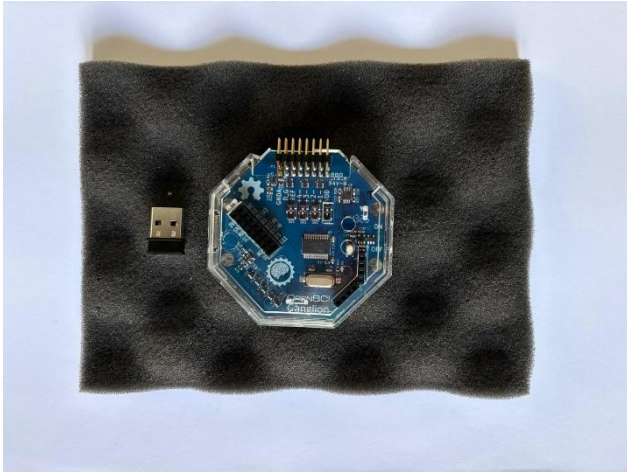
to motor imagery tasks, which are subsequently transmitted to the Ganglion Board through the dedicated wiring.^[24] To provide a clearer representation of the adopted components, images of the selected electrodes are presented below.



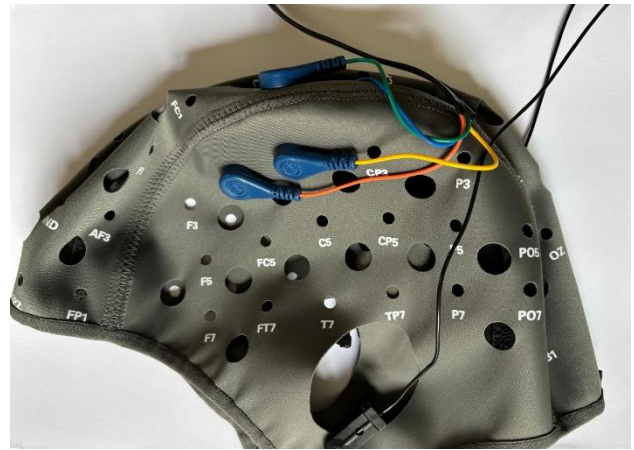
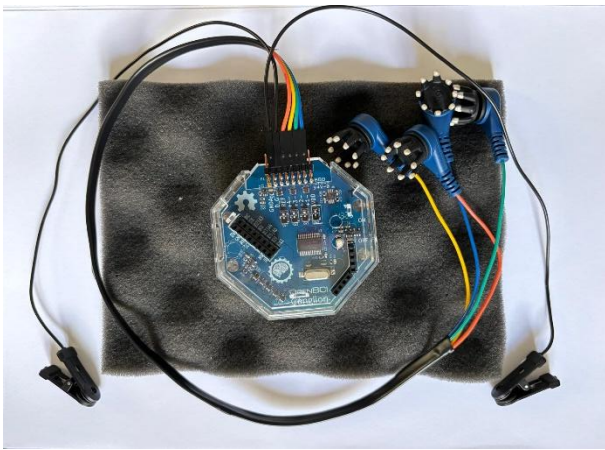
From left to right: EEG Polymer Electrode, Earclip Electrode and EMG-ECG Snap Cables.^{[E7] [F8] [F9]}

- *EEG Cap*: the correct positioning of the electrodes is ensured by a dedicated electroencephalography cap. The selection of this component is not restricted to a specific commercial product, but can be adapted according to the economic and practical requirements of the project. In the present work, the essential characteristics required for the EEG cap include an electrode positioning layout compliant with the International 10-20 System, a 64-channel configuration, in order to include intermediate positions such as FC3 and FC4, and a fabric thickness on the order of one millimetre, thereby ensuring proper contact between the electrodes, the connecting cables, and the Ganglion Board.
- *Processing and Control Device*: considering the objectives of the project, it is necessary to define the device on which the control operations driven by the user's neural signals are performed. For this purpose, a laptop computer was employed both for the design and development of the software environments described in the following chapters and as the target device receiving the commands generated by the BCI system. Consequently, the same device was used throughout both the development and the experimental phases, serving as the final endpoint of the interaction between the user and the BCI system.

In conclusion, after describing the individual hardware components employed in the implementation of the system, following this paragraph, there will be representative images of the fully assembled BCI system together with the device used during the development and experimental activities, providing an overall view of the configuration adopted in the present project.



Images of the assembled hardware components. From left to right: OpenBCI Ganglion Board inside the protective PVC case next to the relative USB connector, OpenBCI Ganglion Board connected to the snap cables.[\[F10\]](#) [\[F11\]](#)



Images of the assembled hardware components. From left to right: basic assembled EEG recording system with relative electrodes, final configuration of applied system assembled to the EEG Cap around the Cz/C1/C3/FC3 positions.[\[F12\]](#) [\[F13\]](#)

2.3.2 Software and Digital Features

The following part of the project involved the development of a *digital environment* through which the hardware system composed of the previously described components could be utilized. The development of this section represented the most time-consuming phase of the project, since achieving the predefined objectives related to device control through neural signals required the implementation of a stable and clearly optimized software framework. To meet this requirement, two parallel software development processes were undertaken, each aimed at creating a specific digital component of the overall system.

The first component of this phase was developed through the *OpenBCI GUI*, which was specifically designed by the hardware manufacturers for the utilization of their devices.[\[25\]](#) The latter environment represents the official tool provided by OpenBCI for the visualization, recording, and streaming of data acquired through the boards developed by the company. This software enables real-time data

visualization, playback of previously acquired recordings, data storage on the computer in *CSV* or *BDF+* formats, and data streaming towards third-party software platforms.^[25] Furthermore, it can be launched either as a standalone application or as a sketch within the Processing environment, a Java-based programming language.^[25] This software was mainly employed during the initial measurement sessions performed using the first system prototypes, during the preliminary stages of the project, with the aim of identifying the most suitable system configuration to be adopted for subsequent official acquisitions. Moreover, it was used during the first acquisition sessions performed with the completed BCI system, for calibration and system initialization purposes, in order to verify that the acquired signal quality was suitable for the predefined objectives. More specifically, the implemented functionalities included signal visualization across the four acquisition channels through *Time Series*, analysis of amplitude distribution as a function of frequency through *FFT Plot*, and representation of the distribution and power of the main frequency bands (δ , θ , α , β , and γ) through the *Band Power* function.

Subsequently, the previously described software was used in combination with an additional environment dedicated to data analysis and processing, namely MATLAB. This platform enables mathematical operations based on matrices and arrays to be directly implemented.^[26] Moreover, the possibility of using executable notebooks, in which code, results, and formatted text are integrated within the same environment, allowed the software development process and signal analysis procedures to be documented in a structured manner.^[26] After an initial experimental phase conducted using the standard version of the software, the development process was transferred to the platform used for the creation of the first standalone application of the project: *MATLAB App Designer*. This environment enabled the development of the first application dedicated to signal analysis, processing, and initial training of the neural signals acquired through the hardware components. Specifically, App Designer allows the creation of professional applications through a graphical environment, where the graphical user interface (GUI) can be designed by arranging visualization components and subsequently defining their associated behaviour.^[27] Within this standalone environment developed through App Designer, an additional toolbox belonging to the MATLAB ecosystem was integrated. This toolbox was appropriately adapted to the OpenBCI open-source environment and optimized for the processing of data acquired from the corresponding boards. The selected toolbox is *EEGLAB*, an interactive MATLAB environment dedicated to the processing of EEG, MEG, and other electrophysiological signals through methodologies such as Independent Component Analysis (ICA), time-frequency analysis, and artifact rejection techniques.^[28] The integration of this toolbox within the application developed through App Designer enabled the creation of a complete digital environment dedicated to EEG signal processing, manipulation, and visualization, providing an

effective tool for understanding the functioning of the acquisition system and analysing the collected signals.

Subsequently, during a second development phase, the implementation of the second software environment was undertaken, conceived as an integrated and complementary component to the previously described platform. This second part resulted in the creation of a standalone application dedicated to additional signal *visualization functionalities*, *active training* procedures for system calibration, and the implementation of the *cursor control* function. This component of the digital environment was developed through *BrainFlow*, a library designed for the acquisition, processing, and analysis of EEG, EMG, ECG signals, and other types of biosensor-derived data.^[29] The library also provides a unified API for data acquisition from all supported boards, allowing the employed hardware device to be replaced without requiring modifications to the developed code and making BrainFlow-based applications independent from the specific board being used.^[29] Furthermore, it provides advanced signal processing tools that can also be employed without a BCI headset, while maintaining the same APIs across the different supported programming languages.^[29] In this case, the development of all the files constituting the second application was carried out through the *Visual Studio Code* (VSC) environment, a free and open-source code editor available for Windows, macOS, and Linux operating systems.^[30] More specifically, *Python 3.0* was adopted as the programming language, due to its versatility and effectiveness in software development, machine learning applications, and integration with the BrainFlow and OpenBCI software environments.

In conclusion, the software tools described in this subsection enabled the development of a digital environment composed of two standalone applications, fully integrated with each other and characterized by *complementary functionalities*, with the aim of achieving a comprehensive analysis of the EEG signal and consequently optimizing the commands delivered to the target device.

FIRST PART

3. NIA for Processing and User Training

After having provided an extensive presentation of the theoretical and practical background in the first part of this dissertation, this section marks the beginning of the description and explanation of the components actually developed and built *ex novo*. Through the digital tools previously mentioned, the work involved defining the desired behaviour of the applications to be developed, followed by the design of the operating pipeline and of the interaction modalities with the hardware components and with the files dedicated to the application's operation. The digital environment took shape precisely with the creation of this first application, which, given the functionalities illustrated below, will be referred to as *NIA – Processing and Training* (short for *Non-Invasive and Adaptive BCI for Processing and User Training*). This represents the first piece of software making up the overall environment, together with the second application, described in the following section of this dissertation. Focusing on the first one, this application was developed using *MATLAB App Designer* and is essentially intended to provide the following functions:

- Displaying on screen the *main information* regarding the subject under investigation, along with some details on the measurement trial carried out for neural signal acquisition. This information is appropriately stored through the creation of dedicated Excel (.xlsx) and text (.txt) files.
- Displaying on screen the *raw data* collected during previous measurement trials via the main OpenBCI GUI, made possible by the integration between the developed software environment and the structure of the files output by the OpenBCI GUI.
- Allowing *processing* and manipulation actions on the EEG signal collected and displayed through the functionality described above. This was achieved thanks to the integration between the software environment developed from scratch and the EEGLAB toolbox, whose functionalities are of crucial importance for correct EEG signal processing.
- Launching an actual *training session* for the BCI system user, aimed at familiarizing the subject with the command-interpretation methods that will characterize the second developed application. This was made possible by creating an interface that, with the support of the application's developer and the on-screen instructions, allows the user to practice in preparation for future measurement trials.

The first step was approaching the software development environment, within which the operating criteria and the organization of the commands at the interface level to be displayed on screen were defined. Once these preliminary steps had been established, work proceeded to writing the code for the respective commands included in the application's GUI. This was followed by the *optimization* and *debugging* phases of the code written earlier, with the aim of strengthening the application and making it more robust when facing the different subjects who would undergo EEG signal measurements. Finally, once the application's stability within the MATLAB environment had been confirmed, it was exported as a standalone version for subsequent use.

Once the development of the application was complete, its actual official use began, preceded by measurement trials carried out via the OpenBCI GUI, conducted following a *standardized methodology*, with the aim of preparing the files to be provided as input to the application. All three implemented functionalities were then employed: EEG signal visualization, its processing, and user training.

In conclusion, considerations were drawn regarding the *overall performance* obtained during the use of the application, in order to assess whether the EEG signal manipulation and interpretation actions had been successful. Furthermore, through the repetition and combination of training and measurement sessions, considerations were drawn on the effectiveness of the methodology studied and implemented to support the user.

3.1 Development and Creation of the Application

The initial phase in the development of the current application involved approaching the MATLAB App Designer platform, with the aim of understanding which functionalities would be useful for achieving the established objectives. Therefore, the next series of sub-chapters will present to the reader the methodologies employed in preparing the first part of the software environment, illustrating the starting foundations, the app building methodologies, and the programming tools adopted. The related explanations will provide the knowledge necessary to understand the *front-end* and *back-end* mechanisms that determine the application's operation. The presented notions are the foundation for the actual experimental use described in the following chapters.

3.1.1 Front-end and User Interface

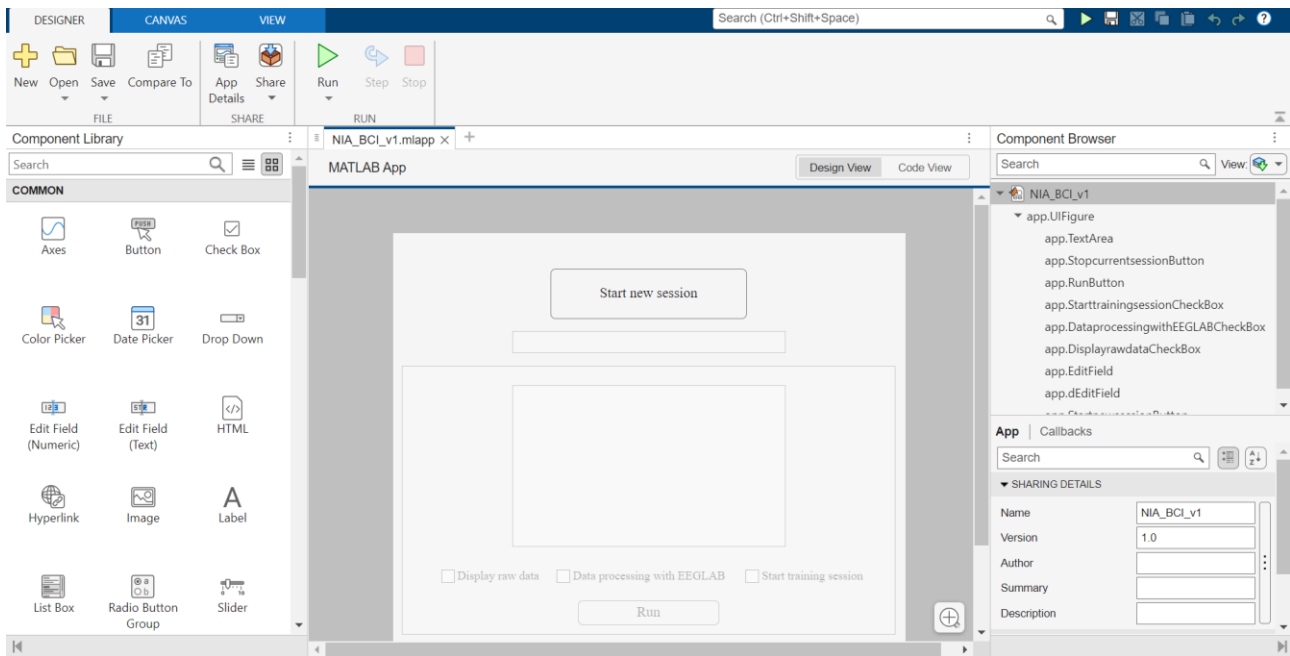
The digital environment through which the user, together with the experimenter, can access the application's functionalities is represented by the *front-end interface*, where the selectable commands were inserted during the programming phase, along with the corresponding accessibility logic in view

of experimental use. The preliminary step was therefore to access *App Designer* to create the various MATLAB files required for the interfaces to work. Together, these provide the "*physical*" commands that the user will actually select in order to access the code sections designed and written during the subsequent *back-end* programming phases. This phase is divided into specific parts, recurring for each file created for the purpose of building the GUI with which the user will interact.

The first step was therefore to launch App Designer, with the aim of creating the application's main control GUI, which would be named *NIA_BCI_v1.mlapp*. Through the MATLAB interface, the *APPS* menu was selected, followed by the *Design App* command. This command opened App Designer, through which, using the Blank App command, the creation of the application's first *.mlapp* file was initiated. This last command opened the project, displaying the various app-building functionalities provided by the platform. Among these, of primary importance is the empty interface displayed on screen within the *Design View*, since all commands were placed onto it through the *Component Library* sections. This section in fact contains the various components, such as buttons, Cartesian axes, text fields, selectable boxes, images, labels, sliders, check boxes, and so on. Among these, the ones needed to compose the visual layout of the GUI were selected and dragged onto the empty interface. In our case, the following components were used: buttons, Text Area, Edit Field, and check boxes. These were placed in specific positions with the aim of creating a natural usage flow that could easily guide the user. The usage flow proceeds from top to bottom, and the purpose of each inserted component is as follows:

- *Start New Session Button*: the first button for starting the actual application usage session is placed at the top of the GUI and provides access to all subsequent commands, as well as to the selection of the files from which the GUI update phases begin, displaying subject information and initiating the signal visualization and processing stages.
- *Subject and Trial Info Field*: after the start button, an auto-populated text field was inserted, which, based on the file selected in the previous step, displays information regarding the current subject number and the trial number being analysed.
- *Background Edit Field*: for aesthetic purposes, and also to direct the user's attention to the parts relevant to the software's use, a background Edit Field was inserted, non-accessible and non-editable, enclosing the subsequent commands within its boundaries.
- *General Info Text Area*: a text area inserted to display, based on the file selected during the session initialization phase, further general information about the subject under investigation (age, sex, any pathologies, etc.), the model and version of the hardware system used, and additional information regarding the methodologies employed during the experimentation.

- *Check boxes for the application of the three functionalities*: once the usage session has started and the previous text sections have been updated, the three aforementioned functionalities become available to the user. These are accessible through check boxes and must be selected individually according to the user's purpose, in order to allow the operating flow to proceed.
- *Run button for the selected functionality*: the Run Button initiates the functionality selected through the previous check boxes. This button therefore allows the subsequent GUIs dedicated to the individual commands, shown below, to open.
- *Stop Current Session Button*: to end the current usage session and thus allow a new session to begin, if desired, the end-session button is selected; once pressed, it resets the application's state, making the start-session button available again.



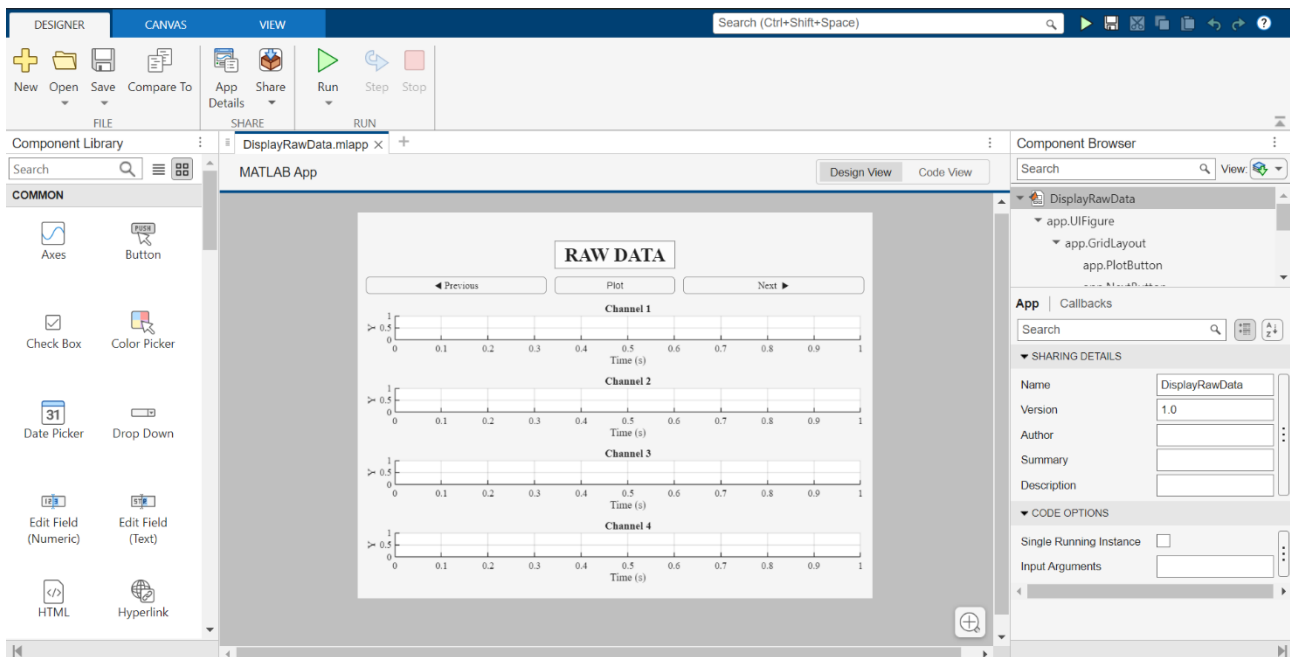
Design window used during the design of the user interface: the image shows the various sections of the Design View in MATLAB App Designer for the creation of the main GUI of *NIA_BCI_v1.mlapp*. On the left is the Component Library, in the centre there is the final GUI composed of the various inserted components, and on the right is the Component Browser section, through which accessibility and editing details of the commands inserted into the GUI can be accessed.^[F14]

The *NIA_BCI_v1* GUI represents the starting point through which the application's specific commands can be accessed. For each command, accessible through the check boxes, a dedicated GUI was created, with its own front-end and accessibility features.

The first functionality to be described is *Display Raw Data*, for which the *DisplayRawData.mlapp* GUI was created. Opening this file provides access to the command GUI, which can be managed using the same App Designer sections already discussed during the design of the main GUI.

In this case, the use of the *Component Library* and *Component Browser* was adapted to the specific context, giving rise to the following sections accessible to the user:

- *Edit Field for the GUI title*: at the top of the interface, a non-editable text field was inserted, containing the label RAW DATA, representing the explanatory title of the GUI in use.
- *Plot buttons*: the first available command allows the user to generate plots of the raw data imported during session initialization in the main GUI. Using this button provides access to the four plots.
- *Previous/Next buttons*: given the temporal length of the experimental trials, and in order to allow for a correct visualization of the signal, only a limited section of the data can be displayed on the plots at any one time. To move from one section to another, after selecting the plot command, forward and backward buttons are available to scroll through the time interval, allowing the collected data to be analysed in full.
- *Cartesian axes for signal visualization*: finally, after selecting the plot command, the four figures containing the plots of the signal collected at the corresponding EEG channel, specified in the plot's title, are displayed on screen.



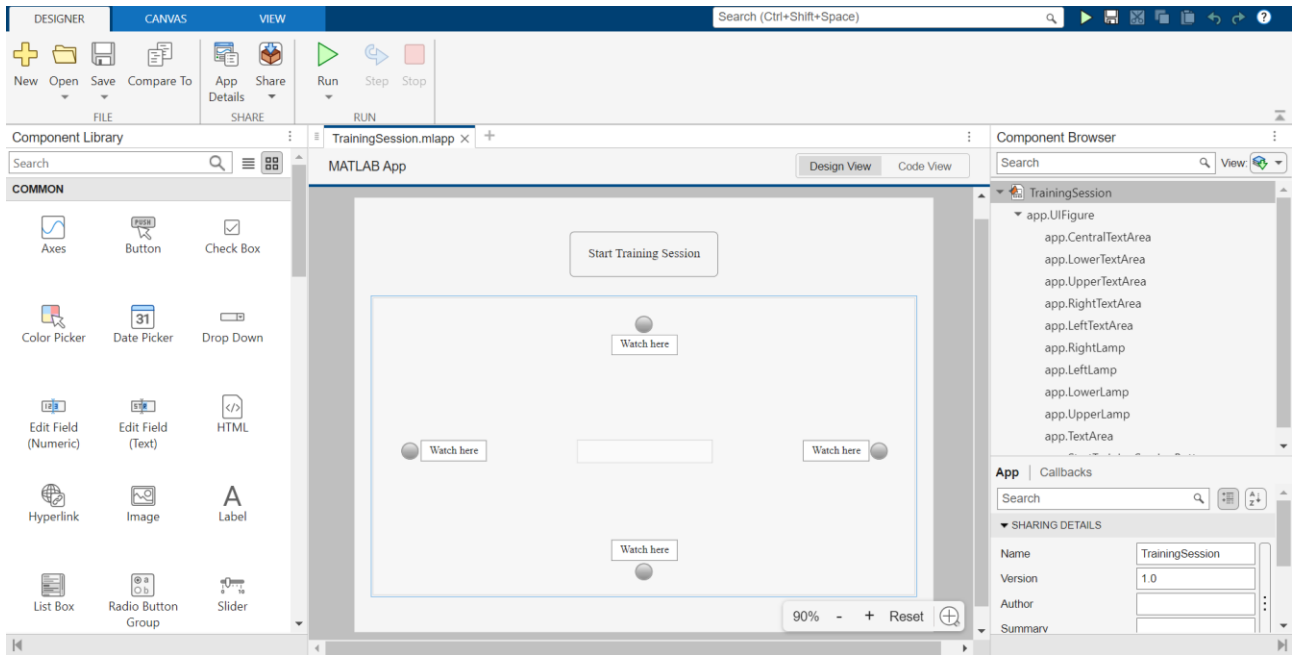
Design window used during the design of the user interface: the image shows the various sections of the Design View in MATLAB App Designer for the creation of the *DisplayRawData.mlapp* GUI. On the left is the Component Library, in the centre there is the final GUI composed of the various inserted components, and on the right is the Component Browser section, through which accessibility and editing details of the commands inserted into the GUI can be accessed. [\[F15\]](#)

The next step was integrating the second functionality selectable from the main GUI: processing through the *EEGLAB* toolbox. In this case, the task involved integrating an already existing toolbox, independent from the environment developed. Most of the integration work was carried out during

the programming and code-writing phase, described in the following chapters. Nevertheless, its accessibility and the opening of its corresponding GUI follow the same procedure, after selecting and confirming the relevant command among the check boxes. Once this is done, the main EEGLAB window is displayed on screen, allowing the experimenter to proceed with the desired processing and filtering operations.

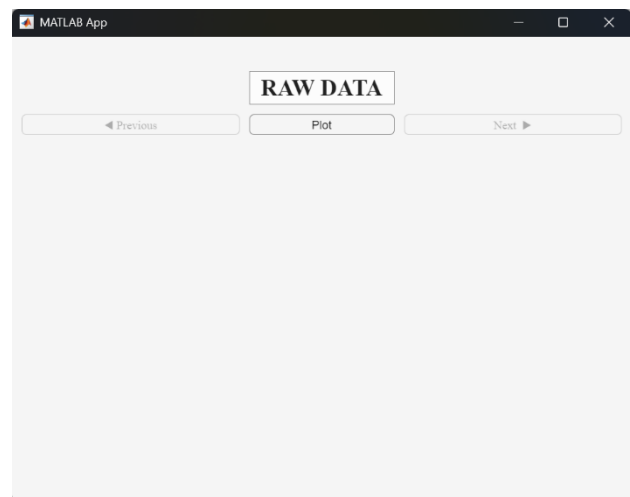
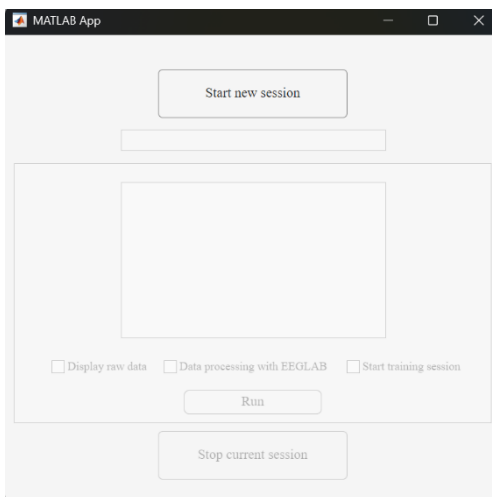
Finally, the last integration followed the same logical flow applied for displaying the raw data. In this case as well, after selecting and confirming the relevant check box, a specific GUI opens, developed within the *TrainingSession.mlapp* file. The initial screen of this implementation is essentially divided into two sections, in turn organized into the following commands:

- *Start Training Session*: a button intended to initialize the user's training process. Once pressed, it becomes no longer accessible and allows the flow to proceed through the section below, the main area in which the training will take place.
- *Background Text Area*: in this case as well, a non-accessible and non-editable Text Area was inserted for aesthetic purposes, with the aim of drawing the user's attention to the relevant section.
- *Central Text Area*: at the centre of the GUI, a dedicated Text Area was inserted whose content, changing throughout the session, provides the user with precise indications, such as the countdown of the ongoing training section, or the start/end of the session itself.
- *Guide LEDs*: the positions the user must follow in order to carry out the training correctly are indicated through dedicated LED components that, when needed, light up red to signal the direction to imagine. Their control is time-based, and, through specific time intervals defined during code writing, the LEDs alternate accordingly.
- *Guide Text Area*: to accompany the guide LEDs, dedicated text areas were inserted to guide the user toward the specific part of the training session on which to focus visually, simultaneously indicated by the guide LED. Here too, the different text areas alternate according to specific time intervals defined during code writing.
- *Guide sounds*: although not physically or visually present in the *.mlapp* file of the GUI, audio feedback was included to guide the user during the eyes-closed training phase, with the motor imagery action guided by sounds at different frequencies depending on the direction to be imagined.

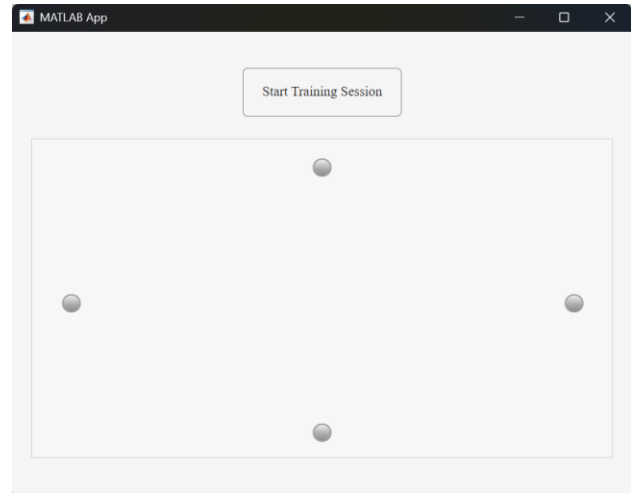
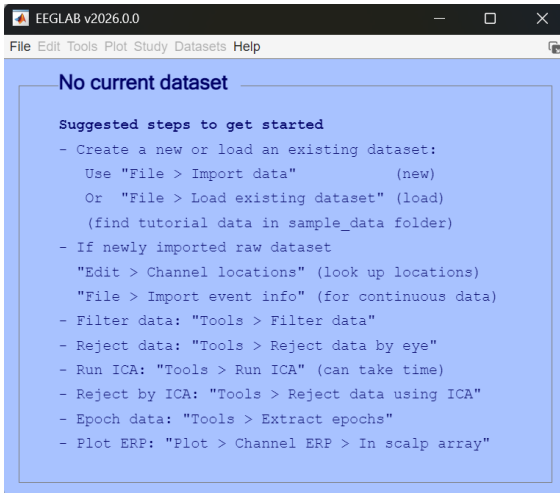


Design window used during the design of the user interface: the image shows the various sections of the Design View in MATLAB App Designer for the creation of the *TrainingSession.mlapp* GUI. On the left is the Component Library, in the centre there is the final GUI composed of the various inserted components, and on the right is the Component Browser section, through which accessibility and editing details of the commands inserted into the GUI can be accessed. [\[F16\]](#)

In conclusion, the starting foundation of our virtual environment consists of four GUIs, chained together and interconnected through dedicated commands accessible to the user in the main application. Once the interfaces and their components were finalised, their configuration during use is shown in the images below.



Interfaces shown on screen after their respective opening via MATLAB App Designer of *NIA_BCI_v1.mlapp* (left) and *DisplayRawData.mlapp* (right). For each, certain elements listed previously are disabled or hidden, following the logic of the code written in subsequent phases. [\[F17\]](#) [\[F18\]](#)



Interfaces shown on screen after their respective opening via MATLAB App Designer of the *Processing with EEGLAB* command (left) and *TrainingSession.mlapp* (right). For each, certain elements listed previously are disabled or hidden, following the logic of the code written in subsequent phases.^{[F19] [F20]}

3.1.2 Back-end and Software Functionalities

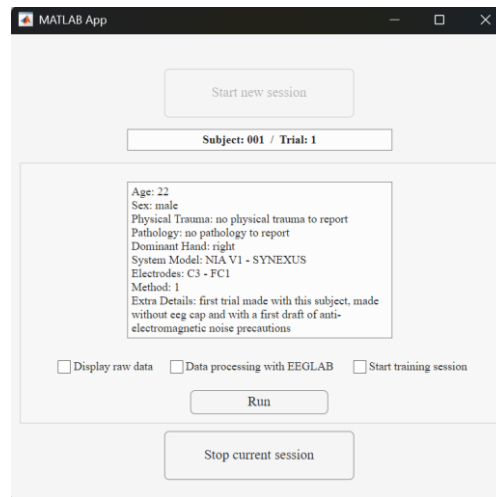
The definition of the operating logic for the applications set up during the construction of the user interface was carried out mainly through code writing in App Designer. The purpose was designing the individual functions defining the behaviour of the components previously created. This phase was crucial, as it made possible creating the pipeline through which the components and the user could interact smoothly and without errors. This part of the work was essentially organized into two consecutive phases for each application or functionality that would ultimately make up the complete final software. In fact, for each one the process began with the initialization of the internal code functions, followed by the writing of the code for each functionality designed for the application. Finally, once the raw code-writing phase was complete, and after preliminarily verifying its operation using test material packages, the debugging and optimization phases of the software were carried out, with the aim of making it robust and versatile with respect to the different types of material it would be provided with during the experimental sessions described in the following chapter. Before proceeding with the description of the code designed, in order to provide clarity to the reader, it should be noted that the complete content of the code will not be reported within this document, for reasons of physical space and for the purpose of protecting the author's work. All material will be available through the GitHub digital platform^[31], which will be linked at the end of this subsection should the reader wish to further explore the topics covered.

The first application to be described is the main one: *NIA_BCI_v1.mlapp*. Once the design of the interface had been completed, the first step in writing the code was to create the corresponding callback for each component, in order to establish the link between the *Design View* and the *Code*

View of App Designer. For each component, this was done through the dedicated option available by right-clicking on the individual component, selecting "Callbacks", and then selecting "Add ComponentName Callback" to generate the corresponding callback. This made it possible to establish an effective link between the two interfaces of App Designer, creating a section of the Code View dedicated specifically to the selected component. Indeed, after performing this action, a new function section, named according to the linked component, became available within the Code View. In this way, it was possible to define the actions and functionalities associated with each component, and consequently to determine the overall behaviour of the application by establishing relationships among the various functions derived from the different callbacks. Specifically, for *NIA_BCI_v1.mlapp*, the components for which callbacks were created included: the Start New Session button (*StartnewsessionButtonPushed*), the Subject Code and Trial Info Edit Field (*dEditFieldValueChanged*), the Stop Current Session button (*StopcurrentsessionButtonPushed*), the General Info Text Area (*TextAreaValueChanged*), the Run button (*RunButtonPushed*), and the three functionality checkboxes (*DisplayrawdataCheckBoxValueChanged*, *DataprocessingwithEEGLABCheckBoxValueChanged*, and *StarttrainingsessionCheckBoxValueChanged*). As a result, the Code View came to be organized into several sections, each dedicated to a specific component. The next step consisted of defining the internal behaviour of each callback and its associated component. To this end, it was first necessary to initialize the variables required for the code to function and to enable communication between the different functions. This was achieved by creating a *properties* section with *Access = public*, containing the variables used throughout the code and within the individual functions of the application. Using the dedicated "Property" command in the App Designer Editor window, the properties section was created and populated with the following initialized variables: *errorMessage*, *eegData*, *dimEegData*, *subject*, *numberOfTrials*, *displayRawDataValue*, *dataProcessingValue*, *startTrainingValue*, *displayRawDataAppHandle*, and *trainingSessionHandle*. Once these steps had been completed, work proceeded with the definition of the application's start-up logic, that is, its behaviour at launch. This was accomplished by creating a start-up function, containing the logic governing the commands and functionalities of the application at launch. This function was named *startupFcn*, and within its specific components were disabled or enabled, and shown or hidden, as appropriate. As shown in the images provided in the previous chapters, some components of the main GUI appear greyed out relative to others: at the beginning of the application, only the Start New Session button is available, so as to prevent access to commands whose execution depends on a session having already been started. Indeed, several GUI components require data and information derived from the file selected through the Start New Session button, and are therefore not meaningful

until that selection has been made. Disabling these components through the command `app.ComponentName.Enable = 'off'/'on'`, according to the intended logic, makes the software more robust against improper use by the user. Furthermore, the commands `app.ComponentName.Editable = 'off'` and `app.ComponentName.Value = 'off'` were used to initialize the edit fields and text areas of the GUI, in order to prevent unintended edits by the user. It was likewise necessary to initialize the values of the functionality checkboxes through the command `app.ComponentNameOfTheCheckBox.Value = 0`. Finally, the Run button and the Stop Current Session button were also disabled through the same approach used for the edit fields and text areas. After defining the start-up behaviour of the application, attention turned to the development of the first function, *StartnewsessionButtonPushed*. This function serves several purposes, the main one being to start a new analysis session by allowing the user to select the file from which the subsequent steps will begin. Through the MATLAB command `uigetfile`^[32], the user selects the desired file, after which its directory and path are stored in MATLAB variables so that the files containing the subject code, the selected trials, and the general session information can be located; this information is subsequently used to populate the first Edit Field and the Text Area. If there are no errors occur during these steps, the relevant components are enabled according to the logic opposite to that defined in the start-up function, while the Start New Session button is disabled. Finally, the function saves the neural signal data contained in the file selected at the beginning of the process, making it available to the application's three functionalities. Before examining the code of the three *.mlapp* files linked to the three functionalities of the main GUI, it is worth discussing *RunButtonPushed*, *StopcurrentsessionButtonPushed*, and *resetAppStateAfterError*. These functions are essential to the logic of the application since, like the start-up function, they define its behaviour when running, when closing, and when facing an internal error. Once the data-saving process has completed successfully and one of the three checkboxes discussed below has been selected, the user can launch the corresponding command through the Run button and its associated function. This function checks which checkbox is currently selected and invokes the corresponding function accordingly. Alternatively, the user can stop the current session through the Stop Current Session button, whose function restores the GUI to its initialized state using the same commands employed in the start-up function. Among the last of these three functions, *resetAppStateAfterError*, is particularly important, as it was created specifically to handle internal code errors. Whenever an error occurs, whether due to user action or to a fault in the code itself, the application must be returned to its initialized state and the user must be notified with a message describing the nature of the issue encountered. This function will, however, be discussed in detail in the debugging and optimization section of this chapter. The application remaining functions are *DisplayrawdataCheckBoxValueChanged*,

DataprocessingwithEEGLABCheckBoxValueChanged, and *StarttrainingsessionCheckBoxValueChanged*. These are essential because, depending on the user's action during the signal analysis process, they update the values of the variables associated with the checkboxes, which are then read by the Run button function in order to open and display the correct GUI based on the checkbox selected. These three functions are therefore linked to *RunButtonPushed*, which, through a series of *if-elseif*^[33] statements, determines which GUI should be opened to proceed with the next steps of the experimentation. Furthermore, the *Data Processing with EEGLAB* checkbox is also linked to a dedicated function within the *methods* section (with *Access = private*), named *DataProcessingWithEEGLABFunction*, whose purpose is to prepare and enable the integration between the custom digital environment created for this project and the pre-existing EEGLAB toolbox.

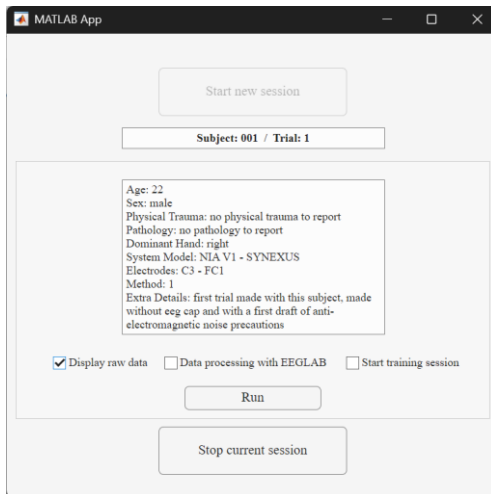


Interface of *NIA_BCI_v1* into the MATLAB App Designer Environment after selecting an example file. As mentioned inside the previous paragraph, the components now become available and accessible to the user. The example subject is in this case the number 001, and the respective trial is 1. The general information is shown inside the central text area, and from here, the user can access to the three implemented functionalities through the check boxes and the Run button.^[F21]

In the next three paragraphs, the available functionalities will be described, along with their internal composition, so as to illustrate the code and its inner workings.

The first is *DisplayRawData.mlapp*, accessible through the corresponding checkbox in the main GUI. The characteristics of the code for this *.mlapp* file are distinctive, since its purpose is essentially to display, and make available for precise modification, a given number of graphs based on the information contained in the file selected in the first GUI. The initialisation code structure is very similar to that of the *NIA_BCI_v1.mlapp* GUI, and therefore consists first of a section in which the variables are initialised, in this case: *errorMessage*, *startTime*, *endTime*, *Fs*, *windowDuration*, *totalDuration*, *numberOfChannels*, and *eegDataStored*. Following this initial phase, a start-up function named *startupFcn* is likewise present, in which the components *PreviousButton*, *NextButton*,

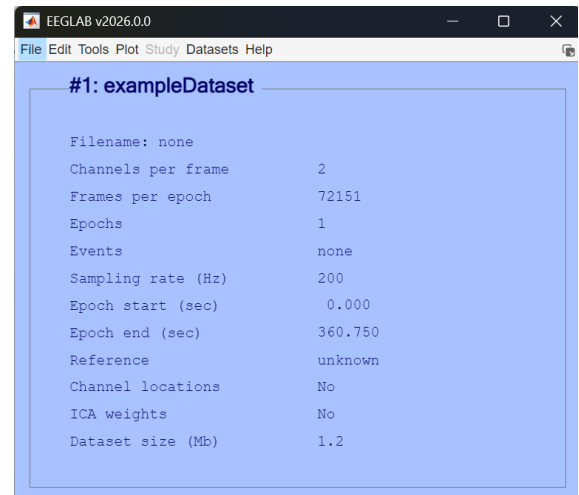
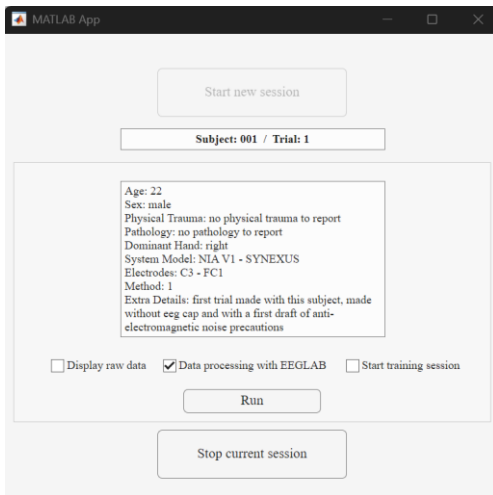
UIAxes, *UIAxes2*, *UIAxes3*, and *UIAxes4* are made invisible or disabled through the command `app.ComponentName.Enable/Visible = 'off'`. The first function to take place after the start-up is *PlotButtonPushed*, whose selection triggers the function *plotRawData*, located within the *methods* section with *Access = private*. Here, the first step consists of retrieving the EEG dataset saved in MATLAB's global memory (a process carried out during the operation of the main application). Then, depending on the number of channels actually present in the dataset, the code makes the corresponding number of *UIAxes* visible and initialises the *startTime* and *endTime* based on the values defined in the *properties* with *Access = private*. Finally, the GUI is updated by enabling the *Next/Previous* buttons and by opening a new function named *plotCurrentWindow*. This latter function is crucial, as its role is to update the graphs displayed in the *UIAxes* according to the values selected through the *Next/Previous* buttons. Every time the user chooses to move along the graph of the recorded neural signal, *plotCurrentWindow* updates the *startTime*, *endTime*, and *windowDuration* of every *UIAxes* made visible within the *plotRawData* function. Finally, as with the main application, a dedicated function was also created for this application to manage errors and reset it in case of anomalies. This function is named *resetApp* and through `app.ComponentName.Enable/Visible = 'off'` renders every component (with the exception of the Title) inaccessible, while simultaneously displaying only the corresponding error message.



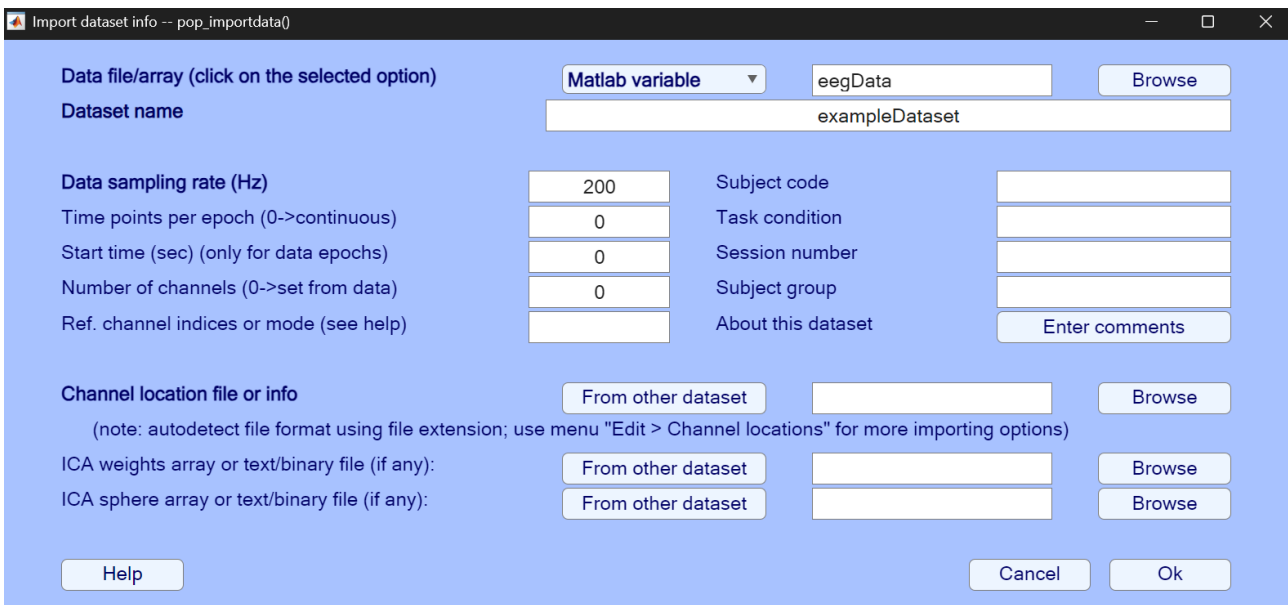
From left to right: *NIA_BCI_v1.mlapp* interface updated with an example file and the Display raw data check box selected / *DisplayRawData.mlapp* Interface based on the information contained in the previously selected file. In this case, the images display an example. The chosen trial was unofficial and there are only two *UIAxes* with the respective two recorder EEG signals. [\[F22\]](#) [\[F23\]](#)

The second step consisted of the design and implementation of the processing toolbox within the *NIA_BCI_v1.mlapp* GUI. As mentioned before, the core concept for this step was understanding how to integrate a finite and independent toolbox such as EEGLAB. First of all, the entire integration process was carried out within the main *.mlapp*, rather than by creating a new GUI as was done for the other two functionalities. Before undertaking the writing of the code, the preliminary step

consisted of understanding the architecture of the target toolbox. EEGLAB is in fact contained within a specific folder named *eeglab2026.0.0*, in which several sub-folders and various files are present. Among these, particular folders and a file can be highlighted, which were analysed in depth in order to understand the method of integration. More specifically, the folders named *function*, *plugins*, *sample_data*, *sample_locs*, and *tutorial_scripts* are fundamental to EEGLAB, since they contain the files and sub-folders necessary for its functioning. For instance, the *function* folder contains *.m* files^[34] in which the MATLAB functions used by the toolbox are present. Instead, the *plugins* folder contains the plugins available for use within the toolbox, saved as *.m* files, each of which provides the user with a specific processing tool. Finally, among the last three folders, it is worth mentioning *tutorial_scripts*, since it contains a series of *.m* files useful for understanding how to make full use of EEGLAB. After understanding the role of each folder, the other crucial file to be integrated into the code of the main GUI is the *eeglab.m* file, necessary to launch the toolbox and display it on the screen. Turning now to the Code View of *NIA_BCI_v1.mlapp*, the integration process begins with the *DataProcessingWithEEGLABFunction* function, which takes place within the pipeline after the selection of the corresponding checkbox (*DataprocessingwithEEGLABCheckBoxValueChanged* function) and the launch of the Run button (*RunButtonPushed* function). The role of *DataProcessingWithEEGLABFunction* is essentially to prepare the EEG dataset for sampling by the toolbox through the MATLAB command *assignin*.^[35] The next step consists of locating the main folder mentioned above through the MATLAB command *fullfile*^[36] and then creating a variable that contains its root. After finding the directory of the folder, the last step consists of locating the main *.m* file of EEGLAB (*eeglab.m*) and then launching it. These latter two steps are crucial, since in order to open a file that is not present in the same directory as the main script, it is necessary to add the path providing the location of the file through the *addpath*^[37] command. As a result, the toolbox is launched and the main menu of EEGLAB is displayed. In order to begin the processing, it is then necessary to import the EEG dataset saved during the start of the session, stored in the variable named *eegData*, by selecting the following items (in the order given): *File* → *Import Data* → *Using EEGLAB functions and plugins* → *From ASCII/float file or MATLAB array*. Once the user reaches the new interface displayed after this command, it is necessary to import the correct variable named *eegData* (as mentioned above) and then choose the desired sampling rate (Hz). After giving a name to the new dataset being created, the EEGLAB interface updates and displays the information about the trial contained in the file selected during the start-up phase of *NIA_BCI_v1.mlapp*. From this point onward, it is possible to manipulate and process the recorded EEG signals as desired.



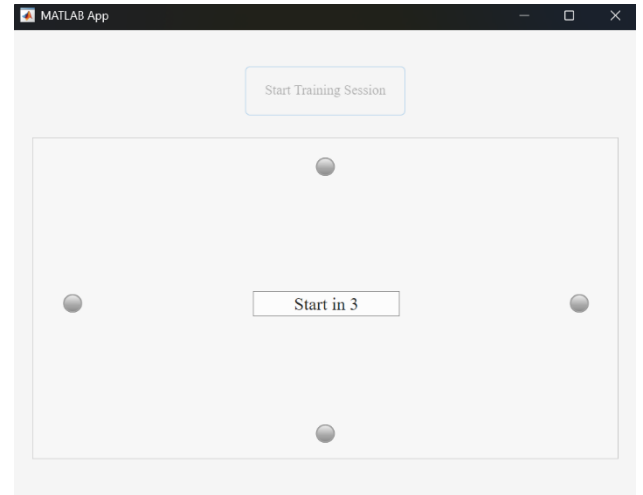
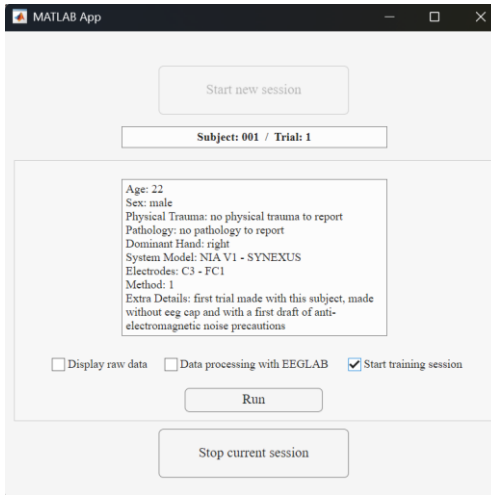
From left to right: *NIA_BCI_v1.mlapp* interface updated with an example file and the Data processing with EEGLAB check box selected / Updated interface of EEGLAB toolbox after importing the EEG data saved during the starting phases. In this case, the images display an example. The chosen trial was unofficial and there is example information of the respective EEG dataset related to the selected file. [\[F24\]](#) [\[F25\]](#)



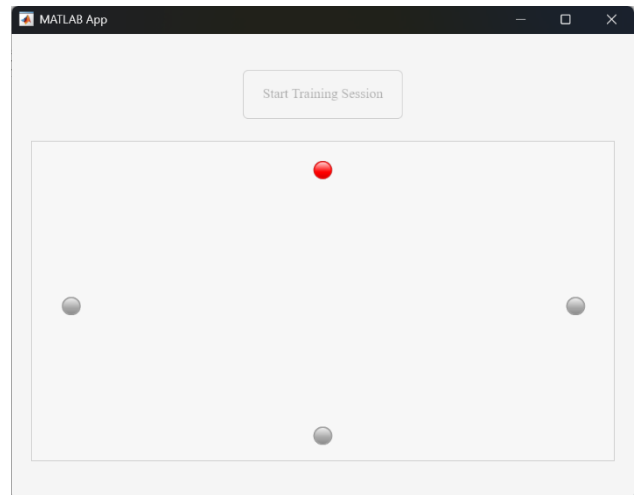
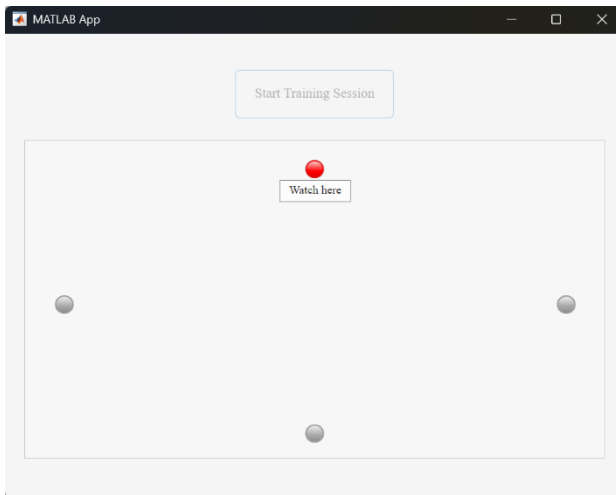
Updated interface of EEGLAB toolbox during the data importation. There are the dedicated part for selecting the EEG dataset (upper part of the window on the right) and the Data sampling rate in Herz (in the middle of the window on the left). In this case, the images display an example. The chosen shown values are arbitrary. [\[F26\]](#)

The last *.mlapp* file that will be analysed is the one related to the training session, whose purpose is to support the user during motor imagery tasks. The application in question is *TrainingSession.mlapp*, and, as with the previous two functionalities, the code writing phase began with the Code View. The first steps, as usual, consisted of initialising the variables used within the code, inside the *properties* section with *Access = private*, and creating the start-up function. The elements involved in these steps are: *countdownValue*, *countdownTimer*, *restTimer*, *lampCountdownValue*, *lampTimer*, *caseOfLamp*, *lampCounter*, *currentCase*, *fSamplingSound*, *soundDuration*, *soundFrequency*, *soundTime*, *soundFunction*, and the *startupFcn* function. In this case, it is worth highlighting that the functioning

of this *.mlapp* file is independent of the variables and dataset present within the main GUI of *NIA_BCI_v1.mlapp*. The logic of the training session is in fact self-contained, and the variables that control it are all defined within the *.mlapp* itself. As regards the functioning of the code, the start-up function initialises the interface by making visible and enabled only the Start New Session button. The function associated with this button is *StartTrainingSessionButtonPushed*, and once it is selected, the training session begins. The starting point is linked to making visible the LED component, the associated Text Area, and the central Text Area, which is essential for guiding the user. The first step of the session is the start of the initial countdown displayed in the central Text Area, which controls the first 3 seconds of the session, dedicated to signalling the start to the user. Time control in this case is achieved through the use of MATLAB commands such as *sprintf*^[38] (which physically prints the countdown inside the central Text Area) and *timer*^[39], which, together with the *updateCountdown* function, makes it possible to update the countdown and print the correct value in the Text Area. This latter is a function created from scratch, which controls the value of the countdown by subtracting 1 second at a time from the current value stored in the variable *countdownValue*. Once the first countdown has finished, control of the interface is taken over by the *updateRestCountdown* function, likewise created from scratch, which manages the rest minute by subtracting 1 second at a time from the *restCountdownValue* variable. The actual training begins after the rest minute, at which point control of the interface is taken over by *updateLampCountdown*. Once this phase begins, two options are possible: the training is divided into "open eyes" and "closed eyes" phases, during which the eyes of the user are, respectively, open and closed. This distinction is handled by means of an *if-else*^[33] statement, which checks the specific phase of the session by monitoring the variable *checkOpen*. The crucial point of the session was then understanding how to control the LEDs during the seconds of training. This was achieved through the MATLAB *switch-case*^[40] statement, based on the *currentLamp* variable, which contains the four possible cases: up, right, down, and left (corresponding to the position of the four LEDs). The active LED changes every 5 seconds, and within this time interval the LED in question turns red (activated state) instead of grey (deactivated state). When the closed-eyes condition is verified, LED activation is accompanied by sound feedback at a specific frequency, different for each LED position. The last two phases just described each last 1 minute, so that the training session, including the rest period, lasts 3 minutes in total. In order to provide a training session comparable to the recording session carried out through the OpenBCI GUI, the phases described above are repeated, in order to achieve a training session lasting 6 minutes in total. Finally, once the 6 minutes have elapsed, the interface is reset by the *resetApplication* function, which restores the state of the application to its original condition, as defined in *startupFcn*.



From left to right: NIA_BCI_v1.mlapp interface updated with an example file and the Start training session check box selected / Interface of *TrainingSession.mlapp* just after the start of the training session, with the first countdown displayed and the four LEDs non activated. [\[F27\]](#) [\[F28\]](#)



From left to right: *TrainingSession.mlapp* interface displayed during the first active phase of the session when the user's eyes are open / *TrainingSession.mlapp* interface displayed during the second active phase of the session when the user's eyes are closed. [\[F29\]](#) [\[F30\]](#)

3.2 Conclusion and Consolidation of the Application

The development of an application does not end with the mere conclusion of the code-writing phase, but continues through further steps that extend beyond the boundaries of the application itself. During the creation and design of the code, it is possible for undesired anomalies to arise, due to either the format of the selected external files or to miscommunication dynamics between variables. Moreover, in most cases the final code is not efficient enough to support a large amount of data, and it may therefore crash as a consequence of overloading. It is as consequence necessary to upgrade the final code by adding functions and implementations that can support its efficiency or provide the user with the correct instructions for use. More specifically, the latter refer to the *debugging* phase and the

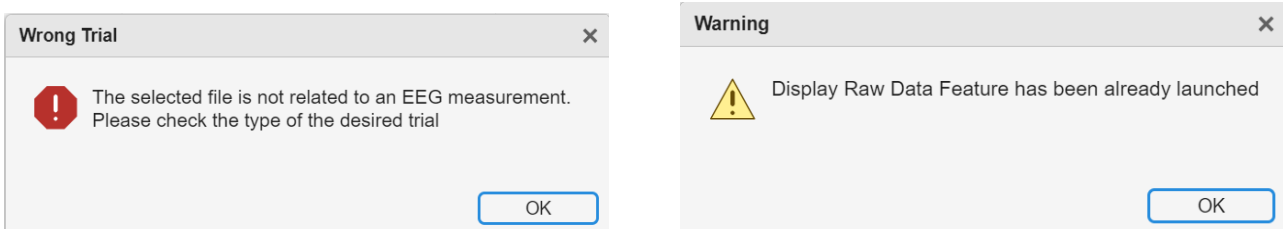
optimisation phase, in which the originally written code is modified in order to correct errors or to prevent the occurrence of others during the functioning of the application, a step that is essential to guarantee the stability and validity of the developed software. These steps were carried out directly within the MATLAB App Designer environment, thanks to the debugging tools provided by the platform. Therefore, within the next two sub-chapters, the methodologies used during the verification of the code, the MATLAB tools employed and the implementations created from scratch will be presented. In addition, this there will be also the explanation of how the application became independent from App Designer. These latter steps will explain to the reader the pipeline followed in order to achieve a standalone application, used directly during the experimentation phases and shareable with other devices.

3.2.1 Debugging and Optimisation

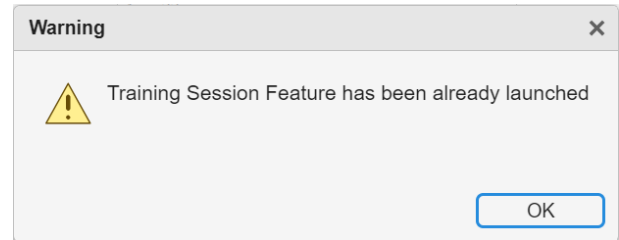
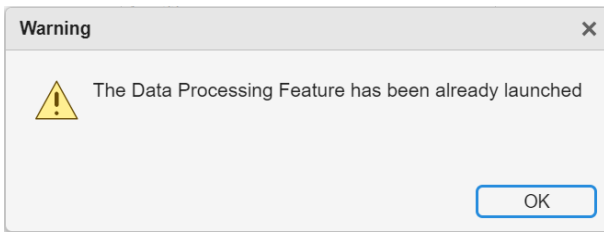
As with the design phase, the *debugging* and optimisation phases likewise followed a standardised procedure, which was repeated for each *.mlapp*. More specifically, both steps were carried out within the App Designer interface, inside the Code View, since the purpose was to check the nature and quality of the code. In order to achieve a high level of verification, it was necessary to check every single string created within the code and control its input and the corresponding output. Every variable devised is linked to another one or directly to another application, so it is essential to be sure that every component provides the desired value. For this reason, the MATLAB built-in function used for the greatest amount of time was the debugging function, accessible through the selection of specific strings by means of so-called *breakpoints*^[41]. At these points, the flow of the code stops, allowing the checker to control the format of the variable and its value. Every time the flow of the code encountered an error directly related to the code, it was therefore necessary to analyse the string in question and the preceding ones, until the anomaly was identified. Movement along the code was made possible through the *Step / Step In / Step Out*^[42] MATLAB commands, which allow the code to be run string by string. Once an anomaly had been identified, the next step consisted of attempting to solve the problem, for example by changing the commands or the relations between the variables, or by creating an alert that displays a message on the screen for the user, highlighting the nature of the error and possible actions to prevent or solve it. This was made possible through the *uialert*^[43] function and *try-catch*^[44] statements, a combination that identifies the anomaly and displays an alert on the *UIFigure* selected as input, with specific characteristics/icons depending on the *uilaert* function inputs. Following the debugging phase was the *verification* phase, whose purpose was to check the stability of the code after the modifications implemented to solve the previous error. If the code ran correctly

and no anomalies occurred within other functionalities, the last step consisted of implementing new functions or commands with the aim of making the code more efficient.

Having provided the reader with the fundamentals of the content explained in this chapter, it is now possible to present the corrections and optimisations applied to each *.mlapp*. The first application to be analysed is *NIA_BCI_v1.mlapp*, where the errors originated mainly from the communication between the three main functionalities, the storing of the EEG data, and the general behaviour of the *UIFigure*. Within *StartnewsessionButtonPushed*, the stability of the file selected after pressing the start button is ensured through try-catch statements, which stop the flow in the event of an incorrect file *format*. Other similar statements are then used to check the correct saving of the *directory*, the correct *nature* of the file (which must correspond to an EEG measurement), and the saving of the subject code. If-else statements then check for the presence of the corresponding *.xlsx* file within the saved directory. During the updating of the interface components, such as the central Text Area, try-catch statements check the correctness of the code flow, in order to provide *TextAreaValueChanged* with the proper values. In relation to this, other try-catch statements control the compiling phase and stop the code in the event of incorrect information written in the *.xlsx* file. Furthermore, for every error identifiable through the statements described above, *resetAppStateAfterError* is present, and it resets the application whenever an unavoidable error is encountered. As for the *optimisation* of the current code, in order to ensure stability and a logical use of the application, *RunButtonPushed* contains various if-elseif statements to control the correct use of the three functionalities. Indeed, since these are theoretically deployable multiple times through selection of the Run button, it is necessary to display an alert to the user highlighting the fact that a given functionality may already be open, and therefore cannot be deployed a second time. Finally, to ensure an efficient process for closing the application at any point, the code includes *UIFigureCloseRequest*.

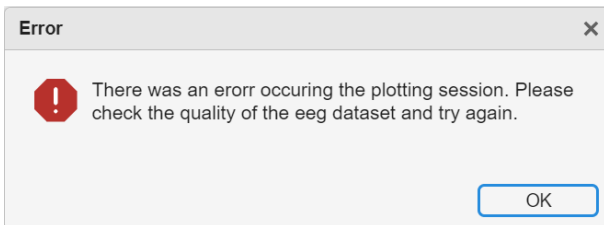


Two of the most common alerts that the user could face during the Application use. From left to right: error related to the selection of a file not linked to an EEG dataset / Warning related to the impossibility to deploy the *DisplayRawData.mlapp* in parallel on two separated windows.^[F31] ^[F32]



Two of the most common alerts that the user could face during the Application use. From left to right: Warning related to the impossibility to deploy EEGLAB in parallel on two separated windows./ Warning related to the impossibility to deploy the *TrainingSession.mlapp* in parallel on two separated windows.[\[F33\]](#) [\[F34\]](#)

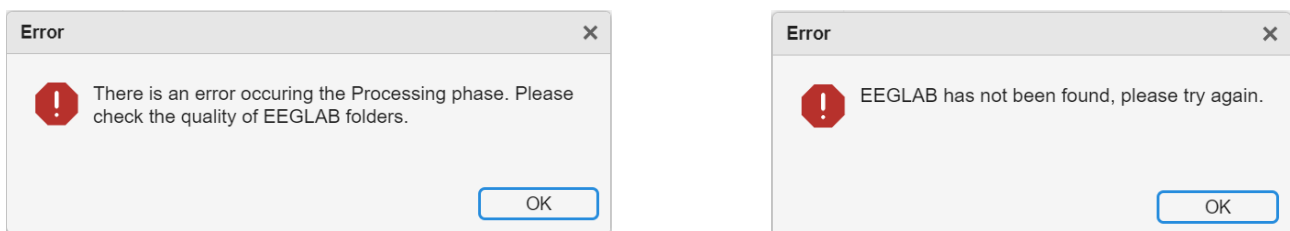
Moving on to the second application, *DisplayRawdata.mlapp*, the process was shorter, since its only job is to display the EEG data provided by the main application. In this case, however, the errors originated mainly from the nature of the provided dataset. The application in question is designed specifically to work with the OpenBCI Ganglion board, which has a maximum of *four channels* capable of recording EEG signals. Therefore, the display application must be constrained to show at most four graphs. In order to achieve this, two checking statements are present within the *plotRawData* function. The first, through a try-catch command, controls the entire plotting process, sending the user an error figure through uialert in the event of anomalies during this phase. In parallel, an if-else statement checks the number of channels provided by the EEG dataset. In case of the selected file provides an overloaded dataset, uialert displays an error figure inviting the user to check the nature of the dataset. As regards optimisation and the prevention of possible errors, if-else statements are present within *NextButtonPushed* and *PreviousButtonPushed*, which prevent the user from going beyond the time limits of the provided trial by disabling the buttons once either the maximum or the minimum is reached. Finally, as with the main application, the code includes *resetApp* and *UIFigureCloseRequest*, which respectively reset the application in the event of encountered anomalies and close the application when the user closes it arbitrarily.



The two main alerts that the user could face during the Application use. From left to right: error inside the creation of the graphs / error related to the number of channels provided by the selected file during the use of *NIA_BCI_v1.mlapp*.[\[F35\]](#) [\[F36\]](#)

Turning instead to the second functionality, related to EEGLAB processing, the analysis takes place entirely within the main code of *NIA_BCI_v1.mlapp*. Indeed, after the selection of the corresponding checkbox and confirmation through the Run button, the flow is directed straight to the *DataProcessingWithEEGLABFunction* function, where try-catch statements check the EEG dataset

saved during the previous phases, in order to identify errors related to its nature, quality, and content. In parallel, a control is carried out to check for a possible already open EEGLAB GUI, which, through `uialert`, displays to the user the same error message described previously in relation to the optimisation of *NIA_BCI_v1.mlapp*. Then, during the opening phase of EEGLAB, a further try-catch statement enables the checking of possible errors during the deployment of the toolbox, so as to invite the user to check not only the provided EEG dataset, but also the downloaded toolbox, and therefore the inner workings of the application. Finally, as with the previous *.mlapp* file, each try-catch statement and if-else control statement is linked to the *resetAppStateAfterError* function, in order to reset the main application and prompt the user or the experimenter to analyse and solve the problem.



The two main alerts that the user could face during the use of EEGLAB implementation. From left to right: Generic error faced during the *DataProcessingWithEEGLABFunction* functioning / Specific alert related to the opening of EEGLAB. [\[F37\]](#) [\[F38\]](#)

Finally, attention turned to the last application, which, like the first one, has a dedicated environment, and our actions were therefore aimed at making it more stable and error-proof with respect to user behaviour. In this case, however, there are some differences from the other two cases described previously, since the training session is contained within a dedicated *.mlapp*, but it does not communicate with *NIA_BCI_v1.mlapp* by sharing a dataset, unlike the functionality for displaying the graphs. Actions were therefore aimed at correcting the internal code, considering as the only external factor the behaviour of the user and the internal logic governing the training. Starting with the latter, the flow of the timing control is managed through specific variables that change their values over time, and through if-else statements that update the user interface in order to provide updated information based on the current phase of the training. More specifically, timing control is made possible through the *pause()*^[45] command, which, depending on the input, halts the flow of the code. The correct flow of the switching phase between the different LEDs is then made possible through switch-case statements and if-else statements, which distinguish between the *open-eyes* phase and the *closed-eyes* phase. Once the logic of the application had been stabilised, it was necessary to optimise it in order to make it stable in the face of erroneous user behaviour, such as closing the platform before the end of the session. In order to achieve this, the *UIFigureCloseRequest* function was implemented, which initiates the closing process. At the end, however, it was also necessary to insert specific if-else statements within the code to support the closing process, thereby avoiding errors caused by the timer continuing to run after the application had been closed. These checks are always present

before and after the updating of the central Text Area, the LEDs and their respective Text Areas, and the sound feedbacks. Indeed, in the case of a closed application with the session still running, the if-else statements check for the existence of the application, and, if it no longer exists, they direct the flow of the code outside the function, and therefore to the resetting phase.

In conclusion, the actions described in the previous paragraphs make it possible to achieve a stable and functional MATLAB environment, ready to face the future experimentation phases described in the later chapters.

3.2.2 Exportation of the Finished Application

The final step preceding the practical use of the application developed is its proper exportation. This phase encompasses the procedures required to transfer the application from the MATLAB environment to an external, self-contained form, through the generation of the files and resources needed for the standalone version. This is achieved by means of a dedicated internal toolbox, which allows the developer to monitor and adjust every final *back-end* and *front-end* aspect of the deployment: the *Standalone MATLAB Compiler*^[46]. This tool packages MATLAB programs for deployment as standalone applications.

Once the creation, debugging and optimisation phases had been completed, the toolbox leads directly to its main interface. However, progressing further first required the creation of a new project. Upon selecting the toolbox, the opening window presents three options: *starting a new project and creating a compiler task*, *adding a compiler task to a recent project*, or *browsing for an existing project to which a compiler task can be added*. Considering that the present case involved building the project from scratch, the first option was selected. After specifying the project name and folder, respectively, *NIA_BCI* and the main project directory, clicking the corresponding button led to a new interface, named *Manage Task*, in which the various ongoing projects can be managed. At this stage, the toolbox also generates two key elements within the selected project folder: the *.prj*^[47] file, named after the main *.mlapp* of the application, which allows the user to return to the same project should the process not be completed in a single session, and the resources folder, containing several sub-folders with the files required for the deployment and operation of the future standalone application. The subsequent steps, in which the application characteristics are actually configured, are accessed by selecting the relevant project, renamed in this case, *NIA_BCI_v1_ProcessingAndTraining*, from within *Manage Task*.

The latter opens the final configuration window preceding the building and packaging of the application, in which the following settings were defined according to the requirements of the present project:

- *Main File*: NIA_BCI_v1.mlapp
- *Application Info*
 - *Name of the project*: NIA_BCI_v1
 - *Author*: Giuliano Agostini
 - *Comments*: Main Interface of NIA Environment
 - *Output Locations*: Output Folder for the Build: the same as the current project / Output Folder for the Installer: the same as the current project
- *Executable Details*
 - *Executable Name*: NIA_BCI_v1_ProcessingAndTraining
 - *Application Type*: treat inputs to the app as a MATLAB character vector
 - *App Help*: create a help file on how to use the app
 - *Images for the standalone application*: the respective logo shown previously
- *Files Required for Standalone to Run*
 - *Required Support Packages*: no support package dependencies detected
 - *Custom Requirements*: eeglab2026.0.0 folder
 - *Discovered Requirements*: DisplayRawData.mlapp, TrainingSession.mlapp
- *Installer Details*
 - *Package Type*: fetch MATLAB Runtime from MathWorks during application installation
 - *Installer Name*: NIA_BCI_v1_ProcessingAndTraining_Installer
 - *Images for the Installer*: same as the images for the standalone application

Once all the required fields had been completed and MATLAB automatic compatibility analyses had concluded without issue, the project was ready for deployment. This was carried out by selecting the *Build and Package* command, which opens a dedicated window allowing the user to monitor the outcome of the export process for both the application and the installer. Upon its successful completion, and before the standalone application could be employed in the experimentation phase, its correct functioning had to be verified. However, the verification could not be performed immediately, as several preliminary adjustments to the internal files of the standalone application proved necessary beforehand. In particular, every file or folder name containing special characters or symbols, such as &, à, -, ?, ! and similar, had to be renamed to remove them, as they were found to

be the source of errors arising during the exportation stage. The most challenging aspect of this process, however, concerned EEGLAB: being an external, well-established toolbox not originally designed with standalone deployment in mind, it required substantial adaptation. This consisted in removing all folders and files not strictly necessary for the standalone application, such as those dedicated to *tutorials* or *example* scripts, together with the modification of specific lines of code within *NIA_BCI_v1.mlapp*, where the *fullfile* command was employed with *ctfroot*, *NIA_BCI_v1_P* and *eeglab2026.0.0* as arguments. After the completion of the exportation process, the user is directed to a dedicated folder, named after the corresponding task defined within Manage Task, containing the build and package sub-folders. The former includes several *.json*^[47], *.txt* and *.log*^[47] files, together with the executable *.exe*^[47] file *NIA_BCI_v1_ProcessingAndTraining.exe*, ready for use.

3.3 Utilisation and Experimental Procedure

The final phase involving this part of the digital system concludes with the actual use of the applications and programs developed. Following the export of the application, as explained in the previous sub-chapters, the direct consequence is verifying its functioning by employing it in combination with the hardware components applied to the user in question. Moreover, the experimentation itself follows a specific procedure, in order to ensure the correct use of the environment and to avoid critical error situations. The objective pursued in this concluding part of *NIA_BCI_ProcessingAndTraining* is therefore to establish an initial connection between the user and the digital device, allowing the experimenter to achieve the optimal configuration of the hardware components and to help the participant become comfortable with the novel neural mechanism underlying the functioning of the system.

3.3.1 Approach and Methods of use

The experimentation involving the aforementioned programs follows a scheduled and structured pipeline, characterised by specific steps aimed at achieving, by its conclusion, a stable and reproducible experimentation that can be applied and adapted to other users. This chapter therefore sets out the specific steps of this stable and carefully considered pipeline, with the purpose of clarifying the choices made and the reasoning behind them, which led the experimenter to select the particular applications and functionalities involved. Before proceeding with the explanation, and in order to render it more realistic, the subject with ID *001* is used as an example user throughout, having properly employed and tested the assembled and complete environment during real-life

experimentation over the course of the project's development. Every piece of information and detail provided is therefore based on real life. At the same time, anonymity is preserved, since the experimentation does not involve the use of private or sensitive personal data. Furthermore, in order to ensure that the experimentation begins in the correct manner, the user is also asked for consent to take part in it, so as to guarantee full awareness and acknowledgement on the part of the user and to avoid exceeding the limits of that consent.

The first point of contact between the system and the user consists of gathering every piece of information that may be relevant to the configuration of the BCI system. Patient intake is fundamental, as its aim is to gather the specific details that can assist the experimenter in assembling and adapting the system to the user, thereby creating a stable and personalised foundation from which the experimentation can proceed smoothly. In this phase, the user is therefore asked to answer questions relating to *anthropometric* and *neuropathological* aspects in a *yes/no* format, supplementing these with additional information where necessary. This data is initially gathered by means of a paper form, which is subsequently transcribed into an *Excel* file, so as to also maintain a digital version of the user's data and make it consultable whenever necessary. Accordingly, the file *001_InfoSubject.xlsx* is created, containing a table requesting the following: *Age*, *Gender*, *Physical Trauma*, *Neuropathology*, *Dominant Hand*, *System Version*, *Electrodes*, *Method*, and *Extra Details*.

Age	22
Gender	FEMALE
Physical Trauma	NO
Neuropathology	NO
Dominant Hand	RIGHT
System Version	NIA V1 - SYNEXUS V2
Electrodes	CZ - C1 - C3 - FC3
Method	1
Extra Details	NO

Image of the Excel Table that contains the asked anonymous personal information.^[F39]

The next step is directly related to the preceding ones, as it involves the same procedure and programs. More specifically, it is carried out principally by the experimenter, its aim being to record information

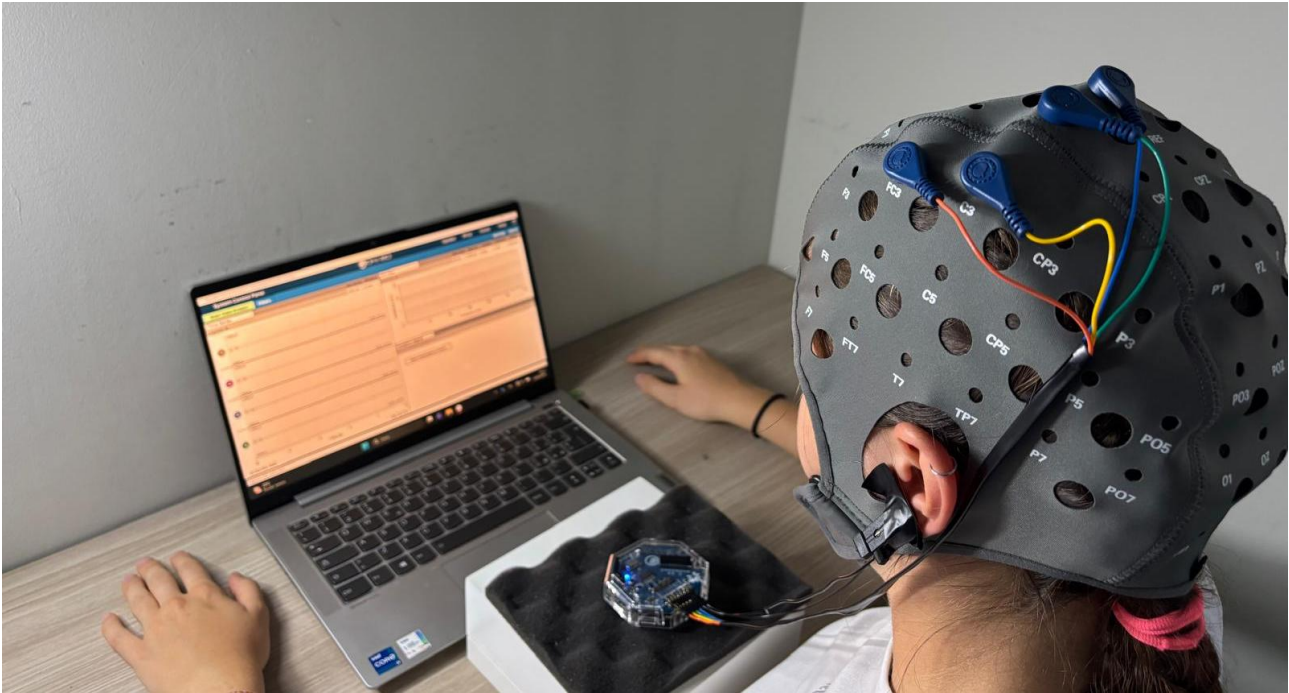
about the trial to be conducted with the user concerned. Moreover, in this phase it is necessary to standardise the experimentation procedure, in order to determine the particular features that can be adapted and altered within the hardware component configuration. Accordingly, an Excel file is created and named *001_1_InfoTrial.xlsx*, so as to specify the subject ID code, the trial number, and the type of information contained within the file. The table fields relate directly to the positions of the sampling electrodes, divided into two columns. The first shows the status of Channels 1/2/3/4 based on an *On/Off* value, while the second specifies the corresponding position of the active electrode indicated in the first column.

Channel 1	ON	Positioning	CZ
Channel 2	ON	Positioning	C1
Channel 3	ON	Positioning	C3
Channel 4	ON	Positioning	FC3

Images of the Excel table that contains the details about the current trial, related directly to the electrode status.^[F40]

The preliminary phases are now complete, and the practical part of the experimentation begins. Indeed, having gathered all the useful information by means of the Excel tables just described, the experimenter must apply this information to the configuration of the hardware component. It is therefore necessary to gather the hardware elements involved, namely: the *EEG cap*, the *sampling electrodes*, the *reference* and *noise-cancelling* electrodes, the *snap cables*, the *USB connector*, and, finally, the *OpenBCI Ganglion board* together with its respective battery and PVC protective case. The electrodes are subsequently placed over the positions covering the area of representation of the dominant hand and arm within the sensorimotor cortex, specifically at the positions indicated in the trial and subject information tables: Cz, C1, C3 and FC3 (given that, in the present case, the dominant hand is the right one). The snap cables are then connected to the placed electrodes, so as to ensure their stability both on the scalp and within the holes of the EEG cap. Following these steps, the EEG cap is properly positioned over the subject's head, and its placement is secured by means of the respective strap beneath the chin. At the same time, the reference and noise-cancelling ear-clip electrodes are placed over the ipsilateral and contralateral earlobes. Everything is then connected to the snap cables, which carry the communication directly to the active and functioning processing board, already connected to the digital device concerned via the USB connector. Before proceeding

with the following steps, however, it is necessary to ensure that the assembled hardware system is not invasive for the user, by checking for possible excessive pressure exerted by the electrodes over the scalp or other parts of the subject's head. At the same time, the experimenter must verify the stability of the system, checking that the electrodes are correctly positioned and properly connected to the snap cables, so as to ensure the correct flow of signal to the processing board.



Images of the Assembled and Completed BCI system applied on the studied subject.^[F41]

From this point onward, the digital part of the experimentation begins, in which the steps involved relate directly to the applications and programs within the chosen and developed digital environment. The first step consists of using the *OpenBCI GUI*, specifically designed to communicate with the chosen instrumentation. Within this application, the connection between the laptop and the processing board is established by selecting, within the main menu, respectively: *GANGLION (live)* → *Bluetooth (BLED112 Dongle)* → *START SEARCH* → the processing board present within the *BLE DEVICES* section → *START SESSION*. Through this workflow, the main interface is reached, where it is possible to visualise the features of interest relating to the neural characteristics of the sampled EEG signal at each electrode. More specifically, three widgets of the *OpenBCI GUI* must be selected, which are particularly useful for ensuring the stability of the system and, consequently, the stability of the subsequent recording phase: *Time Series*, which displays the raw EEG signals at the electrodes of interest, *FFT Plot*, which displays various frequencies on the x-axis and each frequency's respective amplitude, in μV , on the y-axis, and, finally, *Ganglion Signal Check*, used to check the status of each pin/electrode by means of an injected current. Once the layout has been established with the widgets just introduced, it is therefore possible to begin the recording phase by pressing the *Start Data Stream*

button. From this point onward, a scheduled and precise experimental recording begins, with the aim of creating a standardised recording file that is analysed, in the subsequent phases, in the same way for every future subject who uses the complete system. More specifically, the recording phase lasts six minutes and is divided into the following sub-phases:

- *One minute of rest*, during which the user must relax in order to reach a relaxed state in which neural activity is stable and unaffected by critical, unwanted neural features.
- *One minute of open-eyes* motor imagery tasks, during which the user must imagine, every five seconds, a different direction, following the pattern Up → Right → Down → Left, by imagining moving the dominant hand into the positions just described.
- *One minute of closed-eyes* motor imagery tasks, during which the user must imagine, every five seconds, a different direction, following the pattern Up → Right → Down → Left, by imagining moving the dominant hand into the positions just described.

These three sub-phases, each lasting one minute, must be repeated twice in succession, so as to cover the entire expected six minutes of the recording phase. Upon conclusion of the recording phase, provided the quality of the trial is deemed satisfactory by the experimenter, it is possible to proceed with the subsequent steps.



Image of the operational OpenBCI GUI with the three-features layout. [\[F42\]](#)

The result of the previous step carried out within the OpenBCI GUI is a *.txt* file, with a specific internal layout, containing the data sampled during the recording phase. From this point, therefore, the experimentation aims to prepare the data so as to make it ready for the MATLAB application

developed within the *NIA_BCI_ProcessingAndTraining* environment, described in the previous chapters. First of all, it is necessary to rename the output file in a standardised manner that can also be replicated for subsequent trials and future subjects. In the present case, the file is named as follows: *OpenBCI-RAW-2026-06-14_21-53-12.txt*, so as to highlight the nature of the trial (OpenBCI-RAW-) and the date on which it was conducted (2026-06-14_21-53-12). It is then necessary to place every useful file within a common location. Accordingly, for each trial and for each subject, a specific folder must be created, in the present case named 001, containing the sub-folder relating to the trial conducted, in this case *EEG_001_1_2026-06-14_21-27-28*, which in turn contains the two *.xlsx* files and the *.txt* file. Indeed, once the data has been prepared, it is possible to open the *NIA_BCI_ProcessingAndTraining* application and begin the second part of the experimentation. Upon launching the application, the user is asked to select the desired trial file (as explained in the chapter introducing the functioning of the *NIA_BCI_ProcessingAndTraining* environment). Therefore, once *OpenBCI-RAW-2026-06-14_21-53-12.txt* has been selected, the interface is updated, displaying the subject code and trial number in the upper section, and the information contained within the *001_InfoSubject.xlsx* file in the central section.

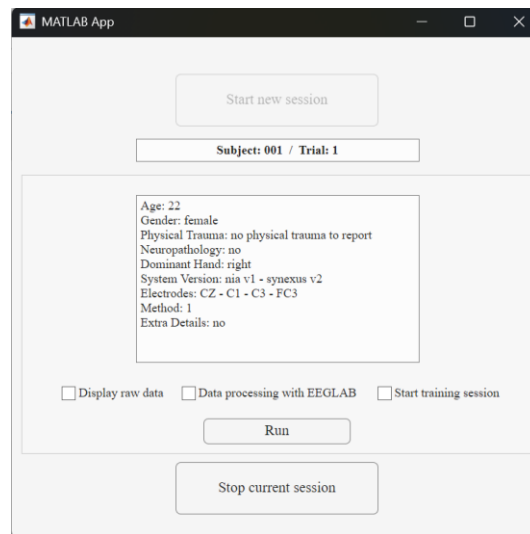


Image of the principal interface within the NIA for Processing and Training sub-environment, with the respective subject data.^[F43]

The first functionality used within the main interface is *Displaying the Raw EEG Signal*, as it enables verification of the expected quality of the signal. Within this interface, it is necessary to verify whether the expected number of channels is displayed, by counting the number of graphs shown, and it is also necessary to verify the correct length of the trial, by using the next/previous buttons to check the beginning and the end of the graphs' x-axes. Then, with direct reference to the EEG signal itself, it is important to check for the presence of critical artefacts during the active part of the recording, such as disconnection or excessively noisy conditions, which could compromise the analysis carried out by the application's subsequent features.

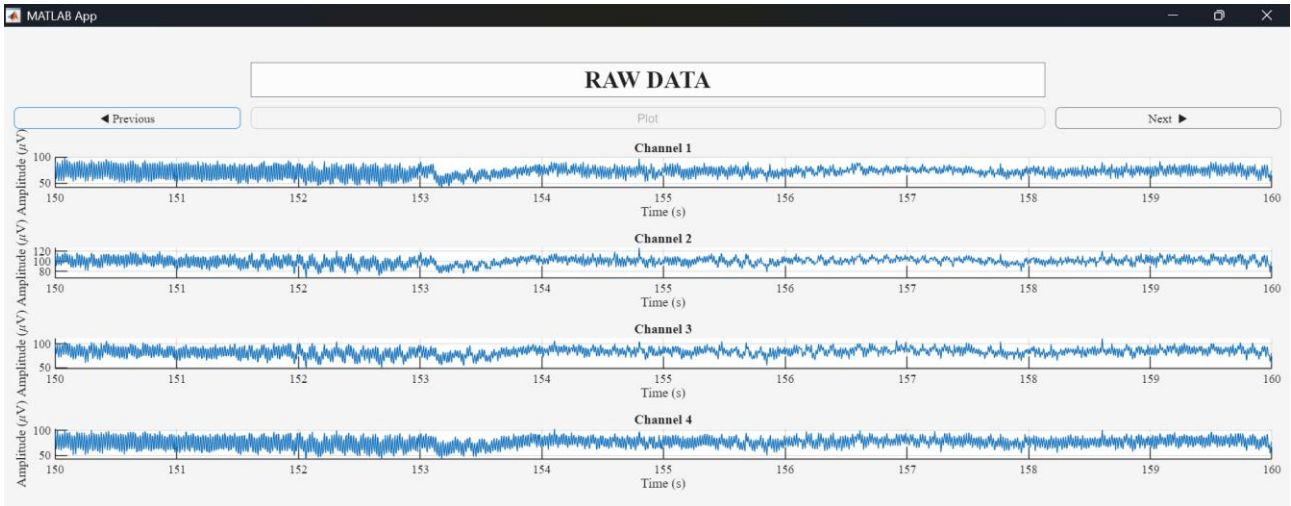
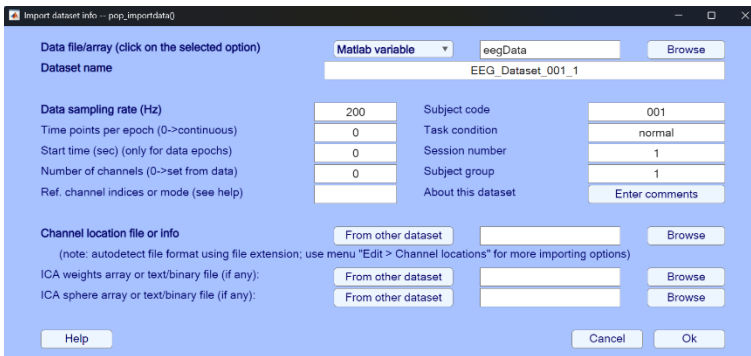


Image of the Display Raw EEG signals functionality with the four channels used during the experimentation. [\[F44\]](#)

Completion of the *Displaying Raw EEG Signal* step leads to the next functionality, whereby, through the EEGLAB toolbox, the raw signal just examined can be cleaned, in order to prepare it for the proper analysis of the neural features under investigation. Accordingly, after selecting this feature within the main interface and opening the respective window, the workflow follows this pattern: *File* → *Import data* → *Using EEGLAB functions and plugins* → *From ASCII/float file or MATLAB array*. In this way, the window that opens provides the experimenter with a series of fields to be filled in, so as to prepare the data contained within the selected *.txt* file for the processing phase carried out within the EEGLAB toolbox. More specifically, the fields concerned, together with their respective required values, are as follows:

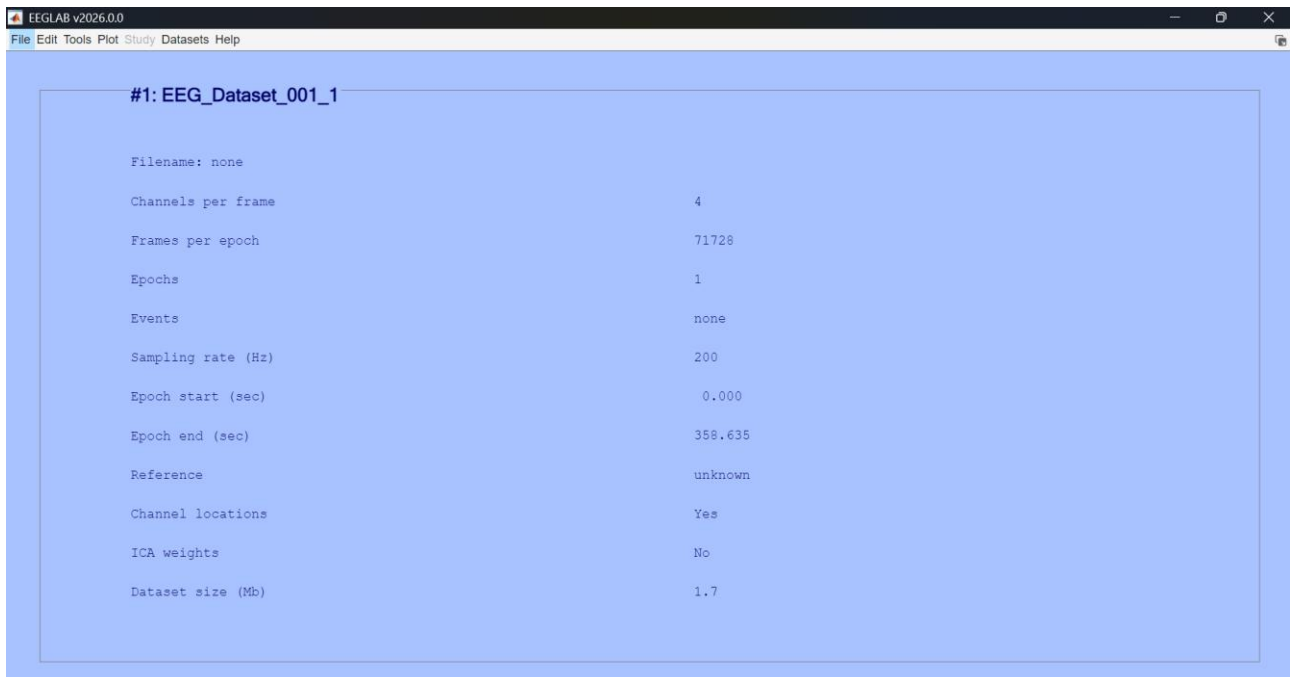
- *Data file/array* = eegdata
- *Dataset name* = EEG_Dataset_001_1
- *Data sampling rate (Hz)* = 200
- *Subject code* = 001
- *Task condition* = normal
- *Session number* = 1
- *Subject group* = 1

Once the fields described above have been filled in accordingly, the importing phase continues with the selection of the OK button, which leads to a second window requesting confirmation of the previously selected dataset name.



From left to right: First window displayed after the selection of the Importing data command with the necessary input details, second window within the Importing phase related to the Dataset name. [\[F45\]](#) [\[F46\]](#)

After having confirmed this window as well by pressing the *OK* button, the main EEGLAB interface is updated with the general information relating to the imported trial, which is now ready to be processed. Indeed, from this point, the next step consists of moving to the *Plot* section, where, similarly to the Displaying Raw EEG Signal feature implemented within the GUI of our own application, it is possible to visualise the current EEG signal, including after the filtering and cleaning processes have been applied.



Screenshot of the updated EEGLAB principal window with the updated Dataset information. [\[F47\]](#)

Prior to any signal processing operation, the channel information of the imported dataset was configured through the EEGLAB channel editor (*Edit* → *Channel locations*). This step is required because EEGLAB relies on accurate spatial and referential metadata to correctly interpret and process the recorded signals, and by default, newly imported datasets are assigned generic placeholder labels rather than the actual electrode identifiers. Each channel was therefore manually relabelled according

to the montage adopted, corresponding to the standard nomenclature of the extended International 10-20 System. Once the channel labels had been correctly assigned, the automatic look-up function (*Look up locs*) was employed to retrieve the corresponding spatial coordinates from the standard channel location database embedded within EEGLAB. This procedure populated the Cartesian, polar, and spherical coordinate fields for each channel, enabling correct topographical representation and supporting subsequent spatial-domain operations such as independent component analysis. It should be noted that the automatic look-up function also introduced three additional fiducial points (*Nz*, *LPA*, and *RPA*), corresponding to the nasion and the left and right preauricular landmarks, respectively. As these points do not represent genuine recording channels but are solely used for anatomical reference and head coordinate alignment, they were manually removed from the channel array to preserve the correct two-channel structure of the dataset. Finally, the reference scheme was explicitly declared through the *Set reference* function, in order to document the physical referencing configuration adopted during acquisition. Given the bilateral earlobe placement of the reference and ground electrodes, the reference field was labelled accordingly (*EarRef*). This step, although not altering the recorded data numerically, ensures that the referencing information is correctly stored within the dataset structure, thereby preventing ambiguity in subsequent re-referencing or interpretation stages.

Edit channel info -- pop_chanedit()

Channel information ("field_name"):

Channel label ("label")	FC3
Polar angle ("theta")	-62
Polar radius ("radius")	0.27778
Cartesian X ("X")	0.35964
Cartesian Y ("Y")	0.67638
Cartesian Z ("Z")	0.64279
Spherical horiz. angle ("sph_theta")	62
Spherical azimuth angle ("sph_phi")	40
Spherical radius ("sph_radius")	1
Channel type	
Reference	EarRef
Index in backup 'urchanlocs' structure	
Channel in data array (set=yes)	☒

Buttons on the right:

- Opt. head center
- Rotate axis
- Transform axes
- XYZ -> polar & sph.
- Sph. -> polar & xyz
- Polar -> sph. & xyz
- Set head radius
- Set channel types
- Set reference

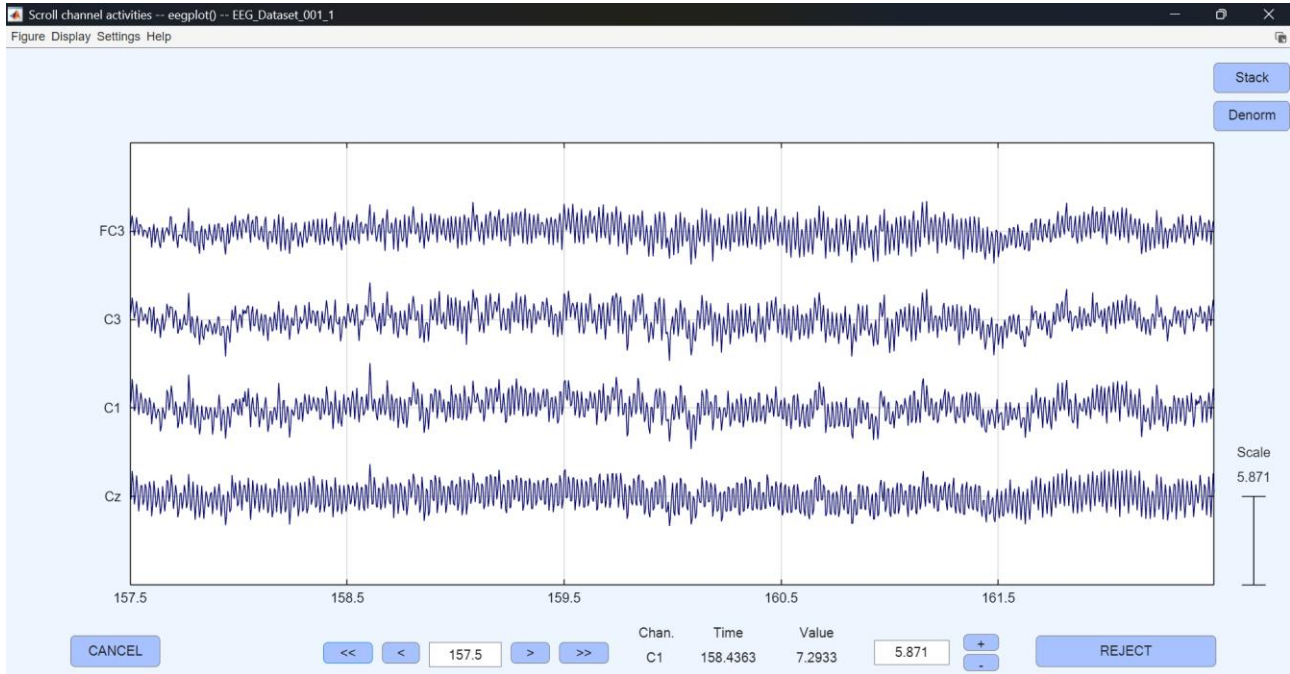
Buttons at the bottom:

- Delete chan
- Insert chan
- Channel number (of 4): 1
- Append chan
- Plot 2-D
- Plot radius (0.2-1, []=auto)
- Nose along +X
- Plot 3-D (xyz)
- Read locations
- Read locs help
- Look up locs
- Save (as .ced)
- Save (other types)
- ☐ Overwrite Original Channels
- Help
- Cancel
- Ok

Edit command window with the necessary input fields filled out.^[F48]

Once the identification of every channel within the system that sampled the provided EEG data has been established, it is necessary, within the main EEGLAB interface, to press the following commands in order to visualise the signal correctly: *Plot* → *Channel data (scroll)* → *Norm* → *Display* → *Remove DC offset* → *Grid* → *X/Y grid on*. Through this workflow, it becomes possible to visualise

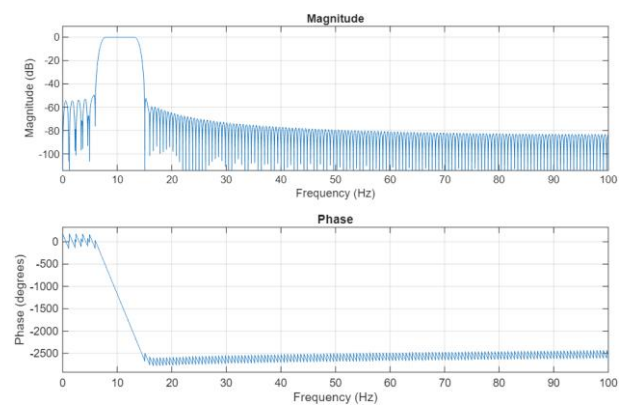
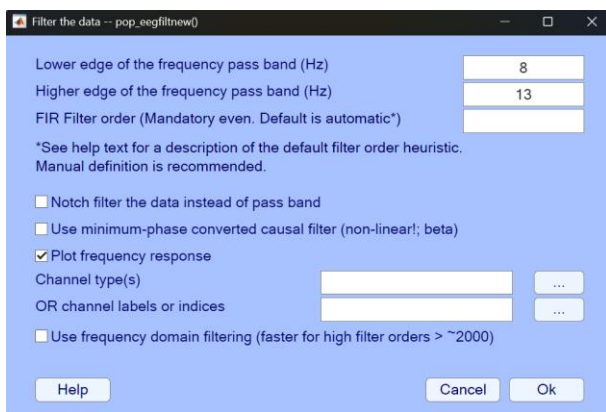
the signals of every electrode aligned rather than stacked. Then, in order to avoid possible stacking throughout the entire duration of the trial, the vertical scale can be adjusted by pressing the +/- buttons within the Values section. Finally, in order to check the overall quality of the current signal, it is possible to navigate along the trial by pressing the </> or <</>> buttons.



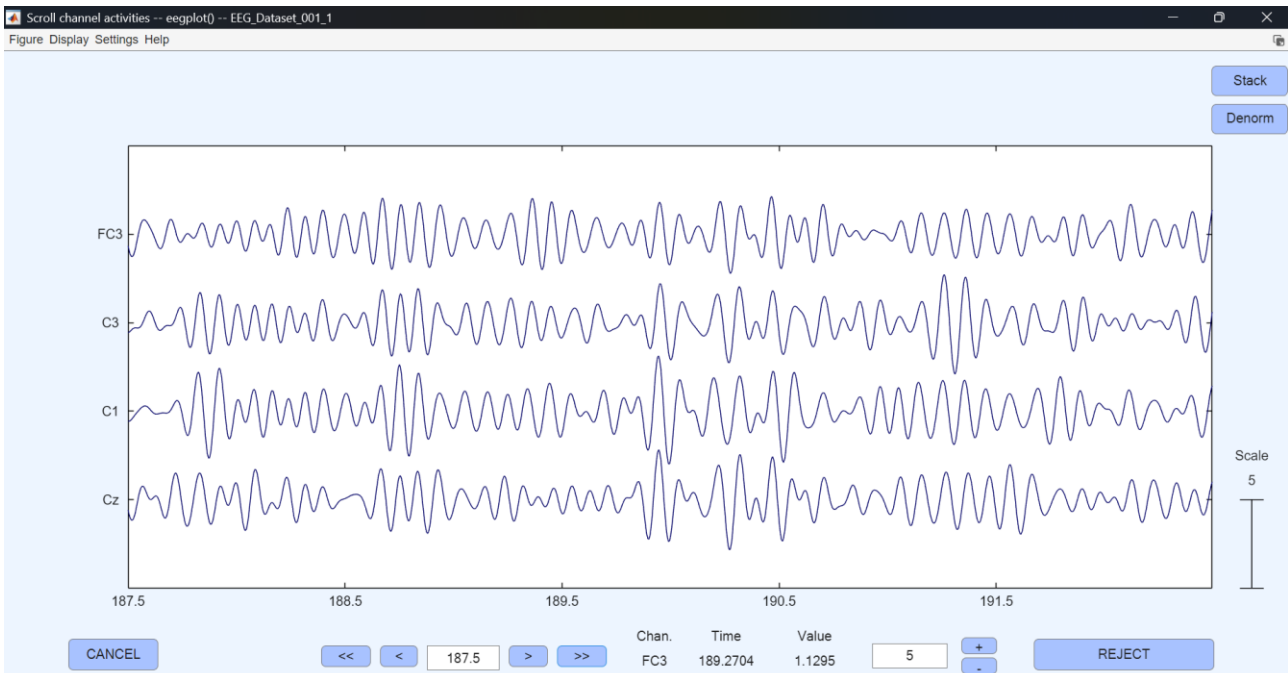
Screenshot of the portrayed signals within the plot feature. [\[F49\]](#)

Following configuration of the channel locations and reference scheme, the raw signal was band-pass filtered between 8 and 13 Hz using the *Basic FIR filter* implementation provided by EEGLAB (*pop_eegfiltnew*), applied across all four channels. This relatively wide frequency range was deliberately preserved at this stage, rather than restricting the signal directly to the mu band, in order to retain sufficient spectral information for the subsequent independent component analysis; narrowing the bandwidth too early would have limited the capacity of the decomposition to correctly separate genuine cortical activity from artefactual sources. The filtered signal was then visually inspected through the EEGLAB scrolling display (*Inspect/reject data by eye*), allowing manual identification of segments affected by gross artefacts such as sudden amplitude spikes, electrode displacement, or signal disconnection. Any such segments were flagged and excluded from further analysis, ensuring that only physiologically plausible activity was retained prior to the decomposition step. *Independent Component Analysis* was subsequently performed using the *Extended Infomax algorithm* (*runica*, with the 'extended' option set to 1), which is well suited to separating sources with both sub-Gaussian and super-Gaussian statistical distributions, a property particularly relevant given the potential presence of both cortical and cardiac (ECG) contamination in the recorded signal, the latter arising from the auricular placement of the reference and ground electrodes. Given the four-

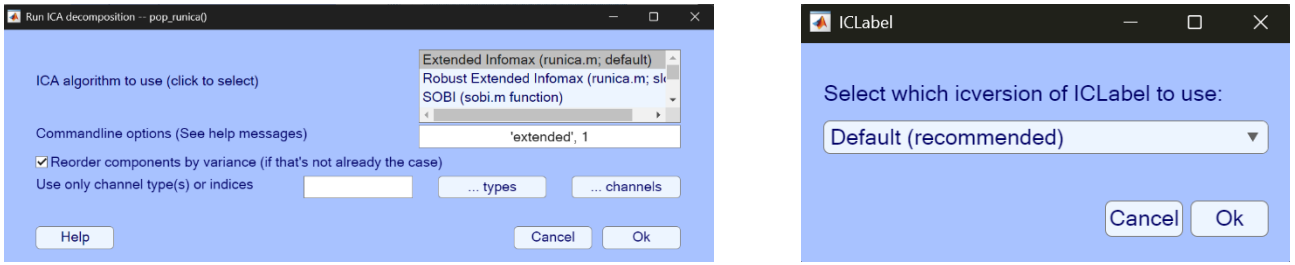
channel montage, the decomposition yielded four independent components, a number constrained by the fundamental limitation of ICA, whereby the number of extractable components cannot exceed the number of available channels. Each resulting component was then automatically classified using the *ICLabel* plugin, which assigns a probabilistic label across seven categories (*Brain*, *Muscle*, *Eye*, *Heart*, *Line Noise*, *Channel Noise*, and *Other*) based on the spatial *topography*, *spectral characteristics*, and *temporal dynamics* of the component. Components identified with a high probability of belonging to a non-cortical category, particularly *Heart*, were earmarked for removal, in order to mitigate the risk of electrocardiographic contamination inherent to the auricular reference and ground configuration adopted in this study.



From left to right: first window of the Basic FIR filter functionality, Output graphs obtained after confirming the settings of the Basic FIR filter. [\[F50\]](#) [\[F51\]](#)

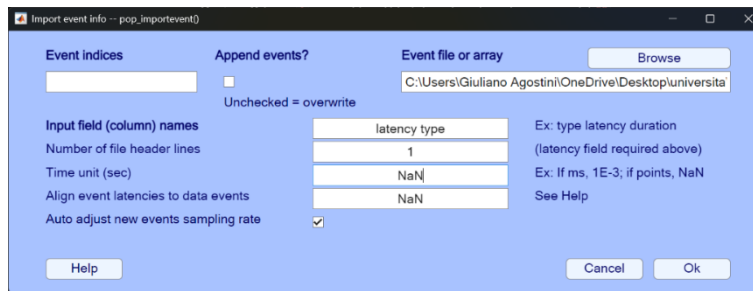


Screenshot of the launched window related to the Inspect/Reject Data by eye feature. [\[F52\]](#)

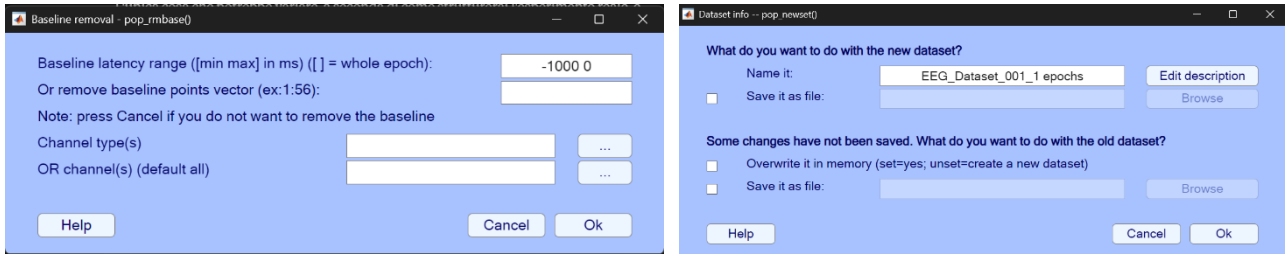


From left to right: First window of ICA analysis, ICLabel Selection window.^{[F53] [F54]}

Once the independent components associated with non-cortical sources had been identified and removed, the continuous signal was segmented into discrete epochs time-locked to the onset of each motor imagery trial, using the *Extract epochs* function (*pop_epoch*). For each of the four task categories (*up, down, left, and right*), an epoch window spanning from one second before to four seconds after the corresponding event marker was extracted, resulting in a total epoch duration of five seconds per trial. This window was selected to encompass both a pre-stimulus baseline interval and the full duration of the motor imagery task, the latter being consistent with the four-second observation window subsequently employed by the real-time classification module during cursor control. Following epoch extraction, a baseline correction was applied through the *Remove epoch baseline* function (*pop_rmbase*), whereby the mean amplitude computed over the pre-stimulus interval, extending from one second before to the onset of each trial, was subtracted from the entire corresponding epoch, independently for each channel. This step is essential for the subsequent *ERD/ERS* analysis, as event-related desynchronisation and synchronisation are conventionally expressed as a percentage change in spectral power relative to a pre-stimulus reference interval. Without this correction, inter-trial and inter-channel amplitude offsets unrelated to the motor imagery task itself could confound the estimation of such power modulations.

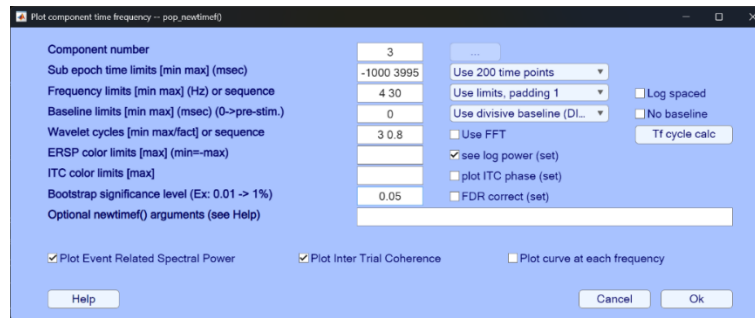


First interface displayed at the beginning of the Extract epochs process, after the selection of the *Import event info* command.^[F55]



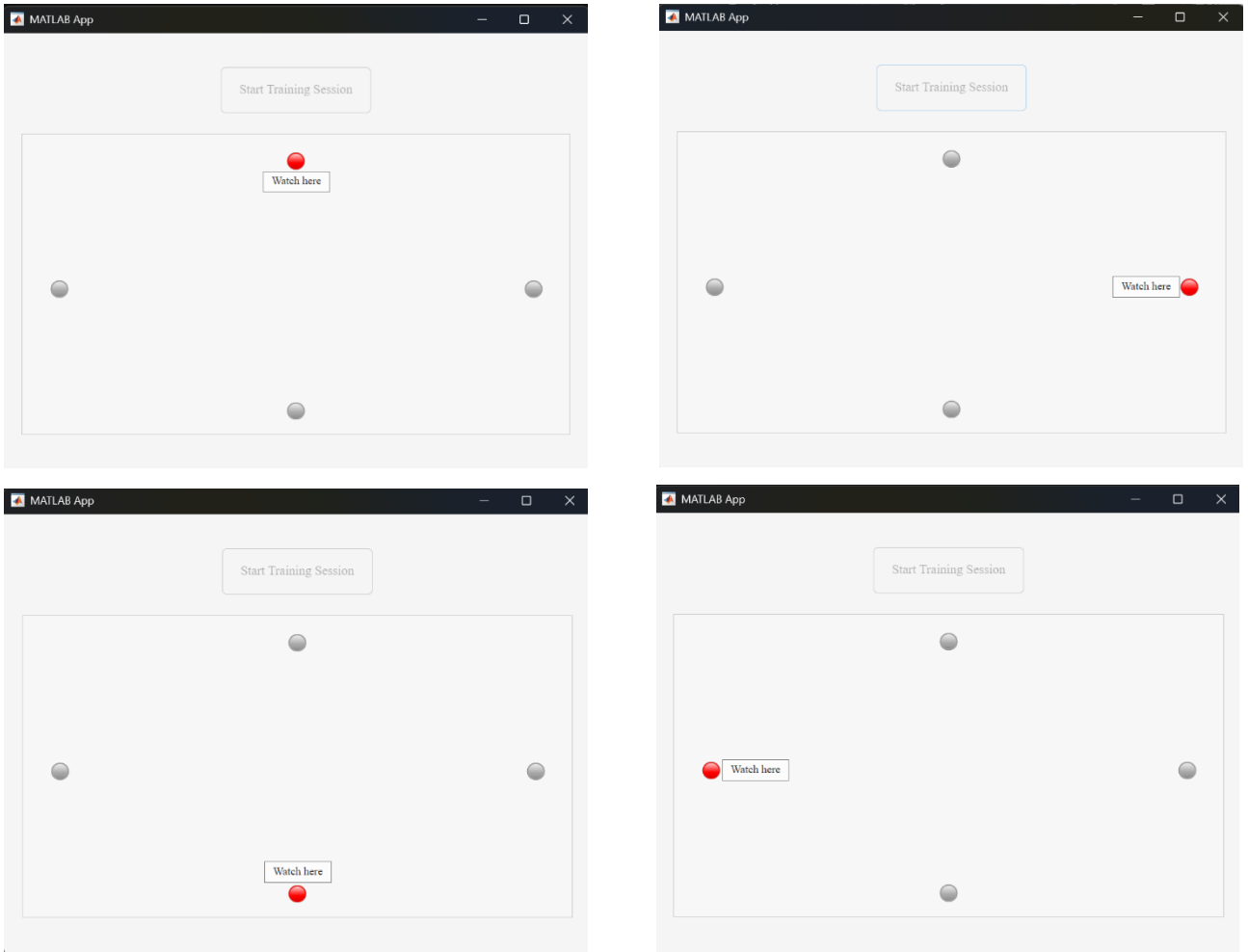
From left to right: First window displayed after the *Remove epoch baseline process*, *Dataset info* window after the completion of the latter process. [\[F56\]](#) [\[F57\]](#)

Once the cleaned and baseline-corrected epochs had been obtained, the time-frequency dynamics of the mu-band activity were examined in order to characterise the *ERD/ERS* pattern associated with each motor imagery task. This analysis was performed through the *Channel time-frequency* function available in EEGLAB (*pop_newtimef*), applied separately to each of the four recording channels. The frequency range was set between 8 and 18 Hz, thereby encompassing the mu band of interest together with adjacent frequency ranges. The baseline period was defined as the pre-stimulus interval preceding the onset of each trial, consistent with the baseline correction previously applied during epoching, while the time-frequency decomposition itself was computed using a variable number of wavelet cycles, increasing linearly with frequency, a configuration commonly adopted to balance temporal and spectral resolution across the analysed frequency range. The resulting output comprises an *Event-Related Spectral Perturbation* (ERSP) plot, expressing the change in spectral power, in decibels, relative to the pre-stimulus baseline, as a function of both time and frequency, alongside an *Inter-Trial Coherence* (ITC) plot, quantifying the consistency of the phase of the oscillatory activity across trials at each time-frequency point. Within the *ERSP* representation, a decrease in power within the mu band following task onset corresponds to event-related desynchronisation (ERD), reflecting cortical activation associated with the execution of motor imagery, whereas a subsequent increase in power, typically observed following task offset, corresponds to event-related synchronisation (ERS), associated with the return to an idle cortical state. It should be noted that the reliability of both the ERSP and ITC estimates is strongly dependent on the number of trials available for averaging. A sufficiently large trial count per task category is therefore required in order to obtain statistically robust and physiologically interpretable time-frequency patterns.



Plot component time-frequency functionality window filled out with the necessary input fields.^[E58]

This latter functionality displayed by EEGLAB represents the last one within the processing sub-environment. Indeed, understanding of the sampled EEG signal, in relation to the neural features analysed within the last figure and obtained following the entire processing pipeline, leads to the next step of the experimentation. This is also the final step relating to the *NIA_BCI_ProcessingAndTraining* environment, and it involves the training of the user, so as to enable the patient to become comfortable with the mechanism that will subsequently be used during the actual control of the digital devices. More specifically, the training phase has been designed so as to reproduce conditions similar to those of the recording phase. Moreover, the fundamental motor imagery movements given to the user are also the same as those that will be used during cursor control, explained in the next part. The training phase begins upon selection of the respective command within the main interface and, as with the recording carried out through OpenBCI, it starts, after a short countdown, with one minute of rest. Following this, the user must follow the activation of the LED displayed within the window. The active part is likewise divided into two sections: in the first, the user follows the LED activation with eyes open, whereas, during the second, the activity is carried out with eyes closed, while the experimenter assists the user by indicating which configuration is to be imagined. Upon completion of the first three minutes, the procedure is repeated a second time, and once this latter phase has also concluded, the experimenter must close the relevant window and, once the patient is ready, repeat the entire pipeline if necessary, so as to achieve, at each round, a clearer and more significant trial, in which the patient develops improved motor imagery skills and the experimenter obtains a more significant and meaningful EEG signal and set of neural features.



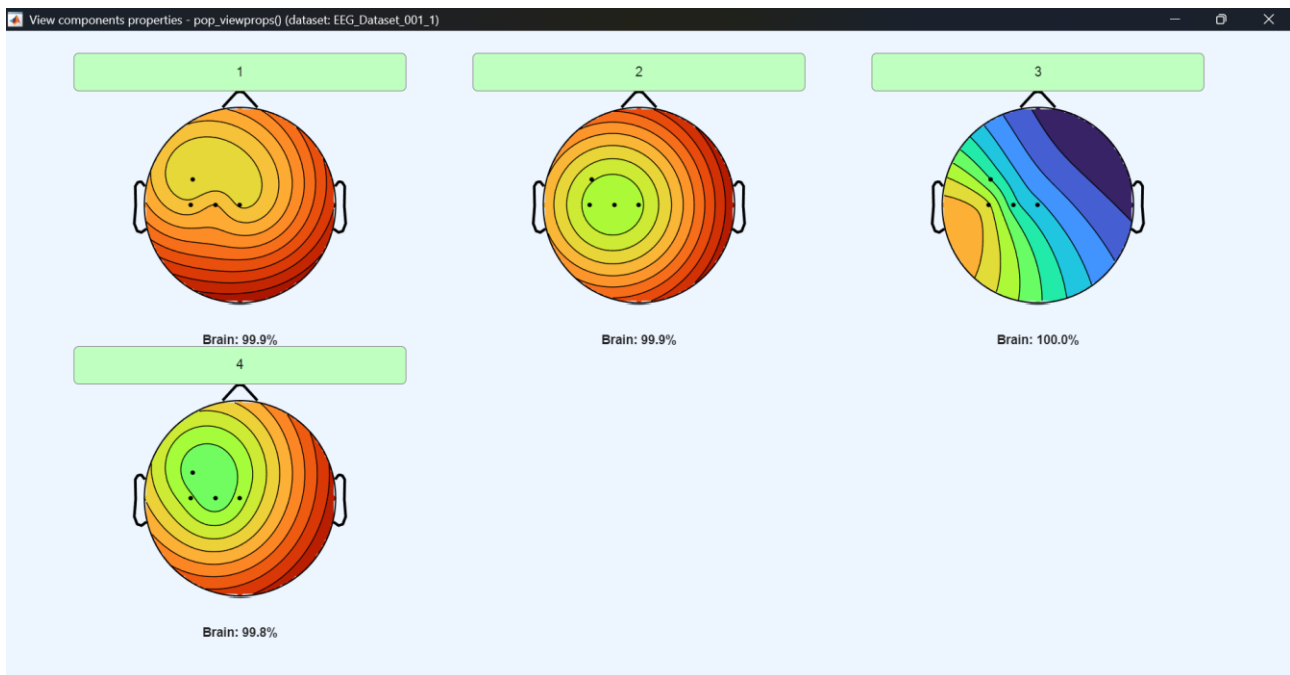
From top to bottom and from left to right: Training window with *UP/Right/Down/Left* commands. [\[F59\]](#) [\[F60\]](#) [\[F61\]](#) [\[F62\]](#)

3.3.2 Results and Considerations from the Trials

The conclusion of the experimentation and the use of the complete NIA_BCI_ProcessingAndTraining sub-environment leads to richer data, strongly linked to the success rate of the application's use. More specifically, the processing phase carried out through the EEGLAB features brings to light graphs and datasets whose analysis is worth noting. These outputs were obtained through the commands available and employed during the experimentation described in the previous chapter. Accordingly, within the current chapter, the final results achieved during the latest phases of EEGLAB processing are revisited and analysed in greater depth than previously, in order to understand the true degree of success reached through the use of NIA for Processing and Training.

The first output graph to be analysed is directly related to one of the steps carried out during the experimentation phase described previously. More specifically, following the use of *Decompose Data by ICA* and the subsequent application of *ICLabel*, EEGLAB provided the images shown below. These relate to the four independent components extracted from the signal sampled at each of the four

electrodes. They are *scalp maps*, showing the weight of each independent component in contributing to the overall signal; the spatial position of the four electrodes used is displayed in the following order: *FC3*, *C3*, *C1*, and *Cz*. Furthermore, the percentage shown below each scalp map corresponds to the probability, calculated by an automatic classifier, that the sampled signal corresponds to cerebral activity. On this basis, it can be observed that components 1, 2 and 4 show a *centred, concentric* pattern, with a maximum localised near the centre of the electrode array, corresponding to a spatially compact pattern without any lateral gradient. By contrast, the third component shows a strongly *lateralised, dipolar* pattern, markedly different from the other scalp maps. Given that the aim is to identify a paradigm confirming the presence of motor imagery activity related to the movement of the dominant hand and arm, a lateralised desynchronisation was expected over the cortical area corresponding to the representation of the dominant hand and arm within the sensorimotor cortex. The gradient shown by scalp map number 3 is therefore compatible with the type of output expected. At the same time, however, it must be borne in mind that this is only a qualitative representation, based solely on the static topography of the sampled EEG signal, as such, it can be regarded as a starting point from which further, more in-depth research may be developed.



Screenshot of the ICLabel Graphs obtained after the *Decompose data by ICA* process at each active electrode. [\[F63\]](#)

The next analysis concerns a further output graph, obtained from an additional EEGLAB functionality outside the conventional processing pipeline presented previously. This represents a more in-depth analysis, based on the quality and quantity of the EEG signal achieved following the processing phase, and ready to be studied more thoroughly. More specifically, the *ERSP* and *ITC* graphs were examined at the level of the *C3* electrode, the most relevant given the representation of the hand and arm within

M1. After launching the *Channel Time-Frequency* command in the *Plot* section, the output produced the image shown below. The upper panel shows the ERSP, namely the power variation, measured in *dB*, relative to the baseline, as a function of time on the x-axis and frequency on the y-axis. The colour bar ranges from blue, corresponding to -1 *dB*, to red, corresponding to $+1$ *dB*. The lower panel, instead, shows the ITC, namely the phase coherence between the instants of the trial, following the same time-frequency structure as the ERSP panel. The majority of the graph appears green, as the output is statistically masked, whereby non-significant areas are forced to zero. This is beneficial for the analysis, as it filters out random noise and highlights the relevant patterns. Considering possible artefacts, dark bands are visible at the extremities of the interval, namely at $-500/0$ *ms* and at $2900 - 3600$ *ms*, at high frequencies around $15 - 30$ *Hz*. These correspond closely to methodological edge artefacts, typical of wavelet decomposition, and have therefore not been considered indicative of genuine neural activity. Moreover, at $2200 - 2500$ *ms*, an anomalous burst is present, visible within the ERP panel as a pronounced peak accompanied by strong ITC. This phenomenon may be associated with an artefact synchronised across the instants of the studied trial, rather than an expected physiological process. By contrast, at $200 - 1400$ *ms*, within the $5 - 10$ *Hz* frequency range, a dark zone is visible, which has been assessed as being strongly related to a genuine physiological pattern, consistent with the preliminary hypothesis. Indeed, this zone corresponds to a decrease in power, indicative of an *ERD* phenomenon within the mu band. This therefore represents precisely the type of desynchronisation expected from motor imagery tasks.

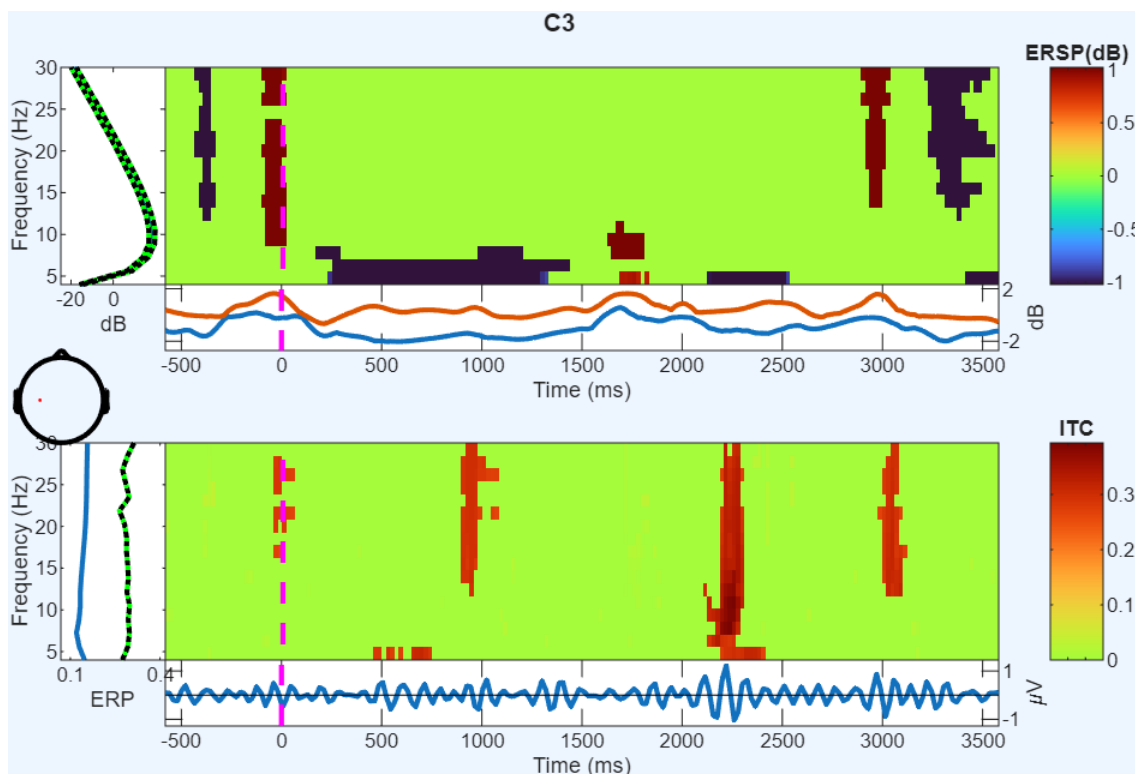


Image obtained by the use of *Channel Time-Frequency* feature at the level of C3 electrode in Plot section. [\[F64\]](#)

The next graph analysed relates to the *Channel Spectra and Maps* command within the *Plot* section, its purpose being to show the *Power Spectral Density* (PSD), expressed in *dB*, as a function of *frequency*, with a distinct curve for each electrode, together with a black curve representing the cross-channel average. The five scalp maps at the top show the spatial distribution of spectral power at the selected frequency, which, in this case, is *10 Hz*, as it is centred within the mu band under study. It can be observed that all the curves show a distinct plateau/peak within the mu band interval. This confirms that the dominant oscillatory pattern of the sampled EEG signal is compatible with mu-band activity across the spatial configuration used.

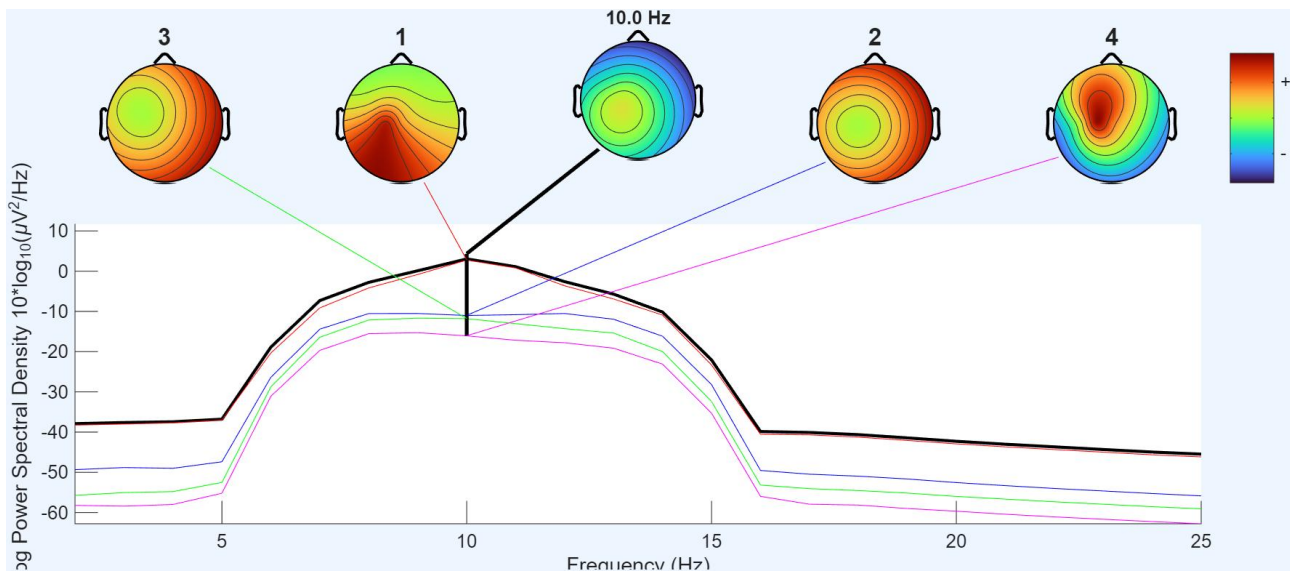


Image obtained through *Channel Spectra and Maps* feature centred at 10 Hz in *Plot* section. [\[F65\]](#)

Finally, the last output analysed within this chapter aims to highlight neural features not strictly limited to the *ERD/ERS* phenomena studied and expected in the previous analyses. In this case, the *Channel ERPs* command in the *Plot* section was used, in order to obtain a decomposition of the average response into the independent components contributing most significantly to it. The black trace represents the observed ERP, the coloured traces represent the contribution of each ICA independent component, and the magenta trace represents the sum of these components. With regard to the values shown, the *ppaf* of *100.00%* can be highlighted, corresponding to a variance value of 100%. This arises not from any genuine selection of the most significant components, but from the fact that, given the limited number of components available, all four have simply been combined. At the same time, a progressive growth in amplitude can be observed after $t = 0$, with the oscillations increasing markedly. This is consistent with the expected behaviour of *ERD/ERS* phenomena, since the pre-stimulus baseline should be relatively quiet, and the growth in amplitude observed over time is compatible with the mu-power modulation being sought. Moreover, at $t = 2.2 - 2.3$ s, a peak in amplitude is visible, driven primarily by *IC1*, which may correspond to the same anomalous burst

previously observed in the time-frequency graphs. This represents a cross-verification, highlighting a convergence of evidence pointing to a specific portion of the trial that could be removed or cleaned more thoroughly, in order to obtain a more stable and meaningful dataset.

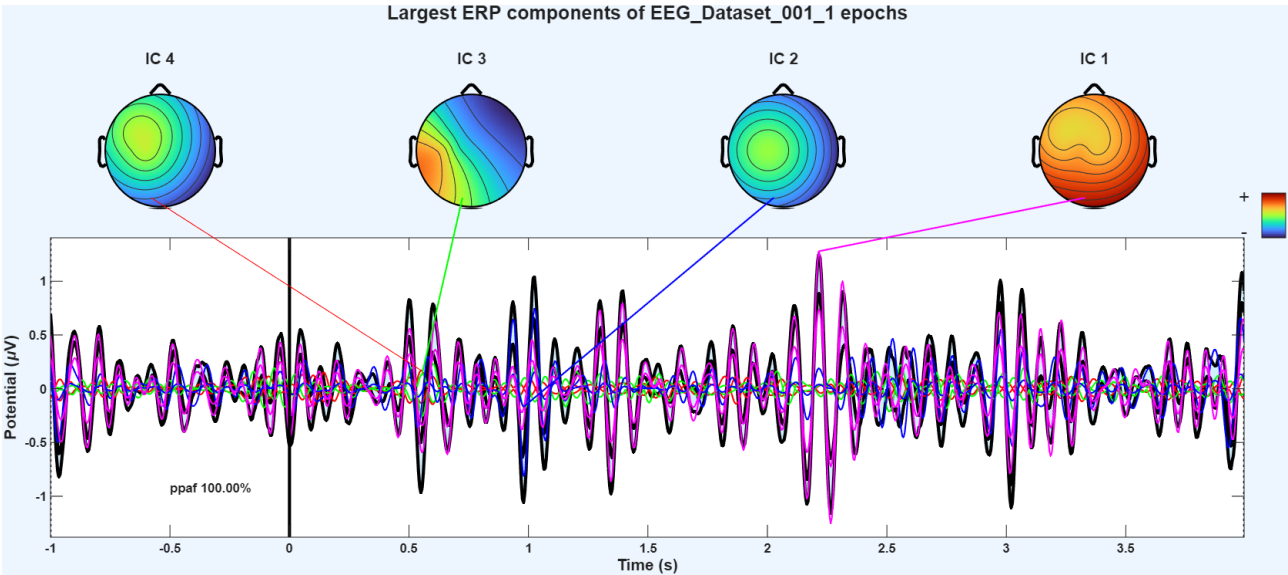


Image obtained through *Channel ERPs* feature in Plot section.[\[F66\]](#)

SECOND PART

4. NIA for Real-Time Control

The second part of the project consists primarily in the development of the second core component of the NIA environment, namely the *NIA_BCI_RealTimeControl* application. The expansion of the environment is not limited to a single piece of software, since the underlying aim of the entire project is the creation of an integrated environment shaped around the purposes and characteristics of *Brain-Computer Interfaces*. The scope of this field extends well beyond the results obtained through the use of the previously described environment, which is instead dedicated to the offline analysis of EEG signals and the training of the classifier. Whereas the MATLAB application is concerned with user training and the development of the classification model, the Python application performs *real-time acquisition, processing and classification* of EEG signals, translating the motor imagery intended by the user into cursor movement commands on screen, according to a four-class paradigm (*up, down, left, right*) analysed through event-related desynchronisation and synchronisation (*ERD/ERS*) in the μ band (8–13 Hz). The ultimate aim of a BCI is to establish a new pathway where the capacities of the user cannot otherwise be reached, owing to pathologies or motor impairments. From this awareness stems the idea of developing a second half of the targeted product, upon the foundations laid by the first. It is therefore necessary to create a standalone, independent and stable piece of the environment in which the knowledge acquired through the use of the first half of NIA is put into practice, performing the tasks of *connection, calibration and control*. These concepts represent the basis of *NIA for Real-Time Control*, the second and final component of the NIA environment, in which the fundamental purpose of a BCI becomes reality: enabling the user to freely control the computer cursor through motor imagery, while retaining the availability of the conventional mouse and trackpad.

Firstly, following analysis of the results and consideration of the use of *NIA for Processing and Training*, attention turned to the identification of new working methods. Indeed, the development of the second application required new programs and digital tools, marking the definition of the new purposes and goals that the final system was intended to achieve. With this in mind, the creation phase began, in which the selection of the necessary tools and programs proved essential in order to enable the subsequent phases in which the actual development took place.

Following the initial phases, the creation of the application began. This involved the development of specific digital components of the overall application, such as *files and folders*, modified and created

through dedicated coding programs. Each element represented a step forward in the practical functioning of the application. Establishing the relations and the workflow pipeline therefore became essential.

Finally, following the verification and checking of every function within the newly created application, the establishing phase took place, in which the functioning of the application, together with each of its features, was verified and controlled. This phase led to the definition of the *criteria* and the *logic* underlying the transduction of neural signals through the *back-end* code of the application. These elements were subsequently linked to the *front-end* experience of the user and the experimenter, in order to verify whether the created environment was functional and ready for use in the final experimental procedure. This stage was aimed at ensuring the correct functioning of the application, allowing the last aspects of the application to be corrected and adjusted. Moreover, during this phase the sampling of the *results* derived from the functioning of the application began, in order to establish whether the core concept of the BCI field, namely the establishment of a *new pathway* between the *CNS* and the *digital device*, had been achieved.

4.1 Target Definition and Starting Phase

Since this new part of the project involved the creation, from scratch, of an application different from the one previously developed, in terms of development *environment*, *features*, *functionalities*, *methods of communication* with the hardware and overall purpose, it was necessary, before properly beginning the working phase, to establish a stable theoretical foundation. It is fundamental to identify the tools and the related objectives to be achieved during the development and creation phase. Therefore, within the following sub-chapters, the methodologies used to define the desired final environment and the necessary tools will be presented, in order to prepare the digital device for the development phase. In conclusion, the reader will thereby acquire the conceptual basis needed to understand what is created through this specific application, and moreover the reasons underlying it.

4.1.1 Tools and Purposes

The initial phase of the development of this part of the environment involved several steps aimed at preparing the devices to effectively create and develop the software from scratch. Therefore, the preparation of the digital tools, together with the definition of their purposes, constituted the first stage of this phase. The subsequent steps involved the verification and preparation of the devices, limited to the installation of the tools necessary for the programming phase, for the programming language employed, and for the creation and activation of the virtual environment. Moreover, it was necessary to verify the version of the libraries to be used and their respective support tools. Finally,

the last step of the preliminary stage consisted in verifying the connection between the hardware component and the digital devices, by attempting to establish a connection through dedicated scripts. The environment generally used for these stages was essentially the native one, such as the *cmd*^[48] window, together with specific development applications such as the previously mentioned *VSC* and the relative online documentation.

First of all, to proceed in more detail, the verification of the programming language was carried out. Indeed, the development of *NIA_BCI_RealTimeControl* involved the use of *Python 3.11.9*, and to check the correspondence, the command *python -version* was used in the *cmd* window. This allowed the workflow to continue with the assurance that the desired version of Python was in use, guaranteeing an updated and functional software foundation. Subsequently, still within the *cmd* window of the personal computer, it was necessary to create the virtual environment where the actual development took place. To this end, the dedicated *bci_env* environment was created through the command

python -m venv bci_env. The environment was then activated, in order to work properly within it, through the command *bci_env\Scripts\activate*. Confirmation of the activation was given by the presence of the label "*bci_env*" at the beginning of the prompt, which allowed the subsequent installation of the required programs in the correct location. Indeed, the next step consisted in identifying the desired programs, tools and libraries to be used in the development of *NIA_BCI_RealTimeControl*, in order to install them through the *pip* command. However, before proceeding with this, two preliminary steps were necessary. The first consisted in updating the package manager through the command *python -m pip install --upgrade pip*. The second concerned the main library underlying the software: *BrainFlow*. Indeed, this library is fully compatible with the OpenBCI environment, and its compatibility with the Python programming language made the workflow entirely integrable and functional. Therefore, after installing *BrainFlow* through the relevant online resources, using the command *pip install brainflow==5.22*, it was possible to make use of its functionalities. The next step consisted in defining the necessary libraries (aside from *BrainFlow*) and their respective support requirements for correct operation. The installed libraries were as follows:

- *PyQt5 5.15.11*: the framework used for the construction of the graphical interfaces
- *matplotlib 3.11.0*: used for the rendering of the EEG graphs embedded within the PyQt5 windows through *FigureCanvasQTAgg*
- *scipy 1.17.1*: used for the estimation of the power spectral density through the Welch method
- *numpy 2.4.6*: used for matrix operations and feature saving in *.npy*^[49] format

- *scikit-learn 1.9.0*: used for LDA classification with stratified cross-validation and *LabelEncoder*
- *joblib 1.5.3*: used for the serialisation and deserialisation of the trained model
- *PyAutoGUI 0.9.54*: used for the programmatic control of the cursor

This installation phase was carried out in parallel with the installation of the support libraries, through the command *pip install PyQt5 matplotlib scikit-learn joblib pyautogui* in the *cmd* window. Finally, the last step consisted in verifying the libraries just installed, through the commands *pip list* and *pip show Brainflow*, together with the verification of the connection of the board with *python name_of_the_test_script.py*.

In conclusion, the steps described in the preceding paragraphs made it possible to establish the foundations on which the proper development of the application was subsequently based.

4.1.2 Creation of the Environment

The core components underlying the application *NIA_BCI_RealTimeControl* have been developed separately, in order to assign to each of them a specific purpose and method of operation. In this way, the final environment has resulted in a multi-modular structure, composed of smaller components that collectively perform the specific tasks required for the functioning of the final application. Therefore, the steps involved in this phase of development consist essentially, in the first instance, in establishing the elements of the application within the digital environment previously created, and subsequently, in creating each command underlying the front-end functionalities of the application.

First of all, a dedicated folder was created with the purpose of containing every file and program related to the future software. The folder was named after the application itself, *NIA_BCI_RealTimeControl*, and inside it, following the creation of the digital environment described in the previous sub-chapters, the sub-folders resulting from the preliminary phases were placed. Indeed, the establishment of *bci_env* involved the creation of the "*bci_env*" folder, which contains the relevant folders related to the scripts and libraries underlying the internal functioning of the software. Alongside this main folder, two further sub-folders are present, essential for the functioning of the software, whose purposes will be explained in the following chapters. These folders are *pycache*, which contains the *.pyc*^[50] files related to the execution of the Python programming language in the back-end of the software, and *Calibration_data*, which contains the *.pkl*^[51] files related to the Calibration feature of the final application, explained later. Moreover, alongside the main folders just described, the *NIA_BCI_RealTimeControl* folder contains the programs specifically developed in VSC, corresponding to the previously mentioned components of the application. The adoption of a

modular structure allows the control of each functionality within a specific, restricted script, which can be modified and optimised through direct coding actions. The components contained in the *.py* files, corresponding to the front-end functionalities that will be presented in the following chapters concerning the use and logic of the application, are:

- *main.py*: the starting point and access point of the application, which coordinates the start-up, initialisation, connection to the Ganglion board through BrainFlow, and the coordination of the other modules of the application
- *signal_visualisation.py*: the program dedicated to the real-time visualisation of the EEG signal acquired by the Ganglion board, useful for checking the quality of the signal prior to the subsequent processing phase
- *monitor_gui.py*: the program that creates the interface used during the monitoring of the hardware system
- *calibration_gui.py*: the program that creates the interface used during the Calibration functionality, in which the EEG signals are sampled and stored in order to build the training dataset
- *feature_extraction.py*: the sub-program responsible for extracting the neural features from the sampled EEG signal during the classification process
- *classifier.py*: the component that implements the classifier based on *Linear Discriminant Analysis* (LDA), in order to discriminate between the four classes of motor imagery required for effective cursor control
- *cursor_gui.py*: the final script of the internal application, which essentially composes the interface used to start and monitor the general purpose of the application, namely the control of the virtual cursor by the user operating the hardware system

In conclusion, the creation of each individual *.py* file was followed by the writing phase of the scripts, defining the internal logic of the functionality just created and its relations with the other programs of the overall application. This phase therefore involved analysing each script and identifying the correct solution to properly process the EEG signal in order to extract the features useful for controlling the virtual cursor. These steps will be explained in detail in the following chapters, corresponding to the explanation of the functioning and the relations of each file.

4.2 Functioning Flow and Verification of the Application

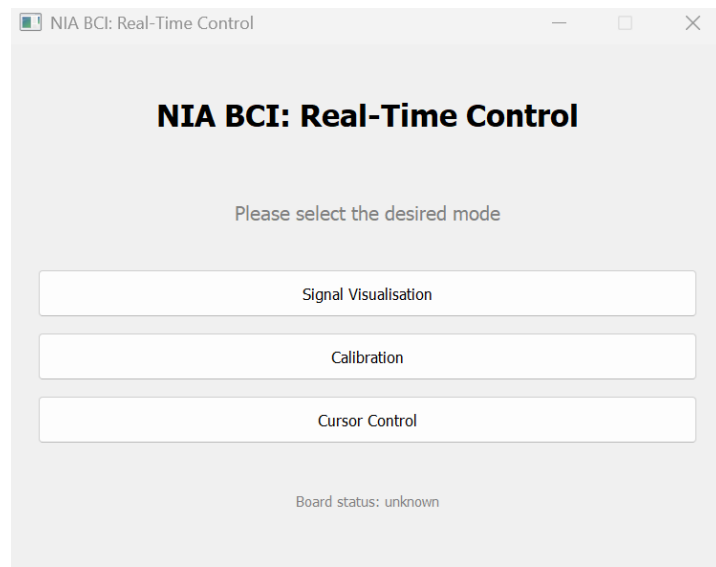
The proper creation of an application, following the definition of its purposes, the necessary digital tools for the programming phase, and the components of the final environment, requires the definition of every specific feature within each file involved in the functioning of the software. Therefore, in order to build the final interface that the user will employ during the experimentation, it is necessary to analyse every file, defining its internal logic and code functioning, its front-end aspects, and finally its relations with the other files within the general environment. These phases constitute the effective realisation of the application, going far beyond the simple definition of the components introduced in the previous sub-chapters. In this section, each file previously introduced will be explained, in order to provide the reader with the theoretical knowledge underlying every front-end component of the *NIA_BCI_RealTimeControl* interface. In this way, it will be possible to understand the characteristics of the internal code and the reasoning behind it, and subsequently to relate them to the functioning of the other files that collaborate with the one in question to accomplish a specific task within the workflow pipeline. In relation to this, the pipeline of the application will then be presented, in order to understand the correct use of the application and the correct relationship between the hardware and software components.

4.2.1 Relations and Connection Between Elements

The interactable interface of *NIA_BCI_RealTimeControl* is composed of several elements, each corresponding to a *.py* file developed within the *VSC* environment. Owing to the modular structure, a main file coordinates the other functionalities, each governed by code aimed at establishing a specific command together with its supporting sub-functionalities, contained in separate files.

The first file introduced is *main.py*. Developing a standalone application requires, alongside the creation of the virtual environment, a main window that serves as the starting point of the user experience, from which the other commands, product information, and the hardware connection status can be reached. The internal code of this file is aimed solely at creating the principal window, without directly managing the sampled EEG signal. The first block imports the classes and methods required to build the window: `import[52] sys / from PyQt5.QtWidgets import (QApplication, QWidget, QVBoxLayout, QPushButton, QLabel) / from PyQt5.QtCore import Qt`. The latter is followed by the imports establishing the relations with the other functionalities: `from monitor_gui import MonitorWindow / from calibration_gui import CalibrationWindow / from cursor_gui import CursorWindow`. The class defined through the class command builds the main window using these imports, with `init[53]` and `_build_ui()` constituting the actual interface, supported by the methods

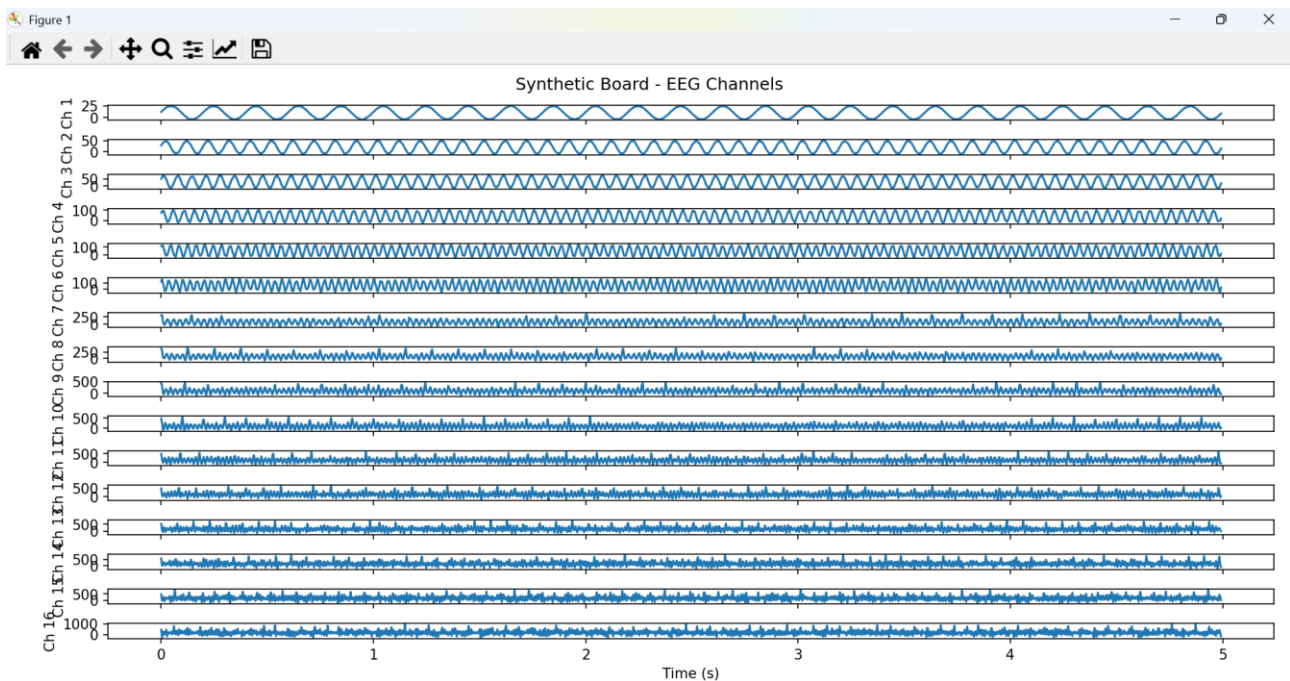
`_set_status_connected()` and `_set_status_disconnected()`, which update the status label accordingly. Once the interface is built, the three core methods `open_monitor()`, `open_calibration()` and `open_cursor_control()` are linked to their respective buttons. Each follows the same logic: creating the corresponding window with `use_synthetic=False`, so as to use the real Ganglion board rather than a synthetic test signal, and then checking the `connection_failed` attribute of the newly created window. Each functionality thus independently verifies its own connection to the board and exposes this status to the main interface. The code ends with the standard Python entry point, `if name == "main":`, which instantiates `QApplication` and enters the event loop that keeps the application running until it is closed. In terms of its relations with the other files, `main.py` acts as the parent file, orchestrating the opening and closing of the others.



Principal interface of `NIA_BCI_RealTimeControl` within the `main.py` file. [\[F67\]](#)

The next file, `signal_visualisation.py`, is the first one linked to a button of the main interface. It imports `BoardShim`, the principal class managing communication with the EEG board, `BrainFlowInputParams`, the container for the connection parameters, and `BoardIds`, used to identify the supported acquisition board. The file defines the classes and methods that communicate directly with BrainFlow to visualise the signal sampled by the Ganglion board. Its core is the class `EEGAcquisition`, which models the entire acquisition process. The constructor `init(self, use_synthetic=True, serial_port=None)` creates the parameter object `self.params = BrainFlowInputParams()`, selects the board through the conditional `if use_synthetic: ... else: ...`, and instantiates `self.board = BoardShim(self.board_id, self.params)`, which manages communication with the board. It also retrieves the sampling frequency and the indices of the EEG channels in use, and initialises the Boolean flag tracking whether streaming is active. The method `connect(self)` opens the session through `self.board.prepare_session()`, while `start_stream(self)` starts continuous

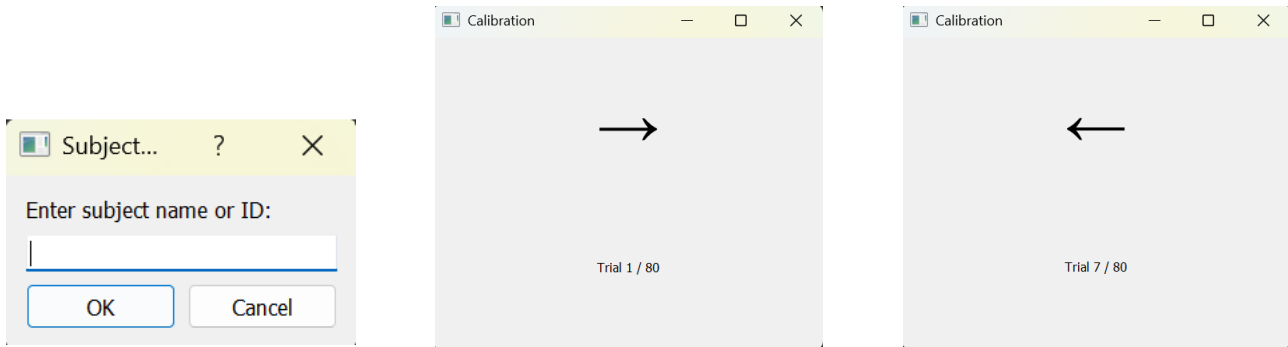
acquisition and sets *is_streaming* to True. The method *get_latest_data(self, n_samples)* returns the latest *n_samples* in the buffer through *self.board.get_current_board_data(n_samples)*. Crucially, this method reads the buffer without clearing it, unlike other BrainFlow methods such as *get_board_data()*, which is the correct approach for continuous real-time visualisation, allowing the graph to be repeatedly updated without discarding data still required for subsequent processing. The method *stop_stream(self)* stops streaming only if it is currently active, avoiding redundant calls on a board that is not acquiring data, and resets the flag to False, while *disconnect(self)* closes the session and releases the occupied resources through *self.board.release_session()*. Regarding its relations with the other files, *EEGAcquisition* is the shared class underpinning not only signal visualisation but also the calibration session and cursor control.



Interface displayed after the launch of the signal visualisation functionality through *signal_visualisation.py* file. The portrayed image is only an example, with input the data from the synthetic version of the OpenBCI Ganglion Board. [\[F68\]](#)

The following functionality, *calibration_gui.py*, is responsible for sampling the EEG signal required to train the classifier subsequently used for cursor control, implementing an entire experimental paradigm for the study and acquisition of motor imagery. Its interface presents the user with a pattern of visual stimuli, similar to those already implemented in the training section of *NIA_BCI_ProcessingAndTraining*. Unlike the latter, however, it introduces a randomised presentation order, extracts the corresponding features, trains the LDA classifier, and saves the resulting model to a dedicated folder. The paradigm is defined by four global variables, up, down, right and left, each linked to a graphic on the interface, with a two-second fixation phase followed by a four-second stimulus presentation, while the number of trials per class is set through the variable

TRIALS_PER_CLASS. The *init* method initialises the session by first requesting the subject's ID through a dedicated dialog, then establishing the connection with the board; once the connection succeeds, the session begins with a randomised trial sequence, ensuring the subject cannot anticipate a predictable pattern. In parallel, *collected_data* is initialised to store, for each trial, a dictionary containing the corresponding EEG data, while the exact number of samples per trial is calculated by multiplying the sampling frequency by the duration of the stimulus phase, before the interface is built and the first trial started. The interface itself, defined in *_build_ui*, is deliberately minimal: a single large central label displays both the fixation cross and the directional arrow, accompanied by a smaller label showing the session's progress (for example, "Trial 5/80"). The logical core of the file lies in three interlinked methods implementing the temporal sequence of each trial, following a cue-based paradigm typical of motor imagery experiments: *_start_next_trial* checks whether all trials have been completed, *_start_cue_phase* determines the class assigned to the current trial from the randomised sequence and displays the corresponding arrow, and *_collect_trial_data* performs the actual acquisition, retrieving from the buffer the number of samples corresponding to the stimulus duration through the *get_latest_data* method of *EEGAcquisition*. Once all trials are complete, a dedicated method coordinates the entire post-processing phase: streaming is stopped and the board disconnected, the *calibration_data* folder is created if not already present, and a unique file name is generated by combining the subject's name with a timestamp. From this newly saved file, the method sequentially calls the functions imported from the other application modules: *extract_features_from_session*, which extracts the neural features from the raw data, *save_features*, which saves them to disk, *evaluate_classifier* and *train_classifier*, which respectively assess through cross-validation and train the LDA classifier on the extracted features, and *save_model*, which saves the trained model together with the associated *LabelEncoder*. It is at this stage that *calibration_gui.py* most clearly acts as an orchestrating module, automatically launching the entire training pipeline rather than merely collecting data. Where cross-validation is possible, the average accuracy is displayed; otherwise, the user is prompted to complete twenty trials per class to obtain a reliable estimate. The file also implements *closeEvent*, which, provided the connection has not failed and streaming remains active, stops it and disconnects the board before the window closes, ensuring hardware resources are correctly released even if the window is closed manually before the session ends naturally, together with a standalone test block (*if name == "main":*, run via python *calibration_gui.py*) allowing the synthetic board to be enabled or disabled for testing without hardware. In terms of its relations with the other files, *calibration_gui.py* depends on *signal_visualisation.py*, *feature_extraction.py* and *classifier.py*.



Images taken from a calibration session within the application during the use of *calibration_gui.py*, *feature_extraction.py* and *classifier.py*. From left to right: initial interface where it is asked the subject code, window that displays the command "right", window that displays the command "left". [\[F69\]](#) [\[F70\]](#) [\[F71\]](#)

The next file analysed, already mentioned in the previous paragraphs, is responsible for the hidden operations directly related to the interpretation of the EEG signal. The file in question, *feature_extraction.py*, has as its essential task the transformation of the raw EEG signal into a set of numerical features usable by the classifier. The underlying principle is the detection of *ERD* during motor imagery, reflected in the spectral power within the $8\text{-}13\text{ Hz}$ frequency band over the M1, relative to a resting state. The file therefore extracts, for each trial and for each channel, the average power within the mu band, building the feature vector subsequently provided to the *LDA* classifier. Regarding the code itself, the first step consists in importing *numpy* and *welch*, followed by the definition of the constant delimiting the *mu* band frequency range. The computation of the band power is handled by the function *compute_band_power*, in which *welch()* is invoked to estimate the *power spectral density* (PSD) of the signal: the Welch method divides the signal into overlapping segments, computes the *Fourier transform* of each, and averages the results, thereby yielding a more stable and less noisy estimate than a single FFT computed over the entire signal. The parameter *nperseg=sampling_rate* sets the length of each segment equal to the sampling frequency, so as to achieve a frequency resolution of 1 Hz. Once the frequencies (*freqs*) and the corresponding power spectral density (*psd*) have been obtained, the function builds a Boolean mask (*band_mask*) that selects exclusively the frequency bins falling within the lower and upper limits of the band passed as a parameter. The band power is then calculated as the mean of the PSD values selected by this mask, and returned as a single scalar value. Following this process, the actual feature extraction is carried out by the function *extract_features_from_session*, which receives the path of the *.pkl* file saved by *calibration_gui.py* and loads it through *pickle.load*, obtaining the session dictionary containing the list of trials, the indices of the EEG channels used, and the sampling frequency. Two empty lists, *X* and *y*, are then initialised, storing respectively the feature vectors (one per trial) and the corresponding class labels. For each trial in the list, the label and the associated raw EEG data are extracted. For each EEG channel indicated in *eeg_channels*, the corresponding signal is extracted from the data, and

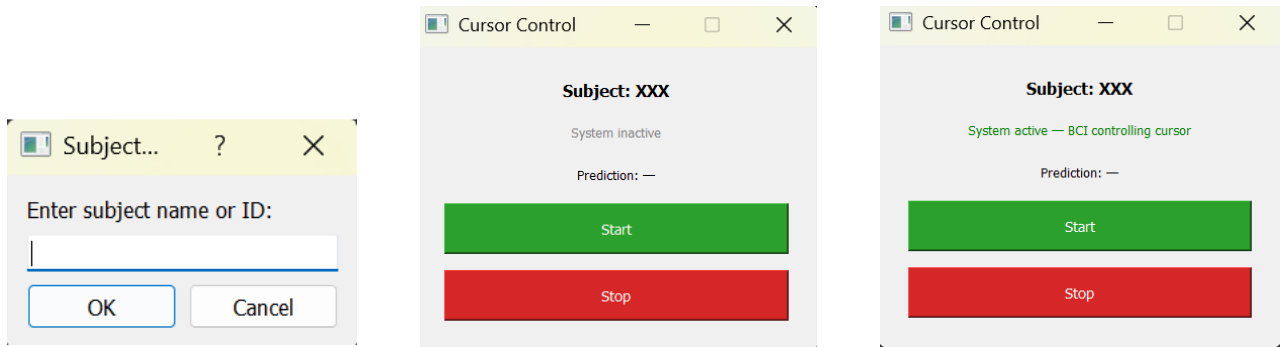
compute_band_power is invoked on it, restricted to the mu band, yielding a single power value per channel. These values are accumulated in *feature_vector*, which, once completed, is appended to the list X, while the trial label is appended to the list y. Finally, the extracted features are saved through the function *save_features*, which writes the feature matrix and the label vector to disk as two separate files in *.npy* format. As with the other files already introduced, a standalone test block is also present, allowing the feature extraction process to be executed independently. Regarding its relations with the other files, *feature_extraction.py* depends on the data produced by *calibration_gui.py*, and is in turn fundamental to the input required by *classifier.py*.

The file mentioned at the end of the previous paragraph, *classifier.py*, is the one analysed now. It closes the training pipeline initiated by *calibration_gui.py* and continued by *feature_extraction.py*. Its inputs are the extracted EEG features and the corresponding labels, used to train an LDA-based classifier capable of distinguishing the four classes of motor imagery, to evaluate its performance through cross-validation, and finally to save the trained model. The import phase includes *numpy*, as in the previous files, together with *LinearDiscriminantAnalysis*, *cross_val_score*, *StratifiedKfold* and *LabelEncoder* from *scikit-learn*, as well as *joblib* and *os*. The feature loading is handled by the function *load_features*, which loads from disk the two *.npy* files produced by *feature_extraction.py*, namely the feature matrix X and the label vector y, reconstructing their file names from a common base path (*base_path + "_X.npy"* and *base_path + "_y.npy"*). The actual training is carried out by the function *train_classifier*, which takes X and y as input and returns the trained classifier together with the corresponding encoder. The first step consists in encoding the labels: since the classes are expressed as strings ("up", "down", "left", "right"), a *LabelEncoder* is used to convert them into integer values (for example, "down" → 0, "left" → 1, "right" → 2, "up" → 3), assigned in alphabetical order, a necessary step, as *scikit-learn* requires numerical labels as input. The *LinearDiscriminantAnalysis* classifier is then instantiated and trained on the entire available dataset through *clf.fit(X, y_encoded)*. Unlike the evaluation phase, no train-test split is performed here, since the goal is to produce the final model to be used operationally, thereby exploiting all the available calibration data. A summary message is then printed, reporting the number of trials used, the number of features per trial, and the recognised classes, after which the function returns both the trained classifier (*clf*) and the encoder (*le*), the latter being required later to convert the numerical predictions back into their original labels. The next function, *evaluate_classifier*, assesses the reliability of the classifier before it is considered ready for use, through stratified cross-validation. Once this process is complete, the model is saved through *save_model* and *load_model*: the former uses *joblib.dump* to serialise to disk, within a single file, both the trained classifier and the corresponding *LabelEncoder*, packaged together in a dictionary, ensuring that model and encoder remain permanently associated

and consistent with one another, thereby avoiding the risk of later pairing the model with a mismatched encoder. The symmetrical function *load_model* performs the reverse operation, loading the file through *joblib.load* and returning the classifier and encoder separately, ready to be reused, most likely by *cursor_gui.py* during real-time operation. Finally, as with the previous files, a standalone test block (*if name == "main":*) allows the entire evaluation and training process to be executed independently. Regarding its relations with the other components, *classifier.py* depends strongly on *feature_extraction.py*, is invoked within *calibration_gui.py*, and its behaviour is essential to the final steps carried out by *cursor_gui.py*.

The final purpose of controlling the on-screen virtual cursor is achieved starting from *cursor_gui.py*. The general purpose of this component is the effective control of the cursor through the previously developed system, by analysing the EEG signal in real time. This file uses the model produced by *calibration_gui.py* and *classifier.py* to classify the incoming signal, continuously updating the classification of each motor imagery task performed by the user. Besides the standard modules and *PyQt5*, the *pyautogui* library is imported, allowing programmatic control of the on-screen cursor. Two fundamental constants are then defined, *CURSOR_STEP* and *WINDOW_SECONDS*, alongside the safety mechanism *pyautogui.FAILSAFE = True*, which immediately interrupts the automated cursor movement whenever the user moves the mouse to a corner of the screen, allowing manual control through the trackpad or mouse to be regained at any time. As already seen in *calibration_gui.py*, the first step is the identification of the subject through a dialog window. In this case, however, if the user cancels the input or leaves the field empty, the window immediately sets *connection_failed = True* and interrupts initialisation, without resorting to a placeholder identifier, a logical choice, since here the subject's name is not used to label new data, but to retrieve an already existing model, and without a valid name the process cannot proceed. Once the subject's name has been obtained, the method *_find_model* is called, searching for the most recent trained model file for that subject, supported by the *calibration_data* folder. If no matching file is found within the folder, the function returns *None*. Otherwise, it returns the path of the file found, and a corresponding error message invites the user to run a calibration session first, interrupting initialisation in the same way. If a model is found, it is loaded through *load_model*. At this point, the *EEGAcquisition* object is instantiated and the connection with the board is attempted. In the event of failure, an error message is displayed and the window sets *connection_failed = True*, consistently with the mechanism already described for *main.py*. If the connection succeeds, streaming is started and *self.n_samples* is calculated, corresponding to the number of samples within the analysis window, obtained by multiplying the sampling frequency by the duration in seconds defined by *WINDOW_SECONDS*. In parallel, the interface is built through the *_build_ui* function. The interface is organised as a control panel

containing a label showing the loaded subject, a status indicator showing the connection state with the board, and a prediction indicator displaying the direction of the classified motor imagery task, together with Start and Stop buttons, the latter allowing the session to be halted voluntarily. These two buttons are controlled by the `_start_control` and `_stop_control` functions, while the prediction loop itself is handled by the `_predict_and_move` function. The first step of this loop is an attempt to retrieve the latest `n_samples` from the buffer, through `self.acq.get_latest_data`. If the connection with the board is lost during this operation, an exception is handled that stops the timer, resets the state of the buttons, updates the status label in red, and displays an error message, the same error-handling pattern already seen in `calibration_gui.py`. If the acquisition succeeds, the mu band power is computed for each active EEG channel through `compute_band_power`, using exactly the same function and the same band constant (`MU_BAND`) employed during feature extraction in the calibration phase, a crucial detail, as it ensures that the features computed in real time are consistent, in terms of computation method, with those on which the classifier was trained. The resulting values are collected in `feature_vector`, then converted into a NumPy array and reshaped (`reshape(1, -1)`) to represent a single sample with one row and as many columns as there are features, as required by the prediction interface of *scikit-learn*. The classifier is then queried through `self.clf.predict(X)`, producing a numerically encoded prediction, which is converted back into its original textual label ("up", "down", "left" or "right") through `self.le.inverse_transform`. The prediction is displayed on screen in the dedicated label, and finally, through a series of *if/elif* conditions, it is translated into an actual cursor movement via `pyautogui.moveRel`, which moves the mouse relative to its current position by a number of pixels equal to `CURSOR_STEP`, in the direction corresponding to the predicted class (up, down, left or right). As with the previously described files, a `closeEvent` function is included to close the interface and effectively stop cursor control by the software, together with a standalone test block allowing the file to be used independently of the main interface. In conclusion, this file is the one that communicates most extensively with the other components, since its functioning depends directly on `calibration_gui.py`, and therefore indirectly on `classifier.py` and `feature_extraction.py`, and its effective use is possible only once the signal visualisation and calibration tasks, available through the other two functionalities in the `main.py` interface, have been successfully completed.



Images taken during the use of the cursor control functionality within the use of *cursor_gui.py* file with input an example subject code. From left to right: initial interface where it is asked the subject code, first window displayed before the control activation, updated window after the activation of the system. [\[F72\]](#) [\[F73\]](#) [\[F74\]](#)

4.2.2 Application Pipeline Founding

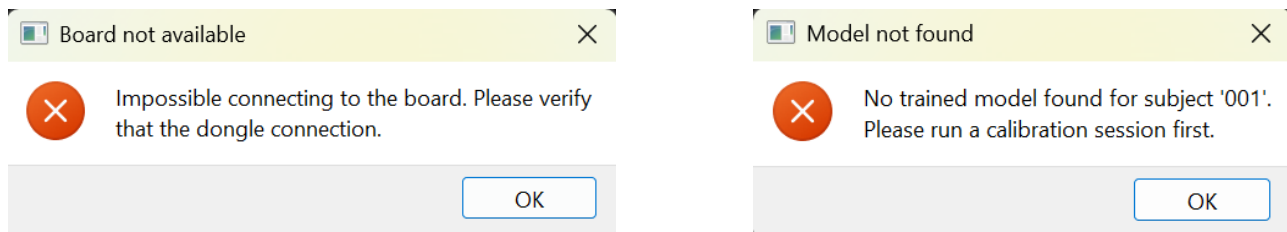
Before describing the actual results obtained through the use of the digital environment presented in the previous chapter, it is necessary to outline its intended method of use. Indeed, the environment created follows a specific pipeline, in which each step is essential to the successful completion of the next, and ultimately to the achievement of the final result. These steps begin with the proper use of the hardware components, preparing them for use with the application, in combination with the correct preparation of the software, including specific verification phases aimed at ensuring the correct functioning of the workflow. This chapter therefore aims to provide the theoretical and practical tools necessary to use the *NIA* environment for real-time user control, clarifying the use of the interfaces associated with each of the files previously described, and making clear the results the user should expect at the end of the experimentation.

The experimentation begins with the preparation of the hardware system, as mentioned above. Indeed, within the scripts of every digital component of *NIA_BCI_RealTimeControl*, a verification phase checks the connection status of the board. Before actually opening the application, the user must therefore assemble the system by connecting the board to the battery and the electrodes, while simultaneously positioning the electrodes at the relevant scalp locations. The Bluetooth connection of the devices must then be activated, since *BrainFlow* communicates with the board via Bluetooth. At this point, it is possible to proceed with the workflow by opening the application, which presents the user with the *main.py* interface. From here, the actual use of the application begins, and the hardware components must remain stable and unaltered, unless connection errors or electrode misplacement occur. Once these aspects have been verified, the user can select the first functionality of the pipeline within the application: the visualisation of the sampled EEG signal, accessed through the corresponding button on the main interface. This button connects the application to the scripts of *signal_visualisation.py*, which first checks the connection status by verifying both the Bluetooth state

of the device and the connection to the *OpenBCI Ganglion* board. Two outcomes are possible at this stage. The first is a connection failure, occurring when the Bluetooth feature is inactive or when communication between *BrainFlow* and the processing board fails; in this case, an error window is displayed, indicating that it is not possible to proceed, since communication with the board cannot be established. Conversely, if the verification process is successful, the signal visualisation GUI opens, allowing the user to observe the raw EEG signal from each electrode, displayed in four separate, dedicated graphs. Once the stability of the recorded signal has been confirmed, the next step involves the subsequent features of the application. This first step is not strictly related to the actual control of the cursor, but it is useful to the experimenter for verifying the correct configuration of the hardware components. After returning to the main interface, the user can therefore select the next feature, related to the calibration of the algorithm responsible for translating the signal into digital commands within the following stage of the pipeline. This refers to the functionality provided jointly by *calibration_gui.py*, *feature_extraction.py* and *classifier.py*, which together guide the user through the preparation for the final phase. The first window displayed in this case requests the subject code, a crucial step since every subsequent action will be referenced to the code entered. Based on this code, the system searches for the calibration data associated with the relevant subject, in order to continuously update the data concerning that user. As with the previous files, the connection status with the board is also verified here, leading to the same two possible outcomes described above. In the positive case, the calibration session begins with the opening of the corresponding window, in which the application presents the user with specific motor imagery tasks, following a logic similar to that used during the training session of *NIA_BCI_ProcessingAndTraining*. Once the calibration session is complete, the data are saved within the *calibration_data* folder, as explained in the previous chapter, and can subsequently be analysed by the experimenter whenever necessary. Once a sufficient number of calibration sessions have been completed to achieve an adequately reliable translation of motor imagery tasks into digital commands, the final phase can begin: the cursor control session. After selecting the corresponding button on the main interface and entering the correct subject code in the first window displayed, a crucial step, since cursor control will be based on the calibration data associated with the subject code entered, the cursor control window becomes available. This interface, implemented within *cursor_gui.py*, essentially consists of two buttons, used to start and stop cursor control. Once the start button is selected, cursor control becomes possible through motor imagery tasks, processed and translated on the basis of the data sampled during the calibration phase. These motor imagery tasks are directly converted into *up/down/left/right* commands through real-time analysis of the neural events sampled by the hardware and processed within the application. Each predicted cursor movement is also displayed on the current GUI, providing direct feedback to the user

and helping them understand how their neural activity has been translated into a command. At the same time, in order to ensure the safety of the system, cursor control through other peripherals, such as the mouse or trackpad, remains available at all times. Once the user decides that the control session should end, it is necessary to press the stop button, which returns full cursor control to the original peripherals.

In conclusion, the result is a stable and practical environment that covers the four pillars of brain-computer interface technology: *recording*, *analysis*, *calibration* and *control*. Each step can be repeated before proceeding to the next, allowing the user to verify possible improvements or simply become familiar with each mechanism of the application, from signal visualisation to the actual control of motor imagery tasks and, consequently, of the cursor.



From left to right: Error window in case of anomalies during the verification of the connection with the hardware system, Error window in case of anomalies during the selection of the desired subject for the cursor control feature. [\[F75\]](#) [\[F76\]](#)

4.3 Final Experimental Procedure

The purpose of developing a standalone application is not limited to defining the individual elements and the relationships underlying their operation. Indeed, in order to verify this part of the *NIA* environment as well, it is necessary to establish a proper foundation upon which the experimentation can take place. This process involves the exportation of the validated application, together with the use of the resulting output files, so as to carry out the experimentation phase that concludes the entire pipeline of the project. This chapter therefore explains the methods of use of this part of the environment, with the aim of demonstrating its correct use as an environment that can be freely deployed without any constraints related to digital or computational compatibility. Furthermore, it describes the use made by the end user and by the experimenter, in order to illustrate the collaborative environment achieved through the realisation of *NIA_BCI_RealTimeControl*.

4.3.1 Individual Methods of Use

Every standalone application is created with a specific purpose, which the collaboration between its sub-elements is intended to achieve. After creating the individual files that form the basis of the newly developed software, and after establishing the operating logic and the relationships between the elements, as described in the previous chapters, it becomes necessary to create an environment that is effectively executable not only by the experimenter on their own device, but also usable by the end user without constraints. Indeed, the first step of the experimentation phase within the *NIA_BCI_RealTimeControl* section consists in establishing these fundamentals, upon which the subsequent steps of the complete experimentation can then be carried out. In this way, the experimentation and verification of the system began correctly, with the aim of also verifying the compatibility between the end user and the functionalities provided by the application. Within this final part of the project, the consequences of use by a specific subject were studied, namely the same subject previously used as an example during the *NIA_BCI_ProcessingAndTraining* section. The subject involved is identified as ID 001, and the reasoning behind this choice is to create a continuous workflow, showing how the two sections of the macro-environment can cover every purpose envisaged at the outset, prior to the creation of the entire system.

This first step was carried out solely by the experimenter, whose main focus was to establish the standalone application as an executable, properly usable outside the *VSC* software, in order to prepare an application that could also be run on the patient's own device. Since this part of *NIA* was developed with Brainflow and built in Python 3.11, the appropriate tool for creating the standalone executable application was *PyInstaller*.^[54] Therefore, after installing this software through the *cmd* terminal and the *VSC* terminal, the following pipeline was followed: *cd + Desired Pathway → Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser → .\bci_env\Scripts\Activate.ps1 → python -m pip install pyinstaller → python -m pip show pyinstaller → python -m PyInstaller --onefile .\main.py*. Following this workflow, the experimenter obtained the *.spec*^[55] file, containing every detail of the exported application, and the *dist* folder, containing the *.exe*^[56] file. The deployment of the application was therefore made possible simply by opening the *main.exe* file, which, after briefly opening the *cmd* window in the background, brings the end user to the main interface of *NIA_BCI_RealTimeControl*. From this window, it was then possible to continue with the experimentation.

Next, after establishing the correct digital environment for the BCI system, it was necessary to prepare the hardware component, in the same manner as in *NIA_BCI_ProcessingAndTraining*, by helping the patient to wear the EEG cap with the relevant electrodes connected to the OpenBCI Ganglion board

through the snap cables. In this case, however, the pipeline proceeded with a different approach compared to the first section. Indeed, after connecting the USB connector to the laptop, it was necessary to activate the *Bluetooth* connection, since communication between *Brainflow* and the *OpenBCI instrumentation* relies on this type of wireless connection. The verification of the correct Bluetooth connection with the processing board was then carried out during the deployment of the first functionality.

Every command within this section of the environment operates inside the created environment itself, and no external software is required, unlike during the preliminary phases of *NIA_BCI_ProcessingAndTraining*. The next step therefore consists in opening the *Signal Visualisation* feature by pressing the relevant button on screen. In this way, the experimenter and the end user can access the window displaying the signal sampled at the level of the four electrodes (again, Cz, C1, C3, FC3), and thus verify the stability of the hardware connection and the quality of the forthcoming input signal. In parallel, this step also performs the Bluetooth connection check, updating the status message at the bottom of the main interface.

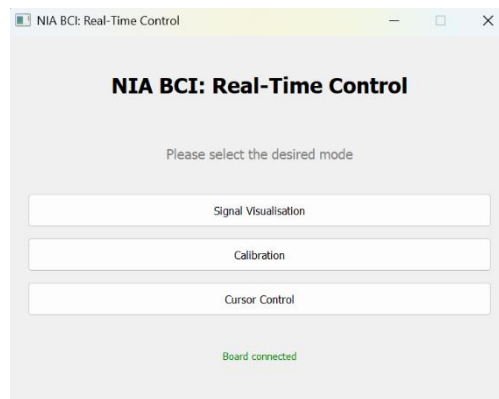
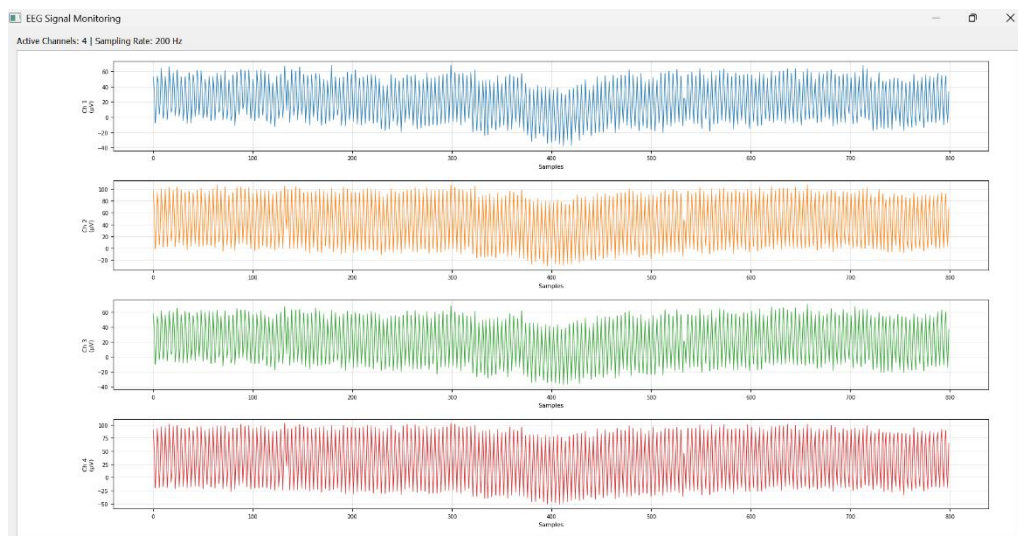
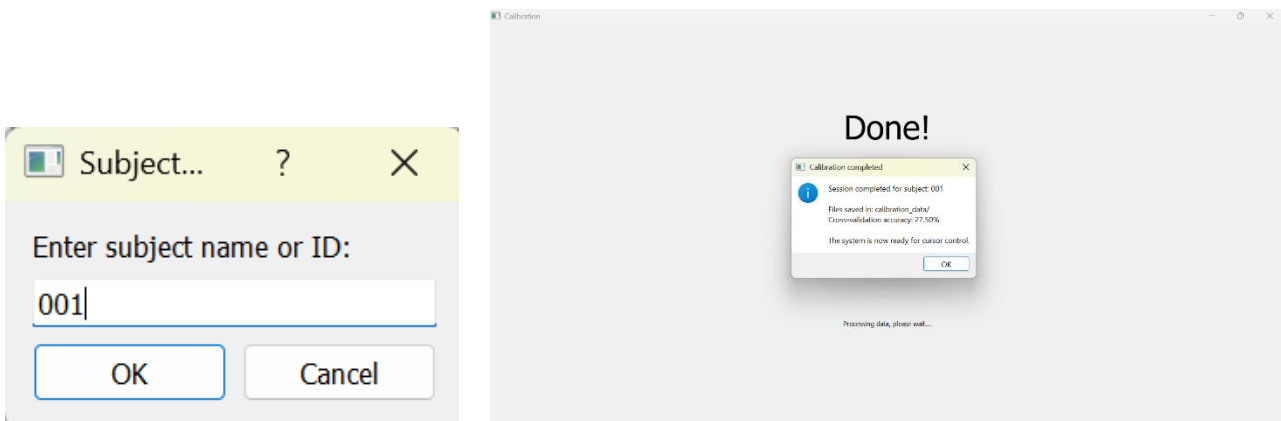


Image of *main.py* interface launched and updated based on experimentation conditions. [\[F77\]](#)



Screenshot taken during the use of the *Signal Visualisation* feature within experimentation conditions. [\[F78\]](#)

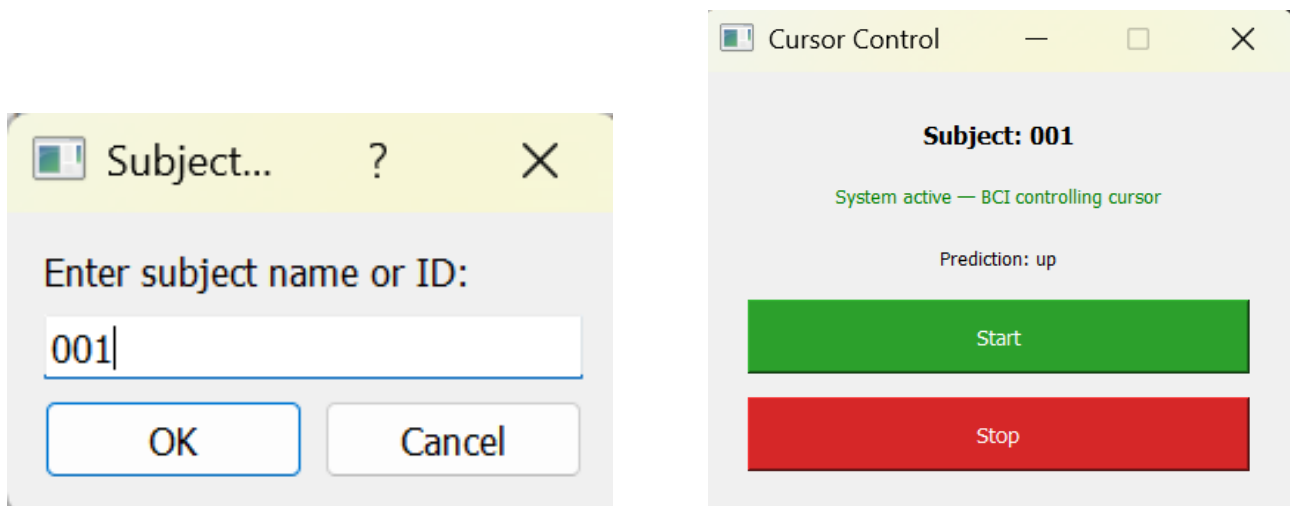
The next step of the experimentation began once the experimenter had fully verified the signal quality through the aforementioned functionality, and had ensured the stability of the hardware components and connection by checking the status message within the main interface. These aspects are crucial, since the next command literally establishes the basis upon which the real-time control will take place. Indeed, after closing the *Signal Visualisation* functionality, the experimenter proceeded with the pipeline by starting the *Calibration session*. The first step within this phase consisted in selecting the subject code of the patient currently using the application. After completing the requested field in the relevant opening window, the calibration session began. The pattern provided by the algorithm followed a similar structure to that of the Training session, but with a more practical and functional approach. Indeed, in the second window, the calibration session displayed a sequence of arrows representing the *Up/Right/Down/Left* directions, though with a shorter interval between each other and a quasi-random pattern, alternated with short resting periods. In this way, it was possible to calibrate the data in a more realistic manner, so as to provide the classification algorithm with a dataset capable of producing a more accurate transduction model. During this phase, the experimenter assists the user by helping him complete it, since, upon its conclusion, the output files become crucial for the experimenter. Indeed, within the *Calibration_data* folder, the files *calibration_001_20260703_101519_X.npy* and *calibration_001_20260703_101519_Y.npy* were now present, both essential for the transduction phase. The experimenter's principal duty was therefore to check for the presence of these files, and also to verify their correct nature and quality.



From left to right: Subject ID Code Interface after the selection of Calibration functionality, final Calibration window. [\[F79\]](#) [\[F80\]](#)

In conclusion, the final step of the experimentation took place at the last functionality of the *NIA_BCI_RealTimeControl* application. Indeed, after completing the calibration session a sufficient number of times, so as to provide the final algorithm with as clean a dataset as possible, and also to familiarise the patient with the mechanism underlying the calibration session, it was possible to try the cursor control feature. As with the previous command, selecting the relevant button leads to the window where the subject code is requested. This step is crucial, since the subsequent phases base

the transduction on the calibration dataset directly associated with the inserted subject code, and it was therefore necessary that the transduction methods be linked to the correct dataset corresponding to the subject currently using the BCI system. The second, and final, window of the application consists of the control interface, where the experimenter and the user can start and end the control session simply by clicking the *start/stop* buttons. The role of the experimenter in this case was to control the cursor with an external peripheral, so as to ensure a safety check in the event of *malfunctions* or *mis transduction* of the system, which could compromise the start or end of the control sessions. Therefore, after selecting the start button, the experimenter granted the patient total, supervised control of the digital device. The end user then controlled the on-screen cursor through the motor imagery tasks reinforced during the calibration session. The preliminary selection of the on-screen elements was carried out in collaboration with the experimenter, who continued to ensure safety by controlling the external peripheral when necessary. Furthermore, on this window it was possible to monitor the algorithm's decision-making and transduction process, since the prediction indicator at the centre of the window displayed the movement predicted by the software, thereby making the patient and the experimenter aware of the current movement transduced by the software and ensuring the correct workflow of the application. Finally, when desired, the session was stopped either by the end user, when able to do so, or by the experimenter, by simply moving the cursor onto the stop button and pressing it.



From left to right: Subject ID Code Interface after the selection of Cursor Control functionality, Active Cursor Control feature. [\[F81\]](#) [\[F82\]](#)

4.3.2 Results from the Collective Environment

The completion of the experimentation within NIA for Real-Time Control raises the need to understand the possible interaction between this sub-environment and the other parts that compose the entire project. Indeed, as mentioned before, the two sections are perfectly compatible and complementary, and it was therefore necessary to carry out an experimentation phase in which it is

possible to highlight the strong points arising when the two are used together at the same time. The following phase does not yield concrete and tangible results, such as graphs or datasets that can be analysed as done in the previous chapters. Instead, this part of the project aims to make the reader understand how the true core of the project also lay in creating a multi-software environment, in which complementary functionalities can interact to form a stable environment.

First of all, one of the initial purposes of this interactional use was the verification of signal quality. The *Displaying Raw EEG Signal* feature was implemented in both applications, in order to identify any similarities or discrepancies between the behaviour of the two. However, they are not perfectly identical, since the aim was not to create a perfect overlap of duties, but rather to introduce specific, subtle differences. More specifically, taking the experimentation conducted on subject 001 as an example, the inter-application workflow foresaw the use of the *NIA_BCI_ProcessingAndTraining* Displaying Raw Signal functionality first, since it is directly linked to the recording made through the *OpenBCI GUI*, and is therefore useful for verifying whether any anomalies arise during the importation of data into the *NIA_BCI_ProcessingAndTraining* application. In parallel, the Displaying Raw Signal functionality within *NIA_BCI_RealTimeControl* serves the purpose of verifying whether the signal conditions match those analysed during the use of *NIA_BCI_ProcessingAndTraining*. Moreover, the feature within the BrainFlow application operates in real time, making it possible to compare the current recording being performed by BrainFlow with the recording made during the use of the OpenBCI GUI and analysed in the MATLAB application. In this way, a proper logical bridge is created between the two sections of the environment, helping to maintain stability across the various experimentation phases.

Then, during the experimentation, it is crucial to connect two further features of the two applications, which in this case are not related to the same purpose as those described above. Here, the two functionalities in question are *EEGLAB* and *Calibration*, and their connection follows logically. Indeed, the analysis that the experimenter can perform within the MATLAB application is crucial for establishing the criteria underlying the classifier, which forms the basis of the calibration methods later used during the actual control phase within the BrainFlow application. Extracting the truly meaningful neural features during the analysis phase through EEGLAB is indeed essential for understanding how to isolate and transduce *ERD* phenomena and the EEG band relevant to the *LDA* processing phase. Therefore, carrying out a clear and well-verified EEGLAB phase indirectly makes it possible to achieve more accurate and precise cursor control by the end user within *NIA_BCI_RealTimeControl*.

Furthermore, linked to this *NIA_BCI_RealTimeControl* feature, its relationship with the *Training* sessions designed within *NIA_BCI_ProcessingAndTraining* is equally crucial. As mentioned before, both sessions apply the same criterion of dividing possible motor imagery actions into *up/right/down/left* directions, but the stimuli presented to the end user during the two sessions differ in specific details. During the use of the *Training* feature within the *MATLAB* application, the motor imagery tasks follow a fixed pattern that repeats itself throughout the entire six-minute session. In this way, the patient can gradually become comfortable with a neural mechanism that may initially seem difficult, but which must be as reliable as possible before the next phase. Then, during the use of *NIA_BCI_RealTimeControl*, the *calibration session*, while following the same criterion for dividing motor imagery tasks, presents the patient with a semi-random pattern of directions, in order to make the back-end algorithm more robust to real-life conditions. However, this session can only be successfully carried out once the training session has been completed, since it is only there that the patient can gain real control over the neural mechanism underlying motor imagery tasks.

In conclusion, the applications are also linked in a more general sense, beyond the level of the individual features described in the previous paragraphs. Indeed, speaking more generally, during the experimentation, the use of each application is considered complete only once all of its internal steps have been completed, and the failure of one step can be resolved by returning to the preceding one, in order to identify the error responsible for it. This applies not only to the individual features but also to the applications as a whole. Indeed, in the event of failure during cursor control provided by *NIA_BCI_RealTimeControl*, despite the successful completion of the calibration and Displaying Raw Signal features, the experimenter can return to the *NIA_BCI_ProcessingAndTraining* application, in order to determine whether the problem originates from preliminary conditions that can only be properly analysed through that application. In this way, every section of the overall environment has a sort of fallback procedure, allowing the process to be restarted in order to fully achieve the subsequent features, ultimately leading to the effective completion of the overall use of NIA.

CONCLUSION

5. Achievements and Developments

The project brings to light results whose analysis is worthwhile in order to understand the true potential and capabilities of the concepts conceived and physically implemented during the development and experimentation phases. The *hardware components* used and the *digital tools* implemented during development highlight outcomes that can be analysed from both a technical and a clinical point of view. Indeed, the *BCI* systems, as described in the experimentation section, demonstrate how the expected control and processing outcomes were successfully achieved, showing how the rationale behind the combination of specific components makes the systems stable and usable by both the end user and the experimenter. In parallel, the *software* and *programs* employed demonstrate how important a stable foundation is for the physical system. The digital environment further highlights the key role played by the collaboration between the end user and the experimenter. Then, from a multidisciplinary perspective, it is worth highlighting the potential of their *clinical application*, based on the medical aspects mentioned at the outset. The adaptation to specific parts of the human organism becomes real and tangible through the analysis of the results produced at the output of the digital environment. In conclusion, no result can be analysed independently, since each one must be linked to the *future* and to the respective improvements that could be applied to upgrade the systems and enhance the value of the expected results. This concluding chapter will therefore briefly analyse the true meaning of the results obtained during the development and experimentation phases, in order to make the real significance of the data and information presented throughout the previous chapters more understandable. Moreover, future improvements will be discussed, since every technology is always linked to areas where future studies can bring further strong points.

5.1 Clinical and Technical Outcomes

The considerations that can be highlighted from the outcomes of the described project can be analysed mainly from two points of view. The first is the *clinical* one, where it is necessary to reflect on the actual methods of use applied to real-life situations and contexts, in order to understand whether the designed project truly makes a difference for the targeted population. On the other hand, the consequences can certainly also be analysed from a *technical* and *engineering* point of view, since the tools involved and the methodologies used derive directly from the study and optimisation of computational, digital and analogue systems, where the experimenter's role is to find a common ground between the different technical fields involved in the development of the project.

BCIs are, in fact, probably by now one of the most multidisciplinary fields of our time, and the ability to understand both the optimisation and interdisciplinary aspects is probably the key strength needed to develop meaningful and revolutionary interfaces. Indeed, linked to this theme, the first outcome that can be highlighted is the project's potential to bring together two distant fields: the *medical* one and the *engineering* one. Being able to create a common environment, versatile and adaptable to specific individual contexts and characteristics, in which *neurology*, *computer science*, electronics and *biomedical sciences* converge, is essential for BCIs, and is probably the strongest point for advancing knowledge on the engineering side of this technology. In parallel, knowing how to apply this multidisciplinary approach to a specific part of the human organism, one as important as the *CNS*, is the step that truly makes the difference. The ability to apply the system to particular phenomena and biological activity is the pathway that must be followed in order to create a system that is truly adaptable and versatile. In relation to this project, the field of human application is therefore the M1 and the EEG signal recorded at its level, where the system has proven able to extract the relevant information and make it useful for the user experience. More specifically, the system demonstrates how focusing the application on a specific neural activity and brain region is crucial if the aim is to isolate the desired phenomenon and make it truly useful for both the transduction and control phases. Focusing specifically on M1 furthermore allows for the resolution of every pathological condition in which its functioning remains unaffected. Creating a system with this type of focus, one specialised in the transduction of movement starting from its actual source, could be the key to isolating other forms of pathological conditions related to different starting points, from which it becomes possible to develop systems aimed at overcoming the consequences of such conditions. Establishing the system therefore means establishing a route to overcoming these limitations, enabling the human body to express itself through a different approach, in which the chosen technology plays a key role in effectively transducing the true will of the patient. In relation to this point, the technological results, together with the clinical outcomes, can be analysed in order to understand the significance of what was used during development and experimentation. More specifically, the use of *open-source* tools and devices became another strong point of the project. The open-source nature allows for the kind of versatility and personalisation that is crucial for building a system tailored to the patient's characteristics. The key aspect was adapting and adjusting the *hardware components* in order to find the correct configuration for each situation and context, an action made possible only through the use of open-source, and therefore adaptable, hardware components. In parallel, on the *digital* side, the use of open-source tools and programs made it possible to create genuine collaboration between the experimenter and the patient, since, in order to produce meaningful results at the output of each sub-

environment created, it is crucial to understand which front-end and back-end features need to be changed or adapted.

The constant modification of the interfaces and the ongoing implementation of various tools across platforms led to the creation of a multi-functional macro-environment, one that can be upgraded by changing internal details or parameters.

5.2 Future Improvements

The true potential of the current system is not limited to the prototype stage. Its true potential extends far beyond the current state, and the description of the project has certainly taken into consideration the possible improvements that could be applied to specific parts of the system. Indeed, despite the results and outcomes achieved and highlighted in the previous chapters, the developed system offers several starting points from which further development phases could begin, in order to improve specific details or macro-sections of the project.

First of all, starting from the *hardware set* that was used, this mainly reflects the current stage of open-source electroencephalographic sampling devices, and improvements can begin with the components in direct contact with the patient. Indeed, BCIs must be as *comfortable* as possible, since the truly meaningful objective currently being pursued is finding a solution suitable for long-term use. Naturally, the use of a conventional EEG cap is ideal for the idea of keeping the system worn for several hours or days, outside of training sessions. Improvements in this part of the project point towards the development of a practical wearable structure, non-invasive and adaptable to the specific anatomical parameters of the patient. This structure must cover the precise area of interest within M1, and must focus on the correct side needed to cover the dominant hand and arm representation in the sensorimotor cortex. Moreover, the structure should be made from innovative materials, so as to be hypoallergenic and biocompatible with the patient's scalp. The next hardware component that can be improved is the electrodes. Future developments point towards the use of *biopolymer-* and *fabric-based hydrogels* integrated with multiple *microarray electrodes*, capable of sampling the EEG signal more accurately and precisely, in order to provide a better signal quality. Furthermore, the communication between the processing board and the relevant digital device must be enhanced, in order to connect the system with other types of digital devices as well, not only laptops but also, for example, *smartphones* and *tablets*.

On the *digital side*, however, the improvements extend far beyond the current multi-software environment created within the described project. Despite the stability provided by the designed environment, which demonstrates that it is possible to achieve the pillars of BCIs through the collaboration between different types of open-source applications and programs, future developments point towards the creation of a *unified digital environment*. Unifying the different sub-environments, and therefore the functionalities covered by specific software, does not mean limiting the possibility of further extending the development of the digital side of BCIs. On the contrary, the creation of a single environment, in which the various existing functionalities remain available, makes it possible to establish a more stable and practical collaboration between the patient and the experimenter. Being situated within a single environment can strengthen communication and the interpretation of the data sampled during trials, thereby facilitating the experimenter during the signal analysis procedure, the physician during patient monitoring, and the end-user in understanding the real capabilities achievable during real-life use of the system.

In conclusion, the potential of this type of neurotechnology is unlimited. The interaction with such a fascinating and complex part of the human organism as the CNS is what gives meaning to the study of BCIs. The fact that so many details can still be improved, not only within this project but within every project that addresses *BCIs* and *human-machine* interaction, is proof of their cutting-edge importance. The true next steps therefore do not depend on the type of materials or digital mechanisms that could be implemented, but rather on how and why they are chosen, and on how they will ultimately be optimised and truly implemented in order to achieve the objective of understanding how to empower what makes us human: the *Nervous System*.

References and Figures

- [L1] Original Logo of NIA Environment
- [L2] Original Logo of NIA for Processing and Training sub-environment
- [L3] Original logo of NIA for Real-Time control sub-environment
- [1] Caiado F., Ukolov A. The history, current state and future possibilities of the non-invasive brain computer interfaces. Elsevier B.V. 2025. 25.
- [2] Chen S., Chen M., Wang X., Liu X., Liu B., Ming D. Brain-computer interfaces in 2023–2024. John Wiley and Sons Inc. 2025. 3(1).
- [3] Ding Y., Udompanyawit C., Zhang Y., He B. EEG-based brain-computer interface enables real-time robotic hand control at individual finger level. Nature Research. 2025. 16(1).
- [4] Lee J.Y., Lee S., Mishra A., Yan X., McMahan B., Gaisford B., Kobashigawa C., Qu M., Xie C., Kao J.C. Brain-computer interface control with artificial intelligence copilots. Nature Research. 2025. 7(9): 1510-1523.
- [5] Cerone G.L., Gazzoni M., Knaflitz M. Elettrodi per il prelievo di biopotenziali. In: Levrotto & bella, ed. Strumentazione Biomedica. Libreria Editrice Universitaria di Gaulini Elisabetta S.a.s.; 2023:195-207
- [6] Cerone G.L., Gazzoni M., Knaflitz M. Sistemi di prelievo per EEG. In: Levrotto & bella, ed. Strumentazione Biomedica. Libreria Editrice Universitaria di Gaulini Elisabetta S.a.s.; 2023:336
- [7] Fazio C., Loeb C. Organizzazione del Livello Spino-Muscolare o dell'Unità Motoria. In: Società Editrice Universo – Roma, ed. Neurologia 1. 3rd ed. 1982:12-18.
- [8] Fazio C., Loeb C. Organizzazione Motoria piramidale: Dati Anatomici. In: Società Editrice Universo – Roma, ed. Neurologia 1. 3rd ed. 1982:18-28.
- [9] Bhattacharjee S., Kashyap R., Abualait T., Annabel Chen S. H., Yoo W. K., Bashir S. The Role of Primary Motor Cortex: More Than Movement Execution. Journal of Motor Behavior. 2021; 53(2): 258–274.
- [10] Wing-kin T., Tong W., Qi Z., Edward K., Zhi Y. Human motor decoding from neural signals: a review. BMC biomed eng. 2019; 1(22).
- [11] Shinji K., Donna S.H., Peter L.S. Muscle and Movement Representations in the Primary Motor Cortex. Science. 1999;285(5436):2136-2139.
- [12] Fazio C., Loeb C. Elettroencefalografia (E.E.G). In: Società Editrice Universo – Roma, ed. Neurologia 1. 3rd ed. 1982:318-328.
- [13] Cerone G.L., Gazzoni M., Knaflitz M. Generazione del Segnale Elettroencefalografico e Sue caratteristiche. In: Levrotto & bella, ed. Strumentazione Biomedica. Libreria Editrice Universitaria di Gaulini Elisabetta S.a.s.; 2023:335-336.
- [14] Casson A.J., Yates D.C., Smith S.J.M., Duncan J.S., Rodriguez-Villegas E. Wearable Electroencephalography. IEEE Engineering in Medicine and Biology Magazine. 2010. 29(3):44-56.
- [15] Cerone G.L., Gazzoni M., Knaflitz M. Posizionamento degli Elettrodi – Standard 10 – 20: Posizionamento degli elettrodi di prelievo. In: Levrotto & bella, ed. Strumentazione Biomedica. Libreria Editrice Universitaria di Gaulini Elisabetta S.a.s.; 2023:338-339.
- [16] Michel J.A.M. van Putten. Exitable cells and Action potentials: Action potential generation. In: Joachim H. Nagel, ed. Essentials of Neurophysiology. Springer; 2025:21-27.
- [17] Michel J.A.M. van Putten. Electroencephalography: Cellular Basis of EEG Activity. In: Joachim H. Nagel, ed. Essentials of Neurophysiology. Springer; 2025:148-153.
- [18] Fazio C., Loeb C. Elettroencefalografia (E.E.G): Le Figure Elementari dell'EEG. In: Società Editrice Universo – Roma, ed. Neurologia 1. 3rd ed. 1982:320-322.
- [19] Saeid S., Jonathon A.C. BCI-Related EEG Components: ERD and ERS. In: The Atrium, ed. EEG Signal Processing and Machine Learning. 2nd ed. John Wiley & Sons. 2022:588-594.
- [20] OpenBCI. Ganglion Board. OpenBCI Documentation. Updated Sep 8, 2025. <https://docs.openbci.com/Ganglion/GanglionLanding/>
- [21] OpenBCI. Electrode Guide. OpenBCI Documentation. Updated Jul 15, 2025. <https://docs.openbci.com/AddOns/Electrodes/ElectrodesLanding/>
- [22] OpenBCI. EEG Polymer Electrodes. OpenBCI Products. Updated 2026. <https://shop.openbci.com/products/ee-polymer-electrodes>
- [23] OpenBCI. Earclip Electrode. OpenBCI Products. Updated 2026. <https://shop.openbci.com/products/earclip-electrode>
- [24] OpenBCI. EMG ECG Snap Electrode Cables. OpenBCI Products. Updated 2026. <https://shop.openbci.com/products/emg-ecg-snap-electrode-cables>
- [25] OpenBCI. The OpenBCI GUI. OpenBCI Documentation. Updated Jan 6, 2026. <https://docs.openbci.com/Software/OpenBCISoftware/GUIDocs/>
- [26] MathWorks. MATLAB. MathWorks Products. Updated 2026. <https://it.mathworks.com/products/matlab/capabilities.html>
- [27] MathWorks. MATLAB App Designer. MathWorks Products. Updated 2026. <https://it.mathworks.com/products/matlab/app-designer.html>
- [28] Delorme A & Makeig S (2004) EEGLAB: an open-source toolbox for analysis of single-trial EEG dynamics, Journal of Neuroscience Methods 134:9-21. [PDF](#).
- [29] Brainflow. Welcome to Brainflow's Documentation. Brainflow Docs. Updated 2019. <https://brainflow.readthedocs.io/en/stable/index.html>
- [30] Visual Studio Code. Get started with Visual Studio Code. Visual Studio Code Documentation. Updated Aug 7, 2026. <https://code.visualstudio.com/docs/getstarted/overview>
- [31] GitHub. Home. GitHub Home Menu. Updated 2026. <https://github.com/>
- [32] MathWorks. MATLAB Help Centre. Uigetfile Documentation. Updated 2026. <https://it.mathworks.com/help/matlab/ref/uigetfile.html>
- [33] MathWorks. MATLAB Help Centre. If, elseif, else Documentation. Updated 2026. <https://it.mathworks.com/help/matlab/ref/if.html>
- [34] MathWorks. MATLAB Help Centre. Live Code File Format (.m) Documentation. Updated 2026. https://it.mathworks.com/help/matlab/matlab_prog/plain-text-file-format-for-live-scripts.html
- [35] MathWorks. MATLAB Help Centre. assignin Documentation. Updated 2026. <https://it.mathworks.com/help/matlab/ref/assignin.html>
- [36] MathWorks. MATLAB Help Centre. fullfile Documentation. Updated 2026. <https://it.mathworks.com/help/matlab/ref/fullfile.html>

- [37] MathWorks. MATLAB Help Centre. addpath Documentation. Updated 2026. <https://it.mathworks.com/help/matlab/ref/addpath.html>
- [38] MathWorks. MATLAB Help Centre. sprintf Documentation. Updated 2026. <https://it.mathworks.com/help/matlab/ref/string.sprintf.html>
- [39] MathWorks. MATLAB Help Centre. timer Documentation. Updated 2026. <https://it.mathworks.com/help/matlab/ref/timer.html>
- [40] MathWorks. MATLAB Help Centre. Switch-case Documentation. Updated 2026. <https://it.mathworks.com/help/matlab/ref/switch.html>
- [41] MathWorks. MATLAB Help Centre. Set Breakpoints Documentation. Updated 2026. https://it.mathworks.com/help/matlab/matlab_prog/set-breakpoints.html
- [42] MathWorks. MATLAB Help Centre. Simulation Stepping Options Documentation. Updated 2026. <https://it.mathworks.com/help/simulink/slref/simulationsteppingoptions.html>
- [43] MathWorks. MATLAB Help Centre. uialert Documentation. Updated 2026. <https://it.mathworks.com/help/matlab/ref/uialert.html>
- [44] MathWorks. MATLAB Help Centre. try,catch Documentation. Updated 2026. <https://it.mathworks.com/help/matlab/ref/try.html>
- [45] MathWorks. MATLAB Help Centre. Pause Documentation. Updated 2026. <https://it.mathworks.com/help/matlab/ref/pause.html>
- [46] MathWorks. MATLAB Help Centre. Standalone Applications Documentation. Updated 2026. <https://it.mathworks.com/help/compiler/standalone-applications.html>
- [47] Tvrdy P., Long J. Office of the Assistant Secretary for Research and Technology, Bureau of, Transportation Statistics, National Transportation Library, File Formats Dictionary. United States. Department of Transportation. National Transportation Library. 2024.
- [48] Microsoft. Window Server. Cmd. Updated Aug 16, 2025. <https://learn.microsoft.com/it-it/windows-server/administration/windows-commands/cmd>
- [49] NumPy. API Reference. numpy.lib.format. Updated 2026. <https://numpy.org/devdocs/reference/generated/numpy.lib.format.html>
- [50] Python. Documentation. py_compile — Compile Python source files. Updated Aug 27, 2026. https://docs.python.org/3/library/py_compile.html
- [51] Python. Documentation. pickle — Python object serialization. Updated Aug 27, 2026. <https://docs.python.org/3/library/pickle.html>
- [52] Python. Documentation. The import system. Updated Sep 9, 2026. <https://docs.python.org/3/reference/import.html>
- [53] Python. Documentation. Classes. Updated Sep 9, 2026. <https://docs.python.org/3/tutorial/classes.html>
- [54] PyInstaller. Home. PyInstaller Manual. Updated 2026. <https://pyinstaller.org/en/stable/>
- [55] PyInstaller. Home. Using Spec Files. Updated 2026. <https://pyinstaller.org/en/stable/spec-files.html>
- [56] Wikipedia. Home. .exe. Updated may 19, 2026. <https://en.wikipedia.org/wiki/.exe>
- [F1] CC BY-SA 2.1 JP <<https://creativecommons.org/licenses/by-sa/2.1/jp/deed.en>>, via Wikimedia Commons (left); 3D brain data is from Anatomography., CC BY-SA 2.1 JP <<https://creativecommons.org/licenses/by-sa/2.1/jp/deed.en>>, via Wikimedia Commons (right). BodyParts3D, © The Database center for Life Science licensed under CC Attribution-Share Alike 2.1 Japan: https://commons.wikimedia.org/wiki/File:Brodman_area_4_lateral.jpg; https://commons.wikimedia.org/wiki/File:Brodman_area_4_medial.jpg
- [F2] Anders Sandberg from Oxford, UK, CC BY 2.0<<https://creativecommons.org/licenses/by/2.0/>>, via Wikimedia Commons: [https://commons.wikimedia.org/wiki/File:Electroencephalography_\(EEG\)_-_FET09_Prague.jpg](https://commons.wikimedia.org/wiki/File:Electroencephalography_(EEG)_-_FET09_Prague.jpg)
- [F3] Andrii Cherninskyi, CC BY-SA 4.0 <<https://creativecommons.org/licenses/by-sa/4.0/>>, via Wikimedia Commons: https://commons.wikimedia.org/wiki/File:Human_EEG_with_prominent_alpha-rhythm.png
- [F4] Brylie Christopher Oxley, CC0, via Wikimedia Commons: https://commons.wikimedia.org/wiki/File:International_10-20_system_for_EEG-MCN.png
- [F5] OpenBCI. Ganglion Board. OpenBCI Documentation. Updated Sep 8, 2025. <https://docs.openbci.com/Ganglion/GanglionLanding/>; Content: OpenBCI Ganglion Overview and OpenBCI Ganglion Layers; License: <https://creativecommons.org/licenses/by-sa/4.0/>
- [F6] OpenBCI. Ganglion Board. OpenBCI Documentation. Updated Sep 8, 2025. <https://docs.openbci.com/Ganglion/GanglionLanding/>; Content: OpenBCI Ganglion Board Circuit Schematic; License: <https://creativecommons.org/licenses/by-sa/4.0/>
- [F7] OpenBCI. EEG Polymer Electrodes. OpenBCI Products. Updated 2026. <https://shop.openbci.com/products/eeg-polymer-electrodes>; Content: OpenBCI EEG Polymer Electrode; License: <https://creativecommons.org/licenses/by-sa/4.0/>
- [F8] OpenBCI. Earclip Electrode. OpenBCI Products. Updated 2026. <https://shop.openbci.com/products/earclip-electrode>; Content: OpenBCI Earclip Electrode; License: <https://creativecommons.org/licenses/by-sa/4.0/>
- [F9] OpenBCI. EMG ECG Snap Electrode Cables. OpenBCI Products. Updated 2026. <https://shop.openbci.com/products/emg-ecg-snap-electrode-cables>; Content: OpenBCI EMG-ECG Snap Cables; License: <https://creativecommons.org/licenses/by-sa/4.0/>
- [F10] OpenBCI Ganglion Board, Protective PVC Case, Battery, USB Connector. Updated Sep 9, 2026. Content: Original photo taken by the Author during the assembling process, the content is written inside the respective caption.
- [F11] OpenBCI Ganglion Board, Protective PVC Case, Battery, Snap Cables. Updated Sep 9, 2026. Content: Original photo taken by the Author during the assembling process, the content is written inside the respective caption.
- [F12] OpenBCI Ganglion Board, Protective PVC Case, Battery, Snap Cables, Dry Electrodes, Earclip Electrodes. Updated Sep 9, 2026. Content: Original photo taken by the Author during the assembling process, the content is written inside the respective caption.
- [F13] Snap Cables, Dry Electrodes, Earclip Electrodes, EEG Cap. Updated Sep 9, 2026. Content: Original photo taken by the Author during the assembling process, the content is written inside the respective caption.
- [F14] MathWorks. MATLAB App Designer User Interface: NIA_BCI_v1.mlapp. MathWorks Product. Updated Jul 19, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.
- [F15] MathWorks. MATLAB App Designer User Interface: Design View of DisplayRawData.mlapp. MathWorks Product. Updated Jul 21, 2026. Content: Original screenshot taken by

the Author during the design process, the content is written inside the respective caption.

[F16] MathWorks. MATLAB App Designer User Interface: Design View of TrainingSession.mlapp. MathWorks Product. Updated Jul 21, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F17] MathWorks. MATLAB App Designer User Interface: NIA_BCI_v1.mlapp. MathWorks Product. Updated Jul 22, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F18] MathWorks. MATLAB App Designer User Interface: DisplayRawData.mlapp. MathWorks Product. Updated Jul 22, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F19] MathWorks. MATLAB User Interface: EEGLAB. MathWorks Product. Updated Jul 22, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F20] MathWorks. MATLAB App Designer User Interface: TrainingSession.mlapp. MathWorks Product. Updated Jul 22, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F21] MathWorks. MATLAB App Designer User Interface: Launched NIA_BCI_v1.mlapp. MathWorks Product. Updated Jul 23, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F22] MathWorks. MATLAB App Designer User Interface: Launched and Updated NIA_BCI_v1.mlapp. MathWorks Product. Updated Jul 23, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F23] MathWorks. MATLAB App Designer User Interface: Launched DisplayRawData.mlapp. MathWorks Product. Updated Jul 23, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F24] MathWorks. MATLAB App Designer User Interface: Launched and Updated NIA_BCI_v1.mlapp. MathWorks Product. Updated Jul 23, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F25] MathWorks. MATLAB Interface: EEGLAB toolbox. MathWorks Product. Updated Jul 23, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F26] MathWorks. MATLAB Interface: Import data info window of EEGLAB toolbox. MathWorks Product. Updated Jul 23, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F27] MathWorks. MATLAB Interface: Launched and Updated NIA_BCI_v1.mlapp. MathWorks Product. Updated Jul 24, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F28] MathWorks. MATLAB App Designer User Interface: Launched and Updated TrainingSession.mlapp 1. MathWorks Product. Updated Jul 24, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F29] MathWorks. MATLAB App Designer User Interface: Launched and Updated TrainingSession.mlapp 2. MathWorks Product. Updated Jul 24, 2026. Content: Original screenshot

taken by the Author during the design process, the content is written inside the respective caption.

[F30] MathWorks. MATLAB App Designer User Interface: Launched and Updated TrainingSession.mlapp 3. MathWorks Product. Updated Jul 24, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F31] MathWorks. MATLAB App Designer User Interface: Launched Error Figure of NIA_BCI_v1.mlapp. MathWorks Product. Updated Jul 26, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F32] MathWorks. MATLAB App Designer User Interface: Launched Warning Figure related to DisplayRawData.mlapp. MathWorks Product. Updated Jul 26, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F33] MathWorks. MATLAB App Designer User Interface: Launched Warning Figure related to EEGLAB. MathWorks Product. Updated Jul 26, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F34] MathWorks. MATLAB App Designer User Interface: Launched Warning Figure related to TrainingSession.mlapp. MathWorks Product. Updated Jul 26, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F35] MathWorks. MATLAB App Designer User Interface: Launched Error Figure of DisplayRawData.mlapp 1. MathWorks Product. Updated Jul 26, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F36] MathWorks. MATLAB App Designer User Interface: Launched Error Figure of DisplayRawData.mlapp 2. MathWorks Product. Updated Jul 26, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F37] MathWorks. MATLAB App Designer User Interface: Launched Error Figure of NIA_BCI_v1.mlapp – EEGLAB 1. MathWorks Product. Updated Aug 8, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F38] MathWorks. MATLAB App Designer User Interface: Launched Error Figure of NIA_BCI_v1.mlapp – EEGLAB 2. MathWorks Product. Updated Aug 8, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F39] Microsoft Office. Excel. Subject Information Table. Microsoft Product. Updated Sep 12, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F40] Microsoft Office. Excel. Trial Information Table. Microsoft Product. Updated Sep 12, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F41] OpenBCI. Hardware and Related Instrumentation. Original Assembled Components. Updated Sep 13, 2026. Content: Original photo taken by the Author during the design process, the content is written inside the respective caption.

[F42] OpenBCI. OpenBCI GUI. Launched GUI. OpenBCI Product. Updated Sep 13, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F43] Windows. Python Application. Launched main.py Active Interface. Original Product. Updated Sep 13, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F44] MathWorks. MATLAB App Designer. Launched and Active DisplayRawData.mlapp Interface. MathWorks Product. Updated Sep 13, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F45] MathWorks. EEGLAB. Import Data Updated Interface. EEGLAB Product. Updated Sep 13, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F46] MathWorks. EEGLAB. Second Import Data Updated Interface. EEGLAB Product. Updated Sep 13, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F47] MathWorks. EEGLAB. Principal Updated Interface. EEGLAB Product. Updated Sep 13, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F48] MathWorks. EEGLAB. Channel Location in Edit. EEGLAB Product. Updated Sep 13, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F49] MathWorks. EEGLAB. Channel Data (scroll) in Plot. EEGLAB Product. Updated Sep 13, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F50] MathWorks. EEGLAB. Basic FIR Filter Input Interface. EEGLAB Product. Updated Sep 12, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F51] MathWorks. EEGLAB. Basic FIR Filter Graphs. EEGLAB Product. Updated Sep 12, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F52] MathWorks. EEGLAB. Inspect/Reject Data by Eye. EEGLAB Product. Updated Sep 13, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F53] MathWorks. EEGLAB. Independent Component Analysis Interface. EEGLAB Product. Updated Sep 12, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F54] MathWorks. EEGLAB. ICLabel Selection Interface. EEGLAB Product. Updated Sep 12, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F55] MathWorks. EEGLAB. Import Event Info Interface. EEGLAB Product. Updated Sep 12, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F56] MathWorks. EEGLAB. Remove Epoch Baseline Interface. EEGLAB Product. Updated Sep 12, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F57] MathWorks. EEGLAB. Dataset Info Interface. EEGLAB Product. Updated Sep 12, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F58] MathWorks. EEGLAB. Plot component time-frequency Interface. EEGLAB Product. Updated Sep 12, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F59] MathWorks. MATLAB App Designer User Interface. Launched and Updated TrainingSession.mlapp: Up Command. MathWorks Product. Updated Sep 12, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F60] MathWorks. MATLAB App Designer User Interface. Launched and Updated TrainingSession.mlapp: Right Command. MathWorks Product. Updated Sep 12, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F61] MathWorks. MATLAB App Designer User Interface. Launched and Updated TrainingSession.mlapp: Down Command. MathWorks Product. Updated Sep 12, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F62] MathWorks. MATLAB App Designer User Interface. Launched and Updated TrainingSession.mlapp: Left Command. MathWorks Product. Updated Sep 12, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F63] MathWorks. EEGLAB. ICLabel Graphs. EEGLAB Product. Updated Sep 13, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F64] MathWorks. EEGLAB. Channel Time-Frequency in Plot. EEGLAB Product. Updated Sep 13, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F65] MathWorks. EEGLAB. Channel Spectra and Maps in Plot. EEGLAB Product. Updated Sep 13, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F66] MathWorks. EEGLAB. Channel ERPs in Plot. EEGLAB Product. Updated Sep 13, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F67] Window OS. Python Application Interface: Launched Figure of main.py. Updated Sep 7, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F68] Window OS. Python Application Interface: Launched Figure of signal_visualisation.py. Updated Sep 7, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F69] Window OS. Python Application Interface: Launched Subject Code Sub-figure of calibration.py. Updated Sep 7, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F70] Window OS. Python Application Interface: Launched Figure of calibration.py with "right" command. Updated Sep 7, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F71] Window OS. Python Application Interface: Launched Figure of calibration.py with "left" command. Updated Sep 7, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F72] Window OS. Python Application Interface: Launched Subject Code Sub-figure of cursor_gui.py. Updated Sep 7, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F73] Window OS. Python Application Interface: Launched Subject Code Figure of cursor_gui.py with inactive system. Updated Sep 7, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F74] Window OS. Python Application Interface: Launched Subject Code Figure of `cursor_gui.py` with active system. Updated Sep 7, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F75] Window OS. Python Application Interface: Error Figure of `signal_visualisation.py`, `calibration_gui.py` and `cursor_gui.py` during hardware connection. Updated Sep 7, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F76] Window OS. Python Application Interface: Error Figure of `cursor_gui.py` during subject selection. Updated Sep 7, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F77] Windows OS. Python Application. Launched `signal_visualisation.py` Active Interface. Original Product. Updated Sep 13, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F78] Windows OS. Python Application. Launched and Active `signal_visualisation.py` Interface. Original Product. Updated

Sep 13, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F79] Windows OS. Python Application. Launched and Active Subject ID Window of Calibration. Original Product. Updated Sep 13, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F80] Windows OS. Python Application. Launched and Active `calibration_gui.py` Interface. Original Product. Updated Sep 13, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F81] Windows OS. Python Application. Launched and Active Subject ID Window of Cursor Control. Original Product. Updated Sep 13, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.

[F82] Windows OS. Python Application. Launched and Active `cursor_gui.py` Interface. Original Product. Updated Sep 13, 2026. Content: Original screenshot taken by the Author during the design process, the content is written inside the respective caption.